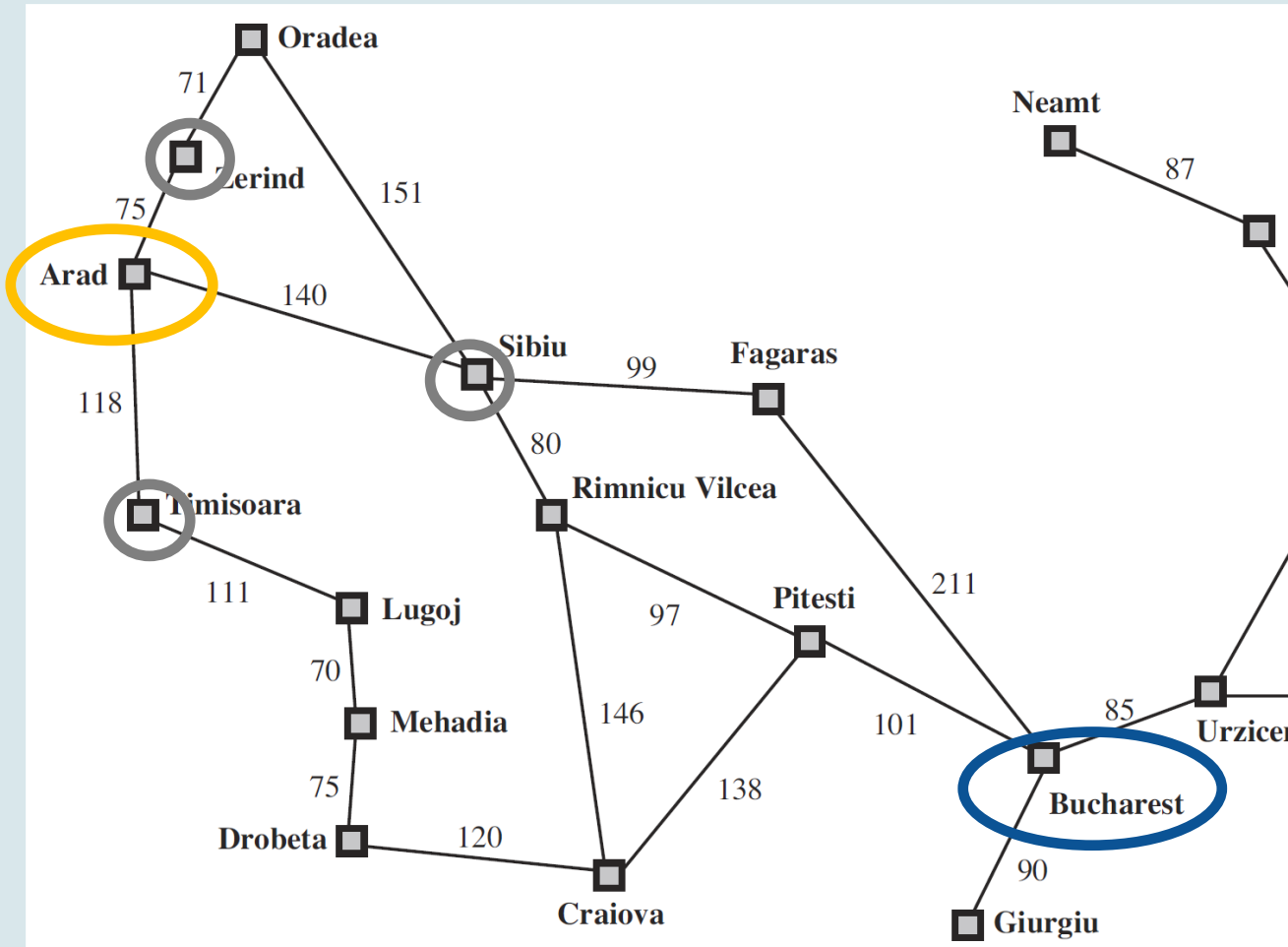
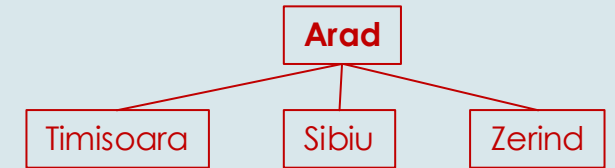


Breadth First Search

0

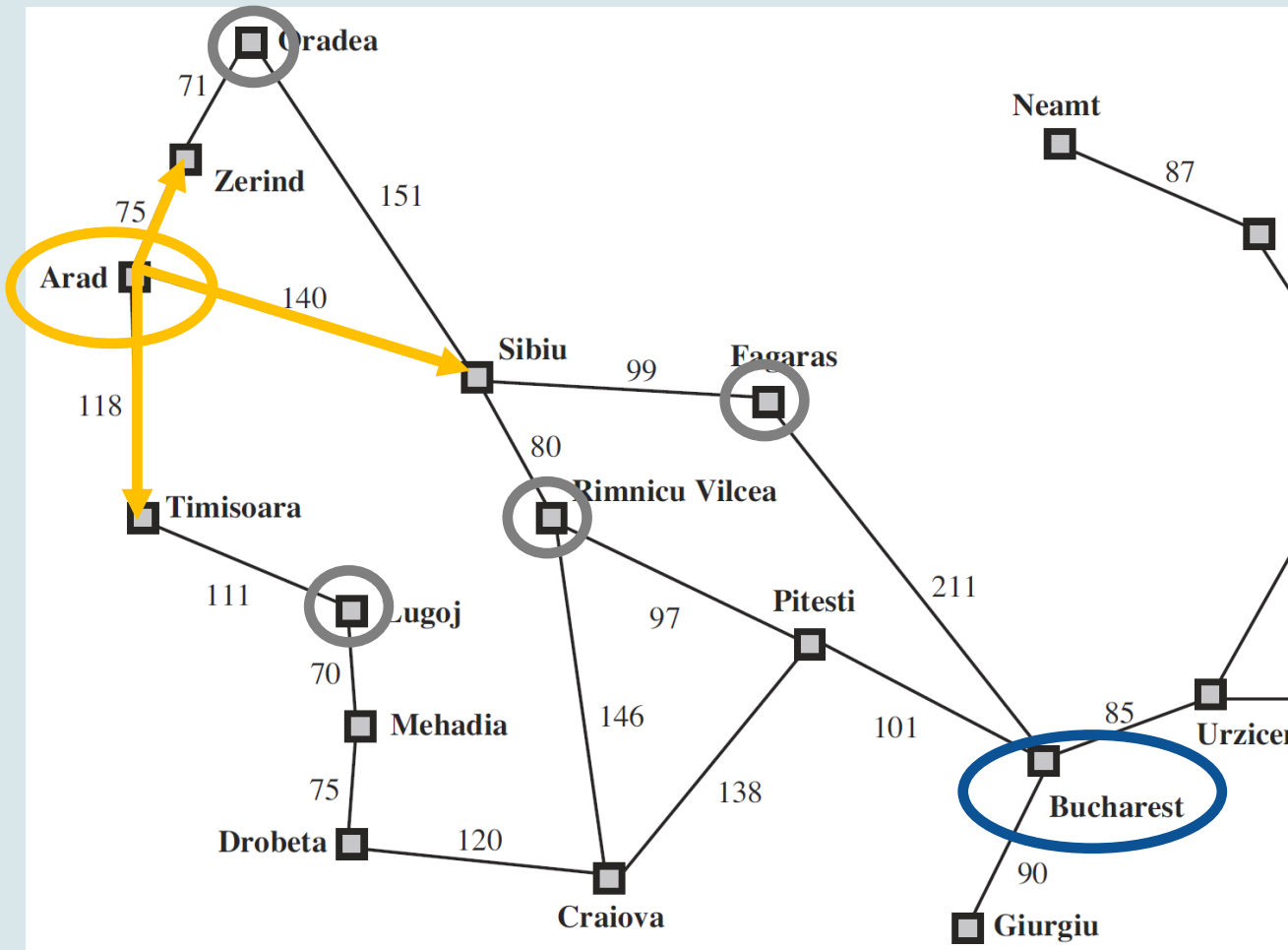


Search Tree

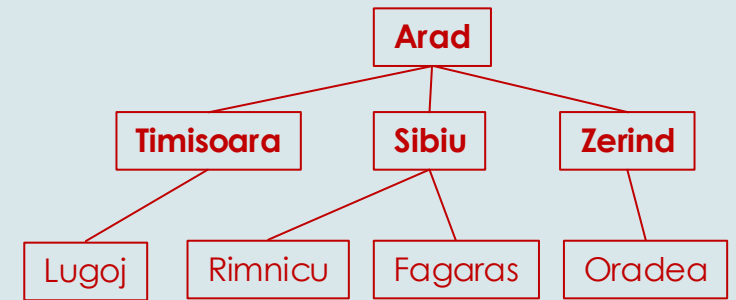


Breadth First Search

1

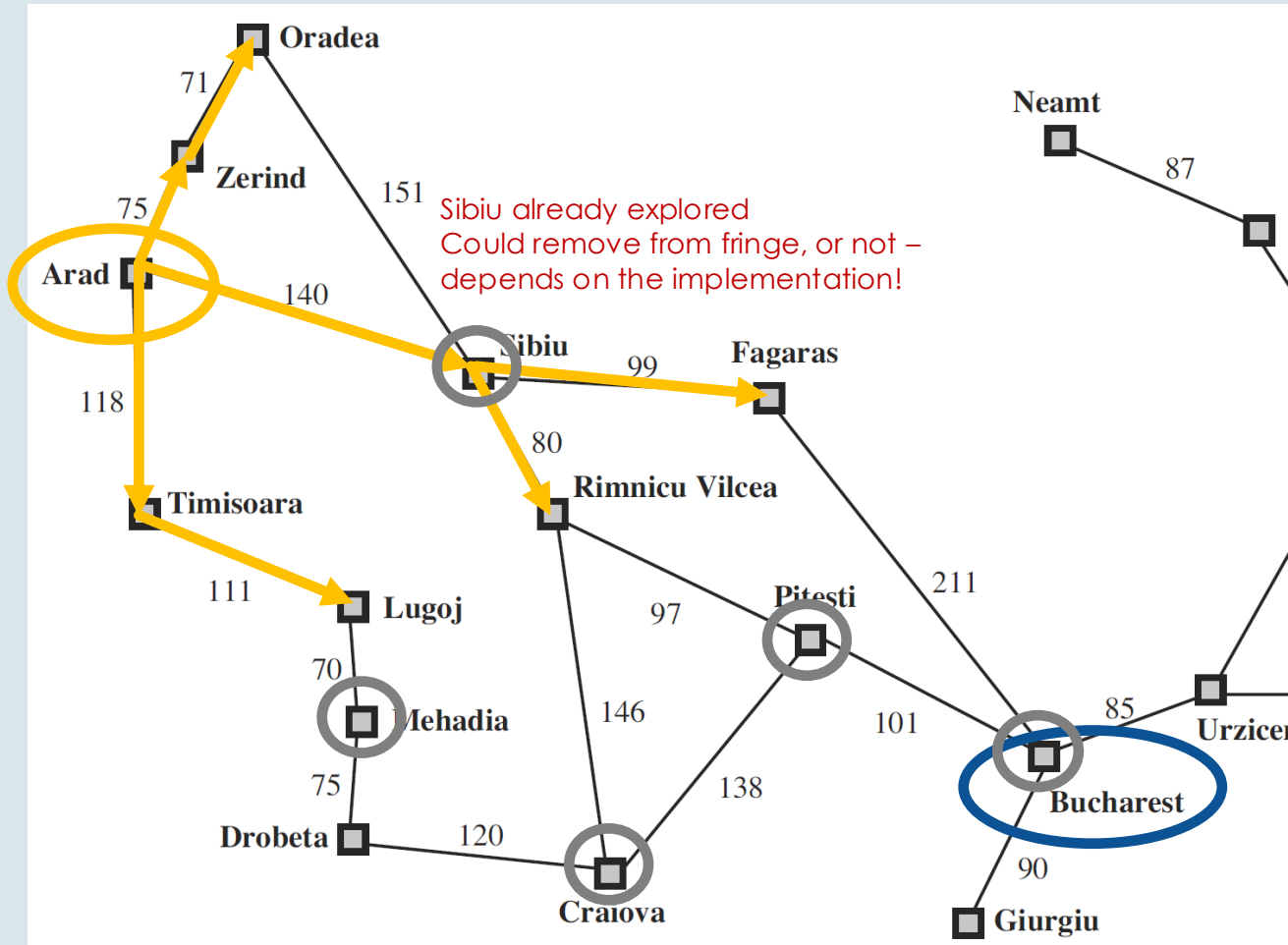


Search Tree

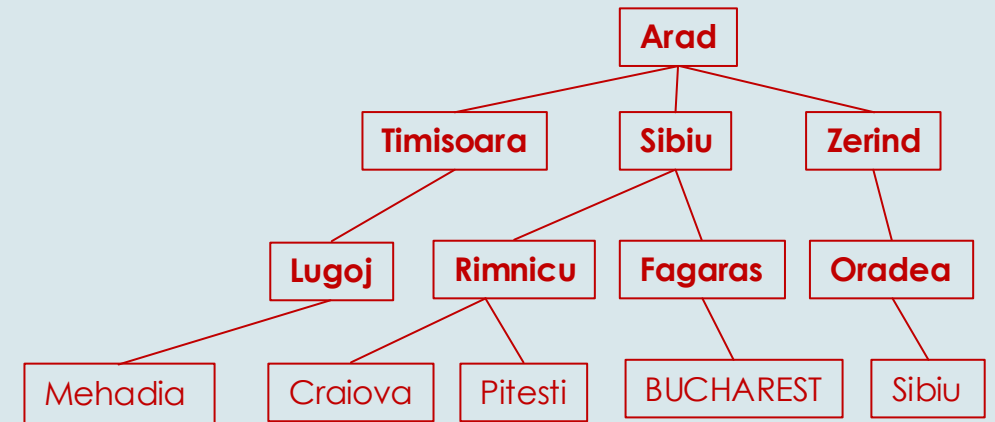


Breadth First Search

2

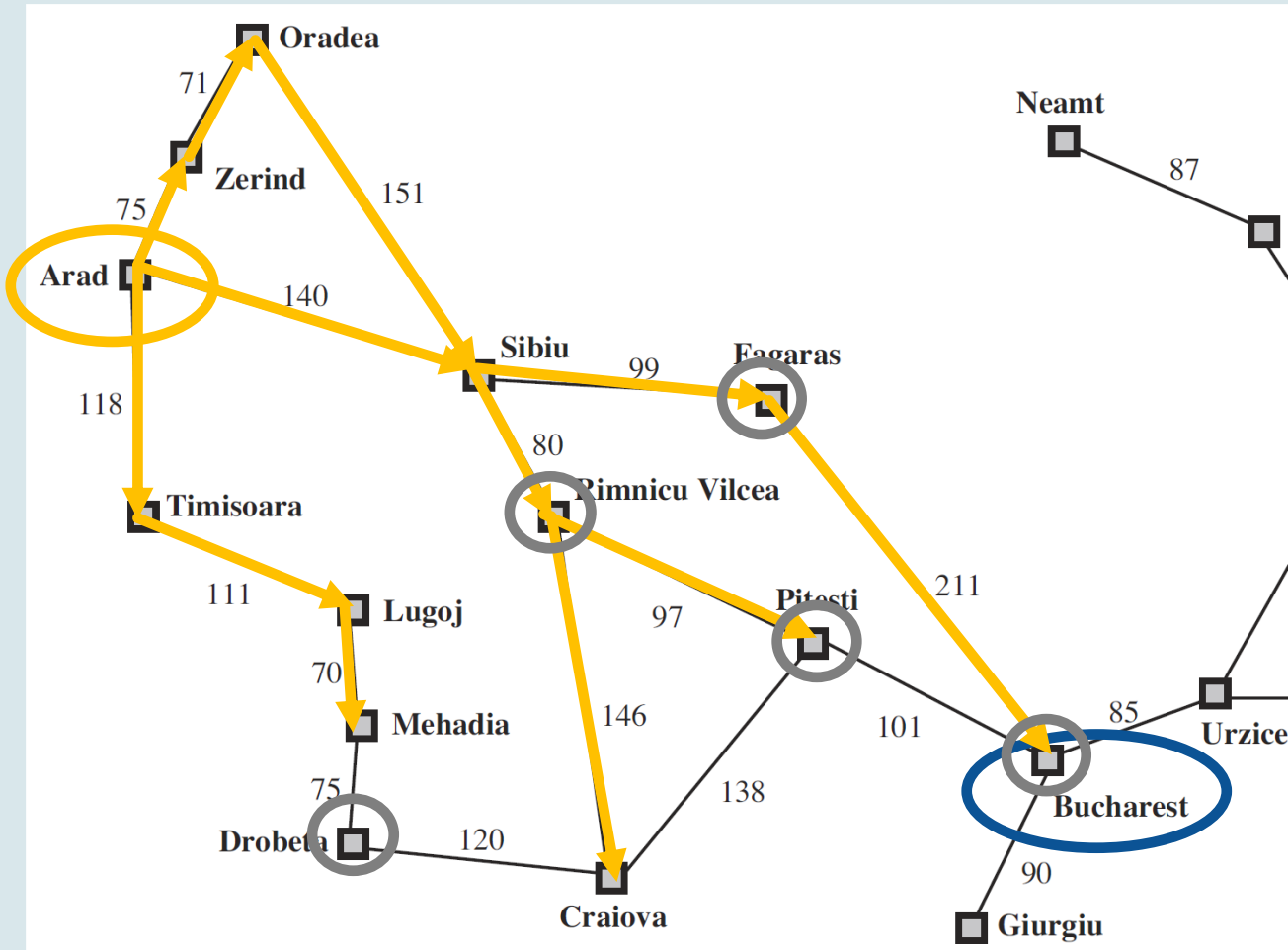


Search Tree

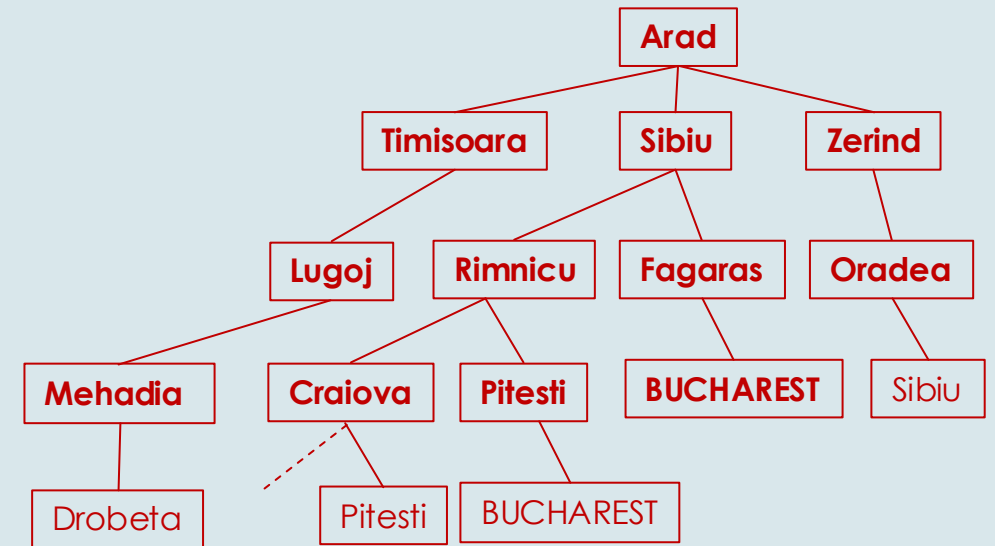


Breadth First Search

3



Search Tree



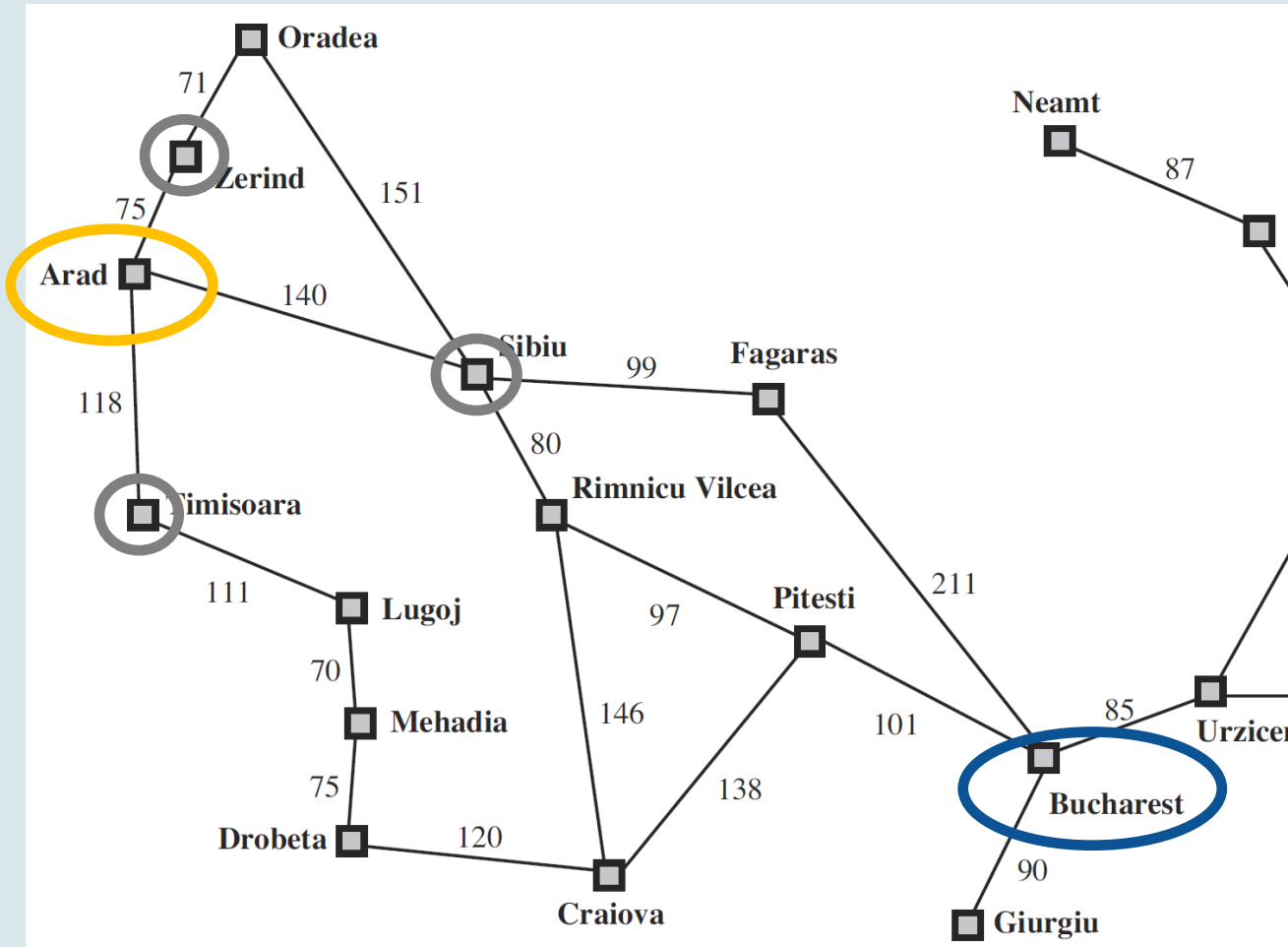
Path length: 3

Nodes expanded: 12

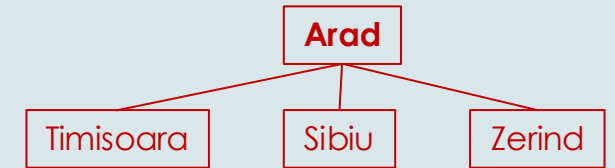
Max. nodes at the fringe: 5

Depth-First Search

4

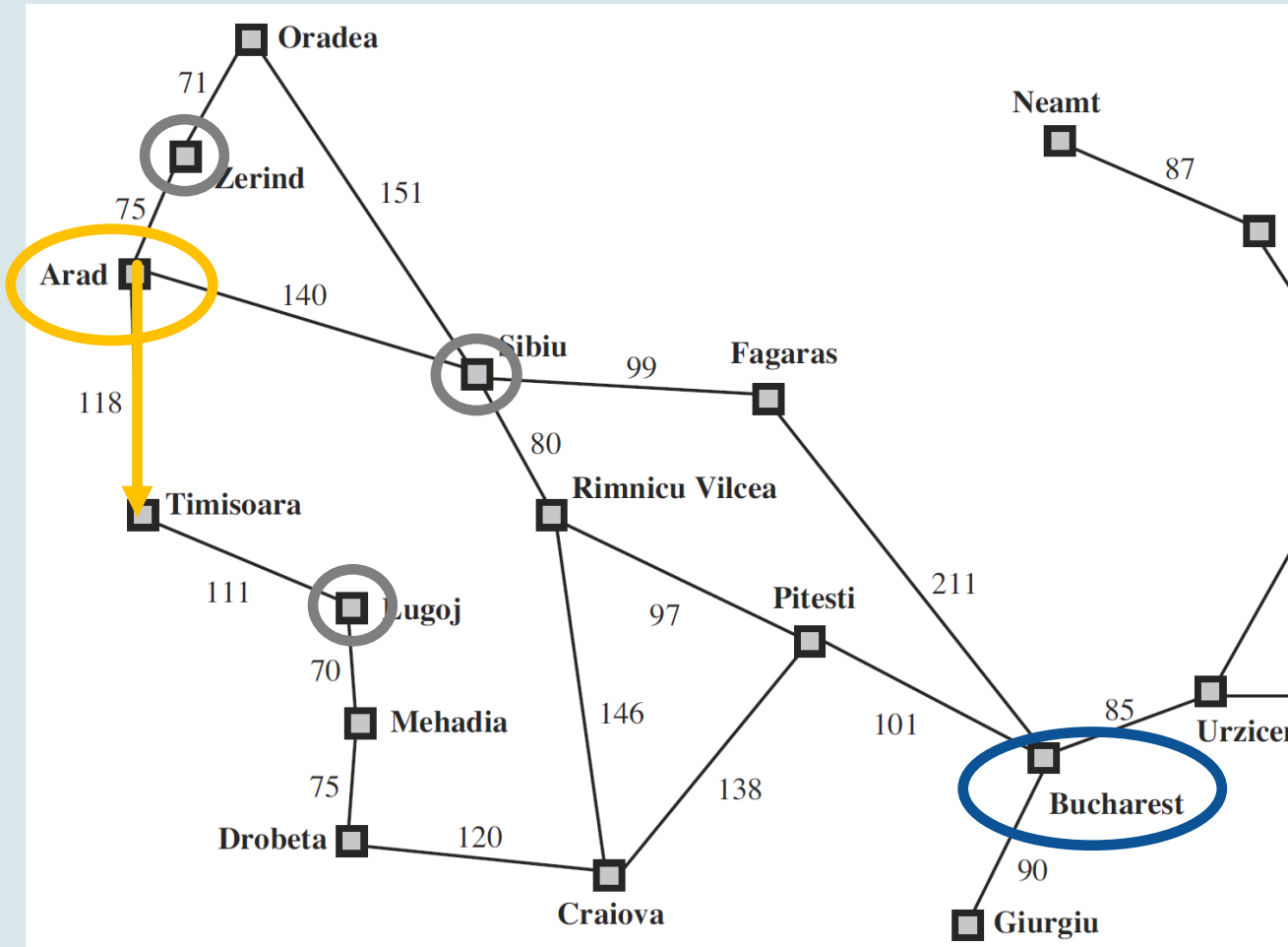


Search Tree

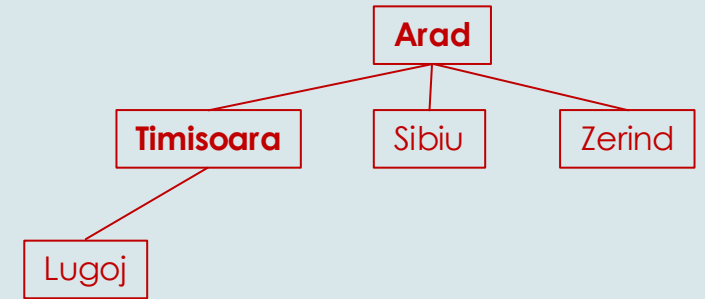


Depth-First Search

5

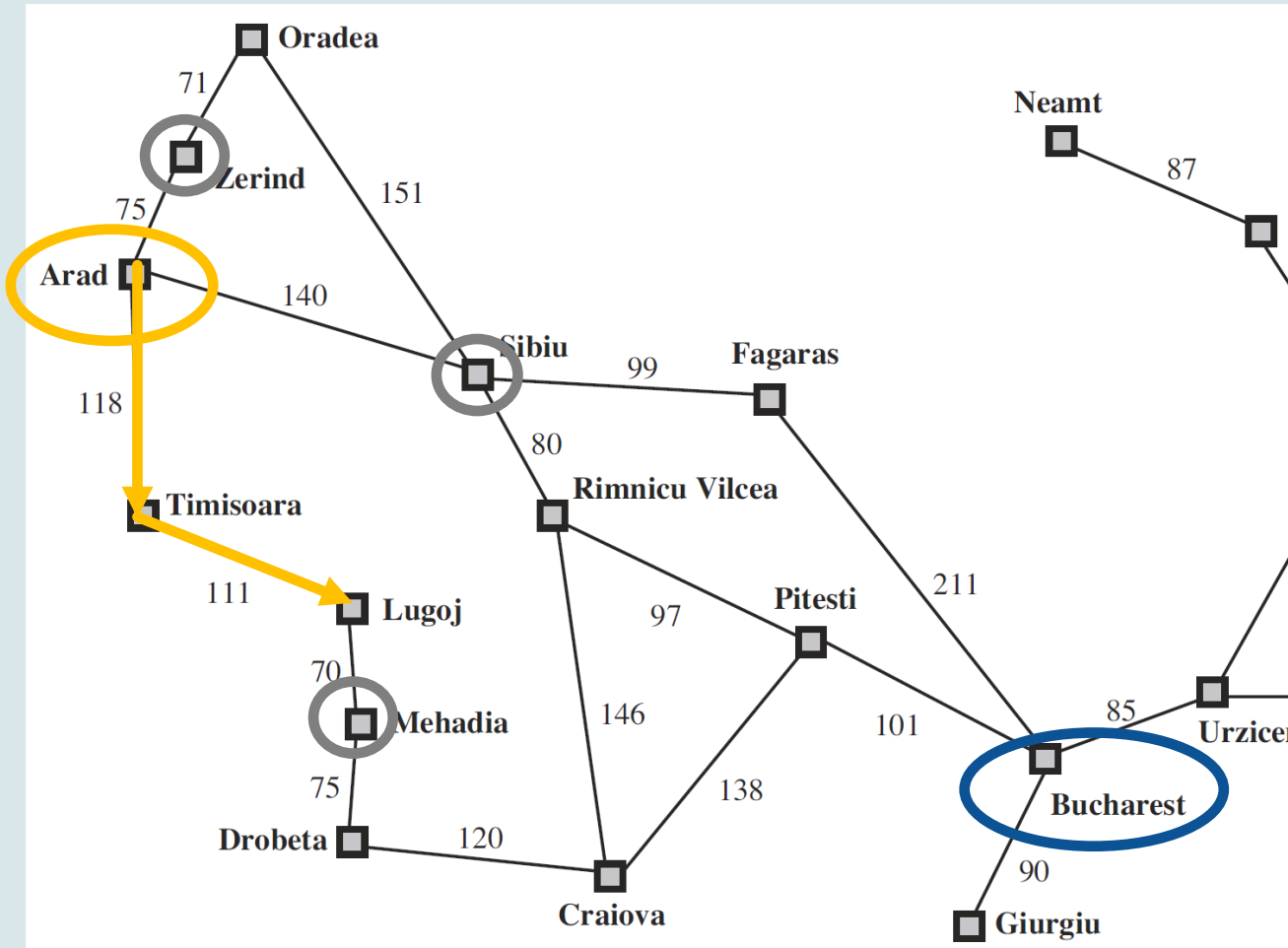


Search Tree

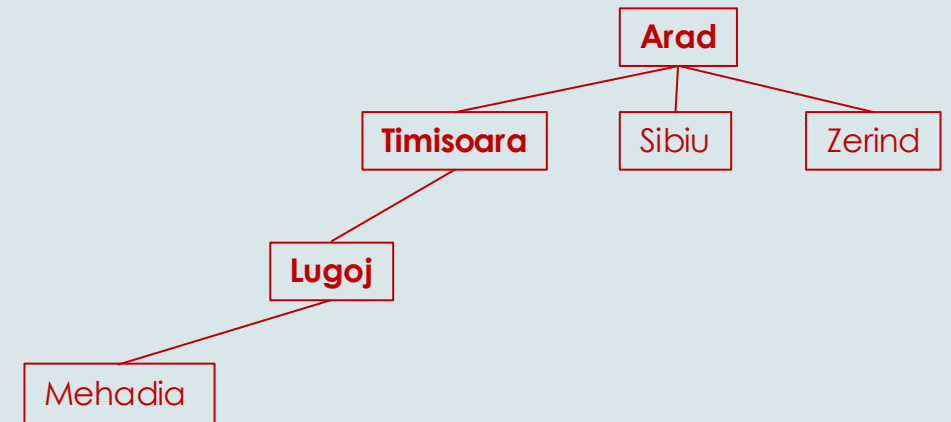


Depth-First Search

6

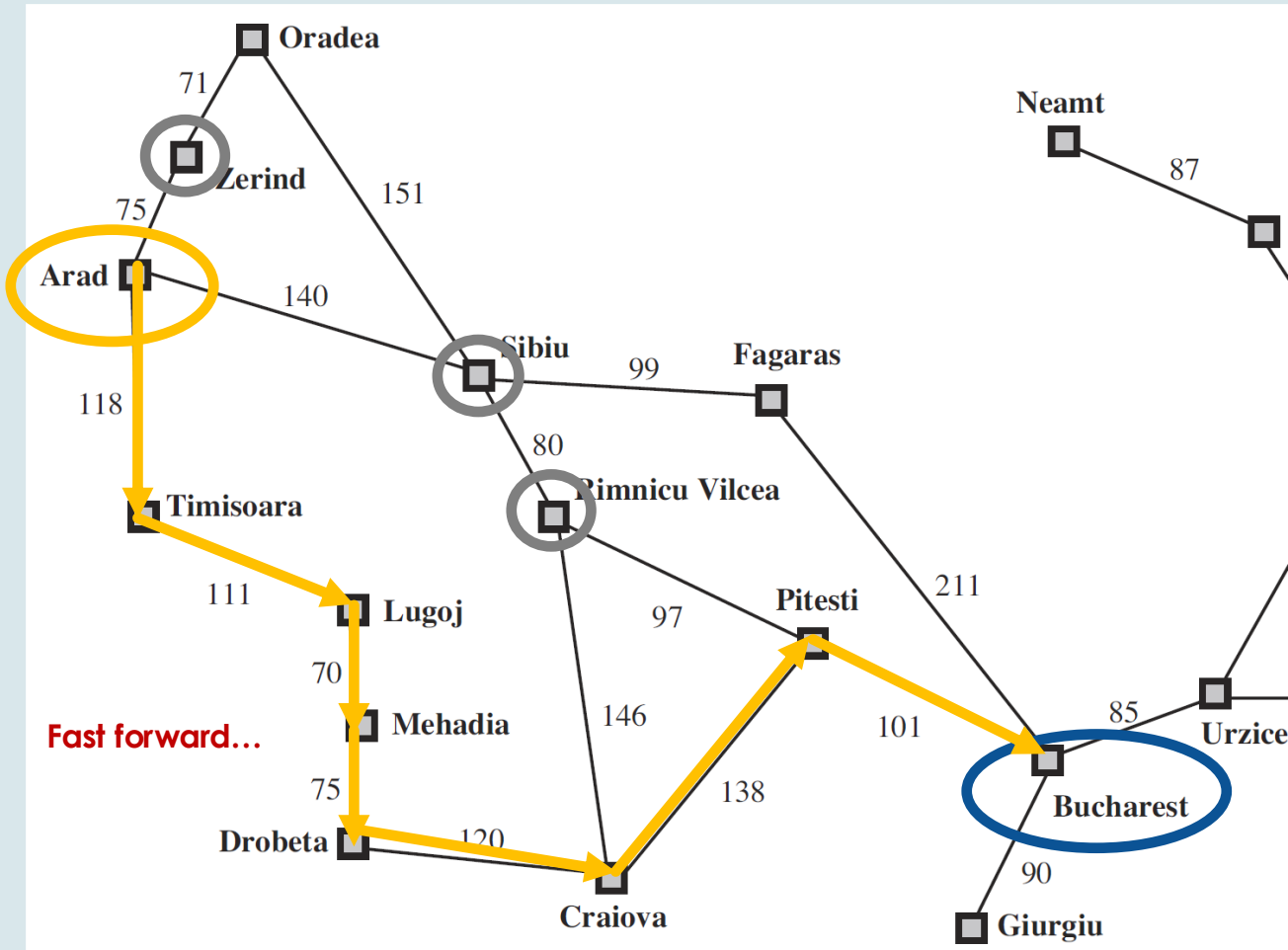


Search Tree

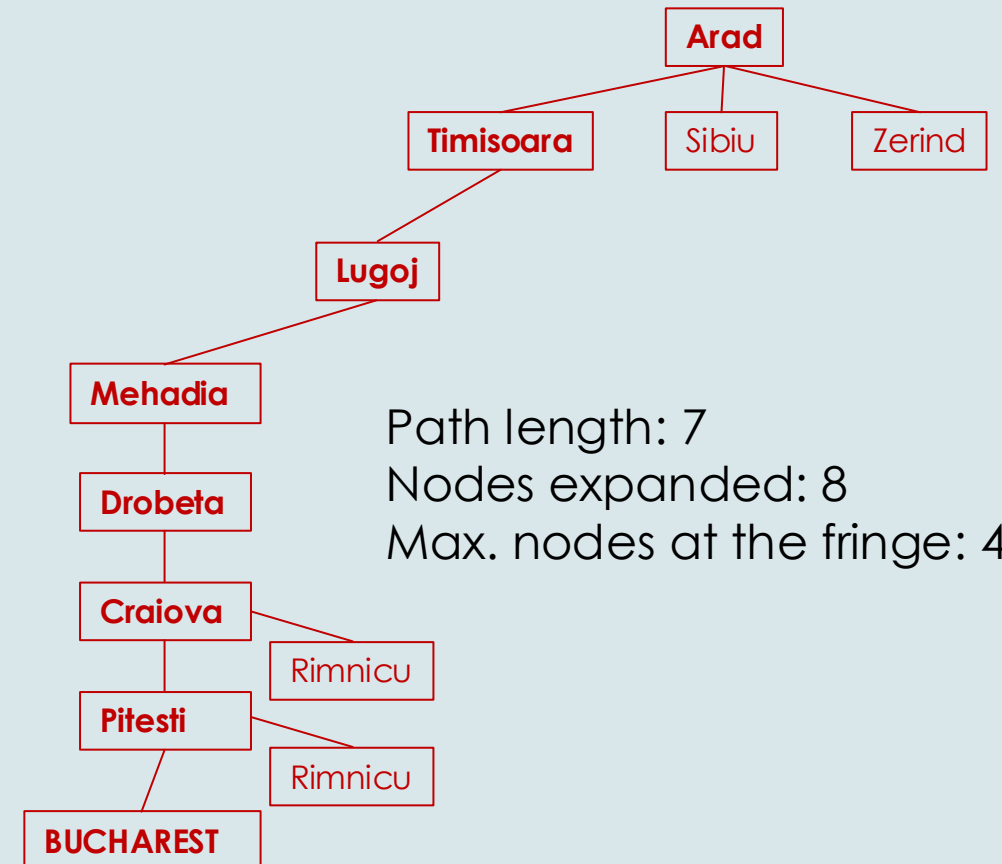


Depth-First Search

7



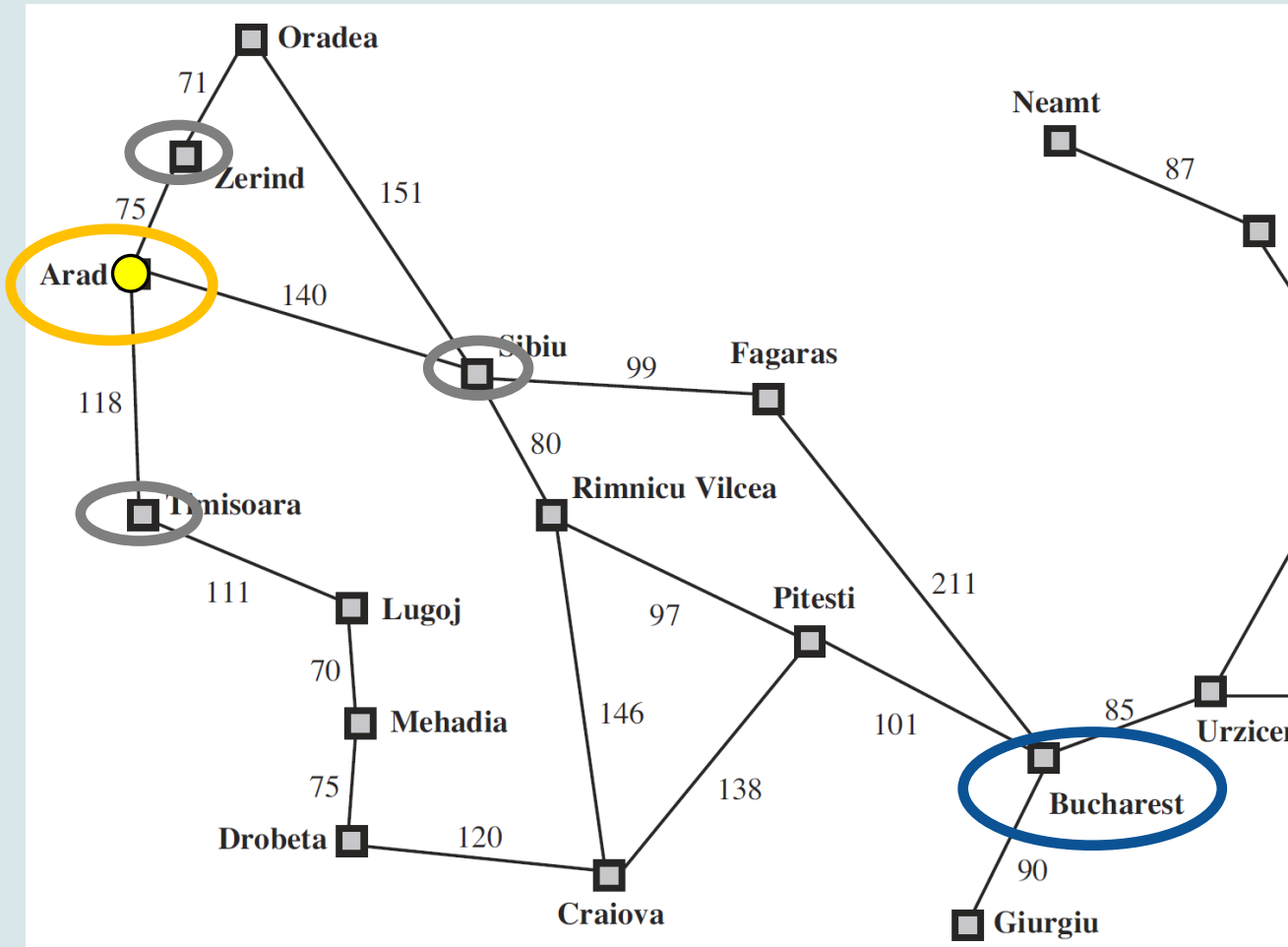
Search Tree



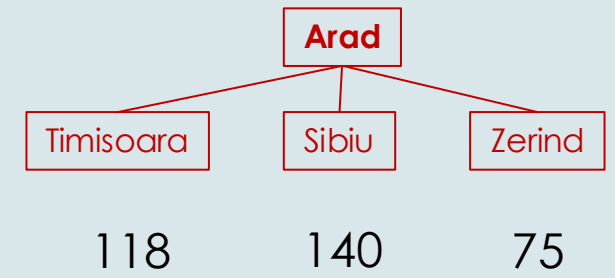
Path length: 7
Nodes expanded: 8
Max. nodes at the fringe: 4

Uniform Cost Search

8

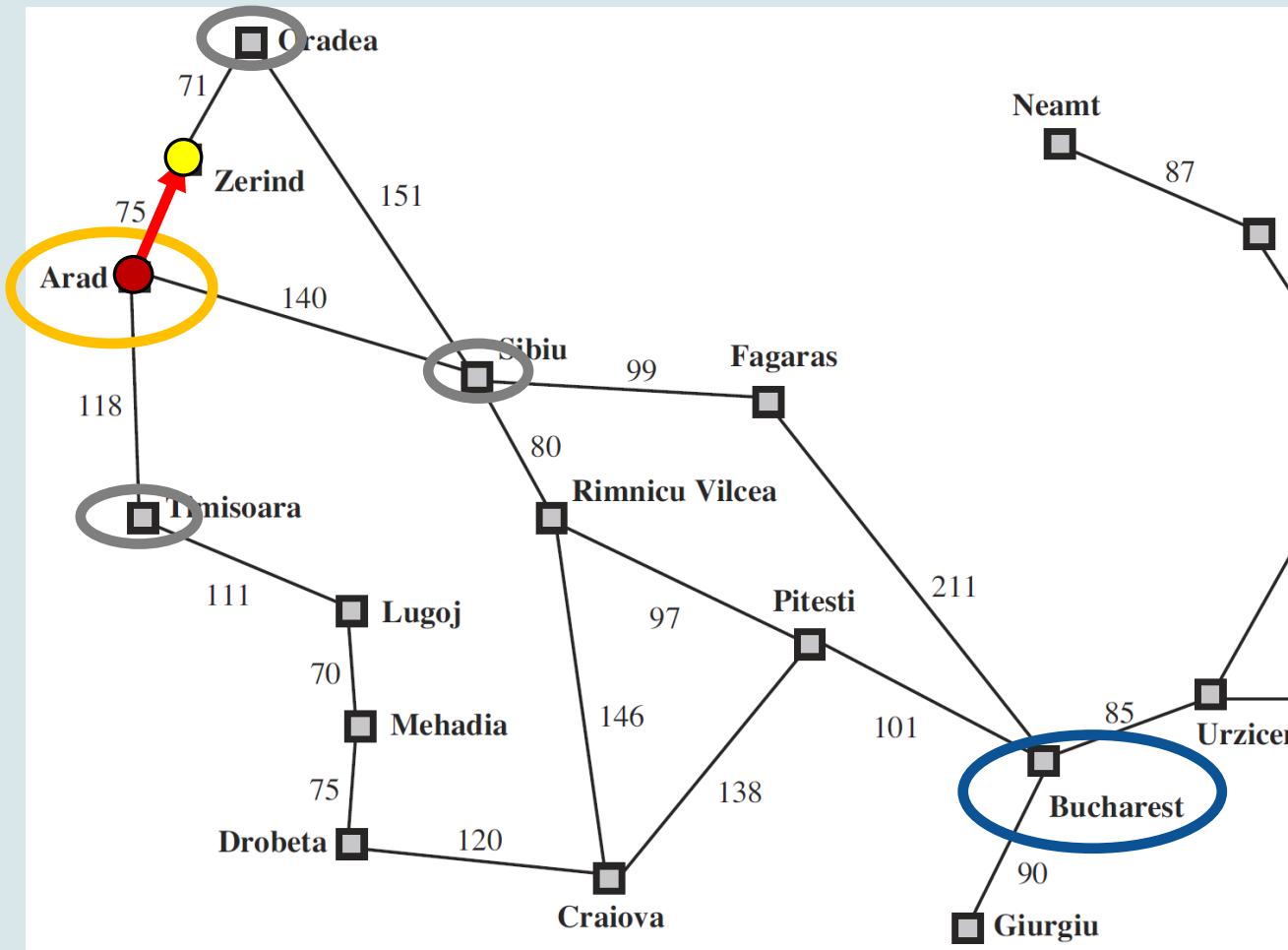


Search Tree

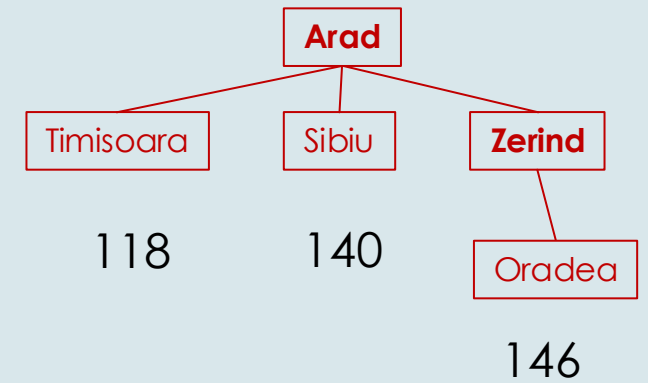


Uniform Cost Search

9

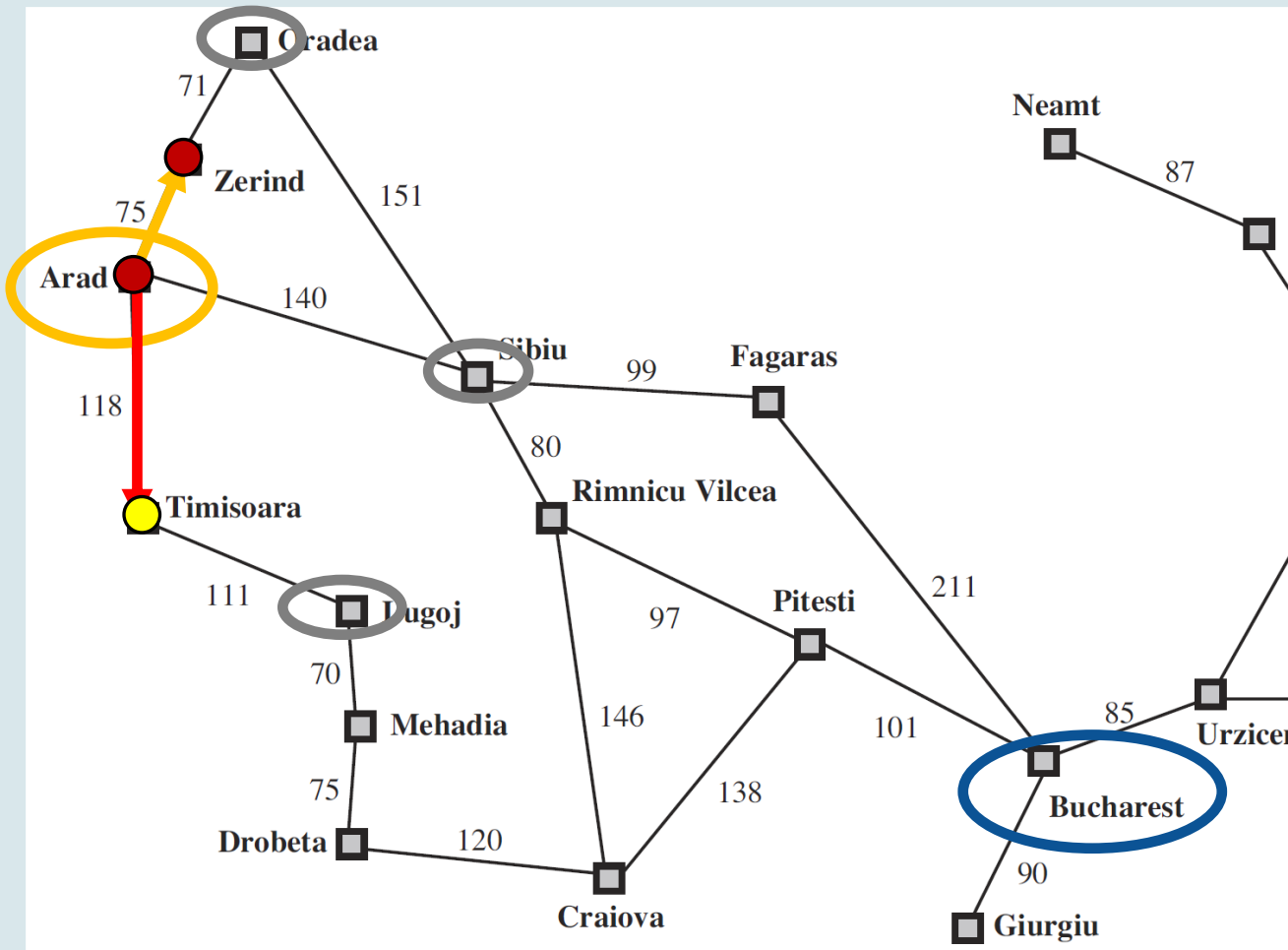


Search Tree

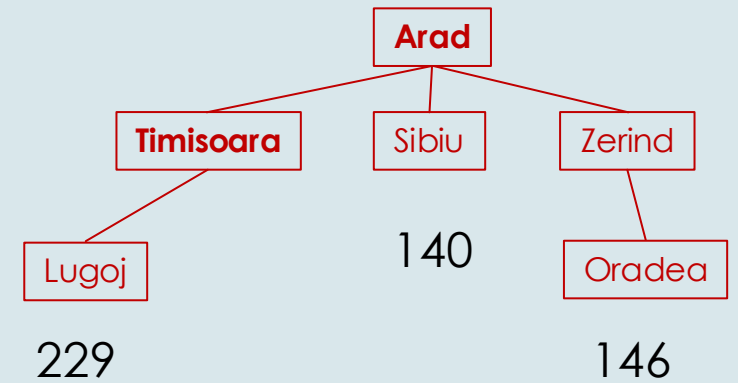


Uniform Cost Search

10

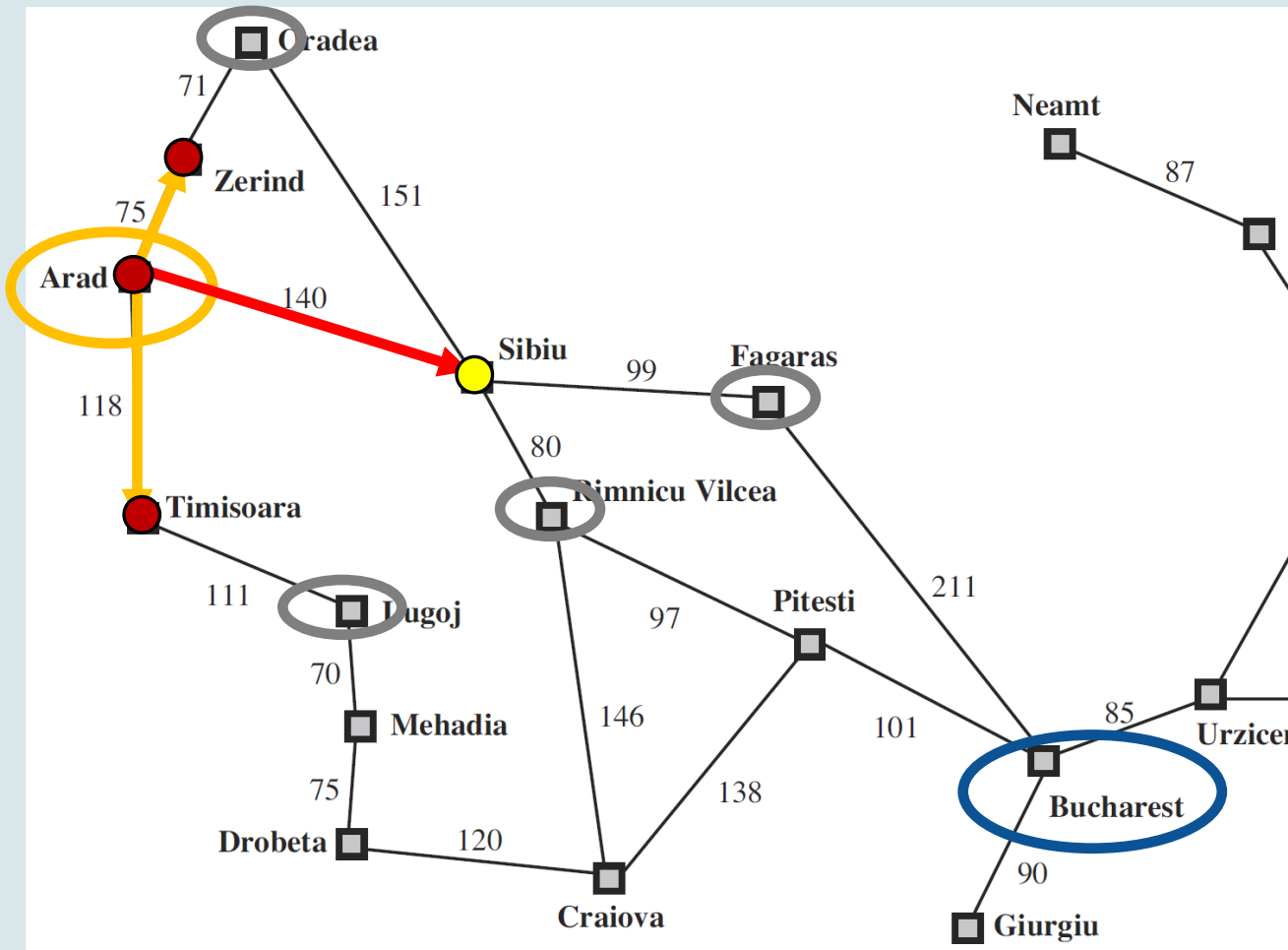


Search Tree

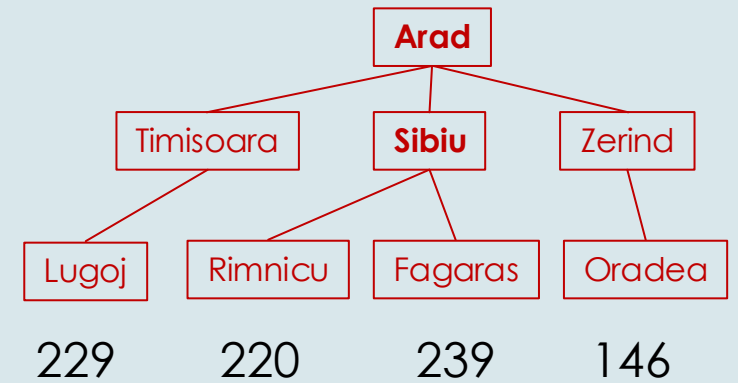


Uniform Cost Search

11

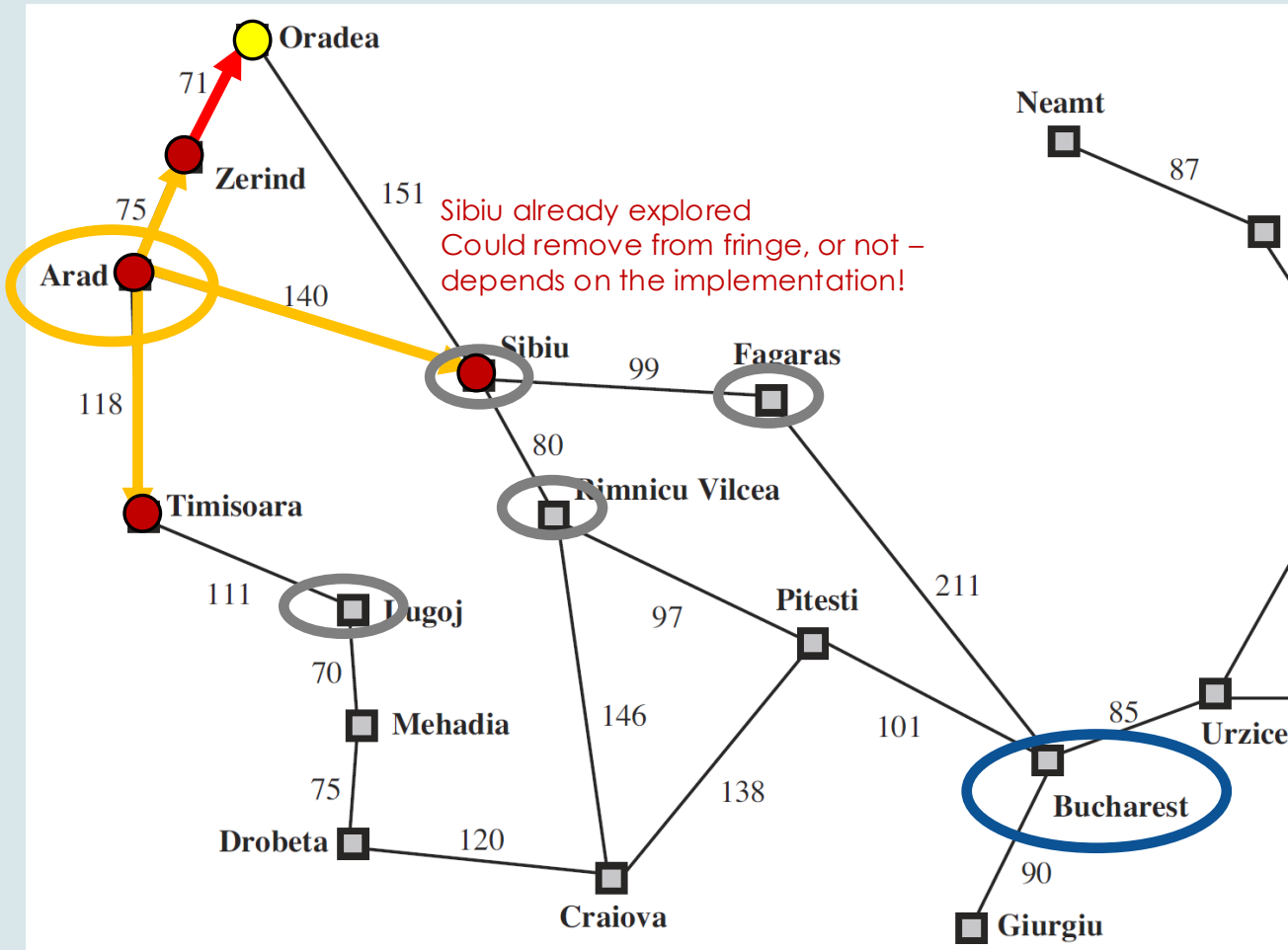


Search Tree

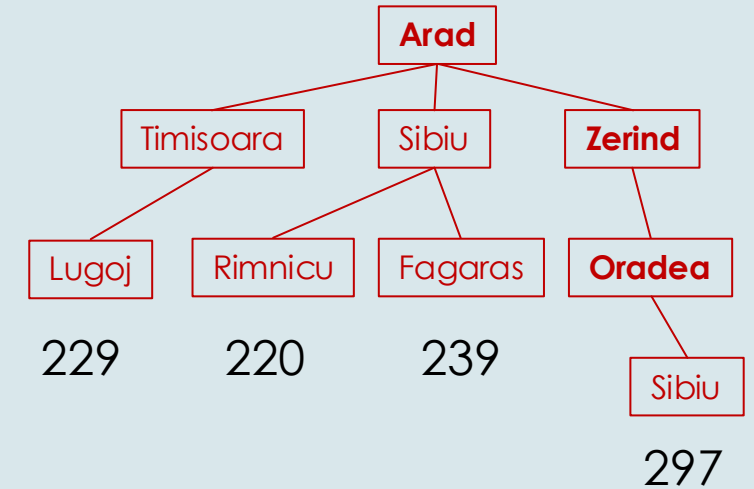


Uniform Cost Search

12

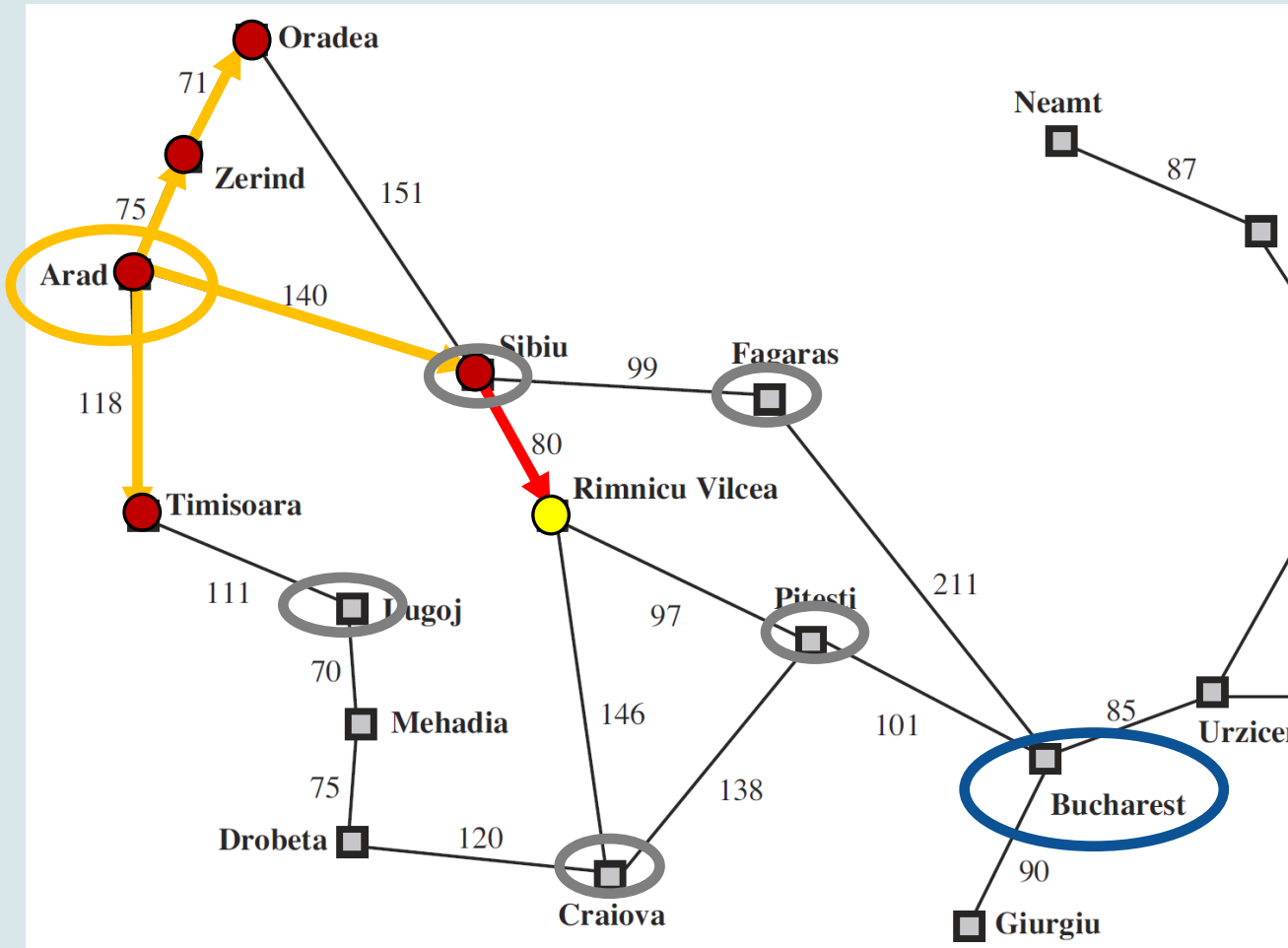


Search Tree

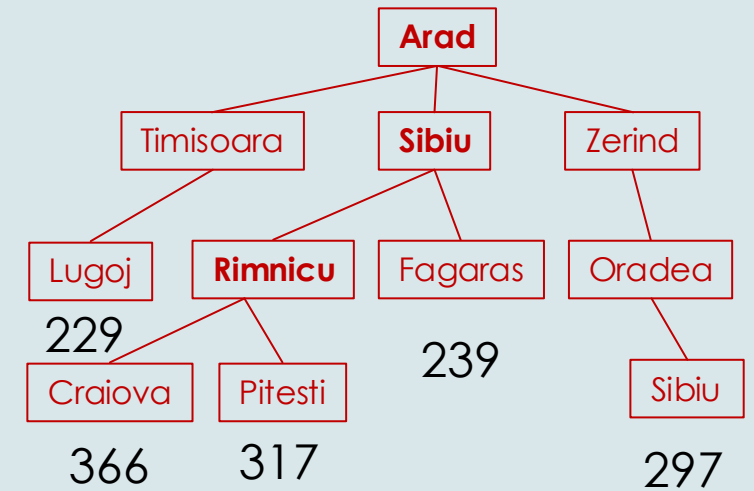


Uniform Cost Search

13

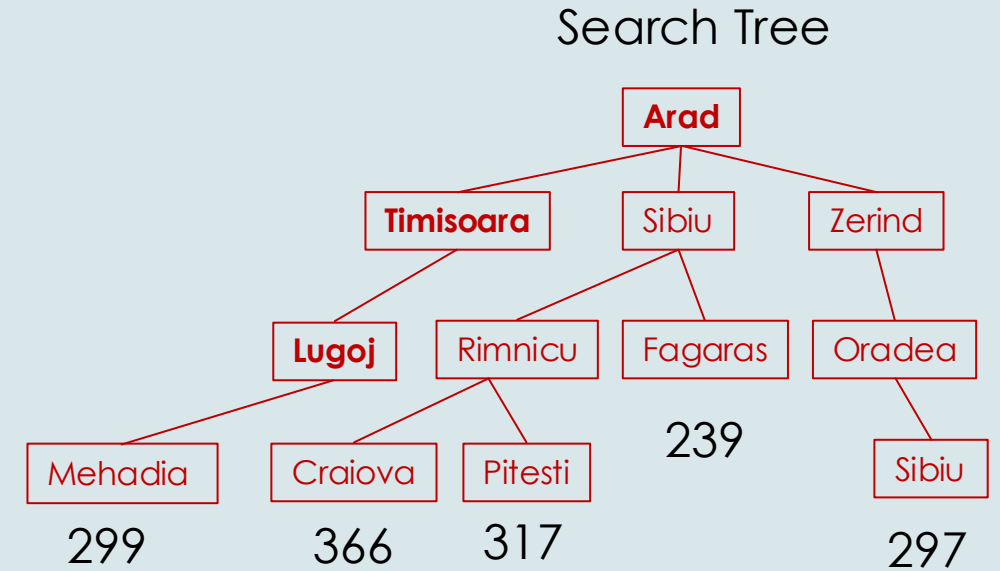
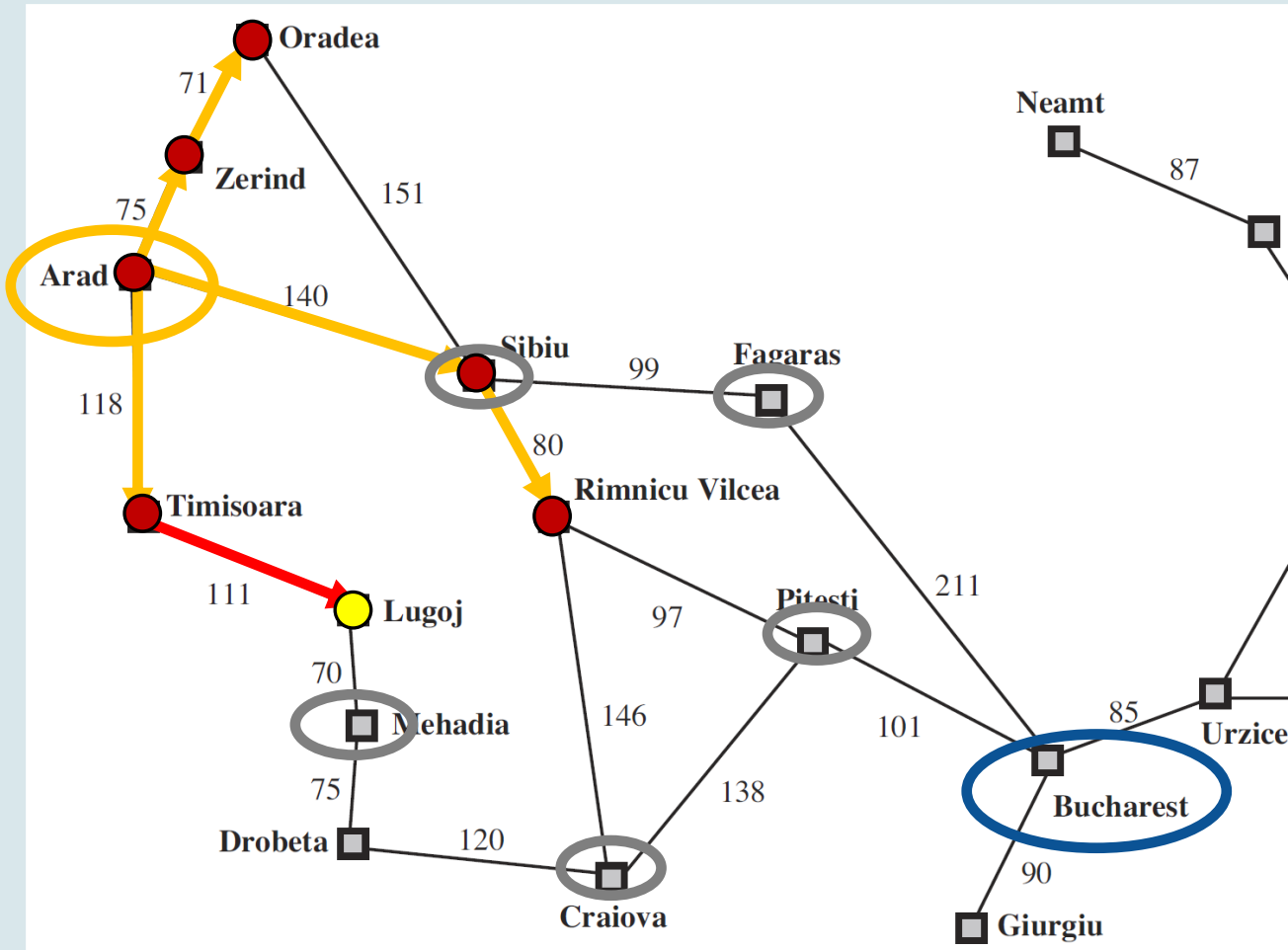


Search Tree



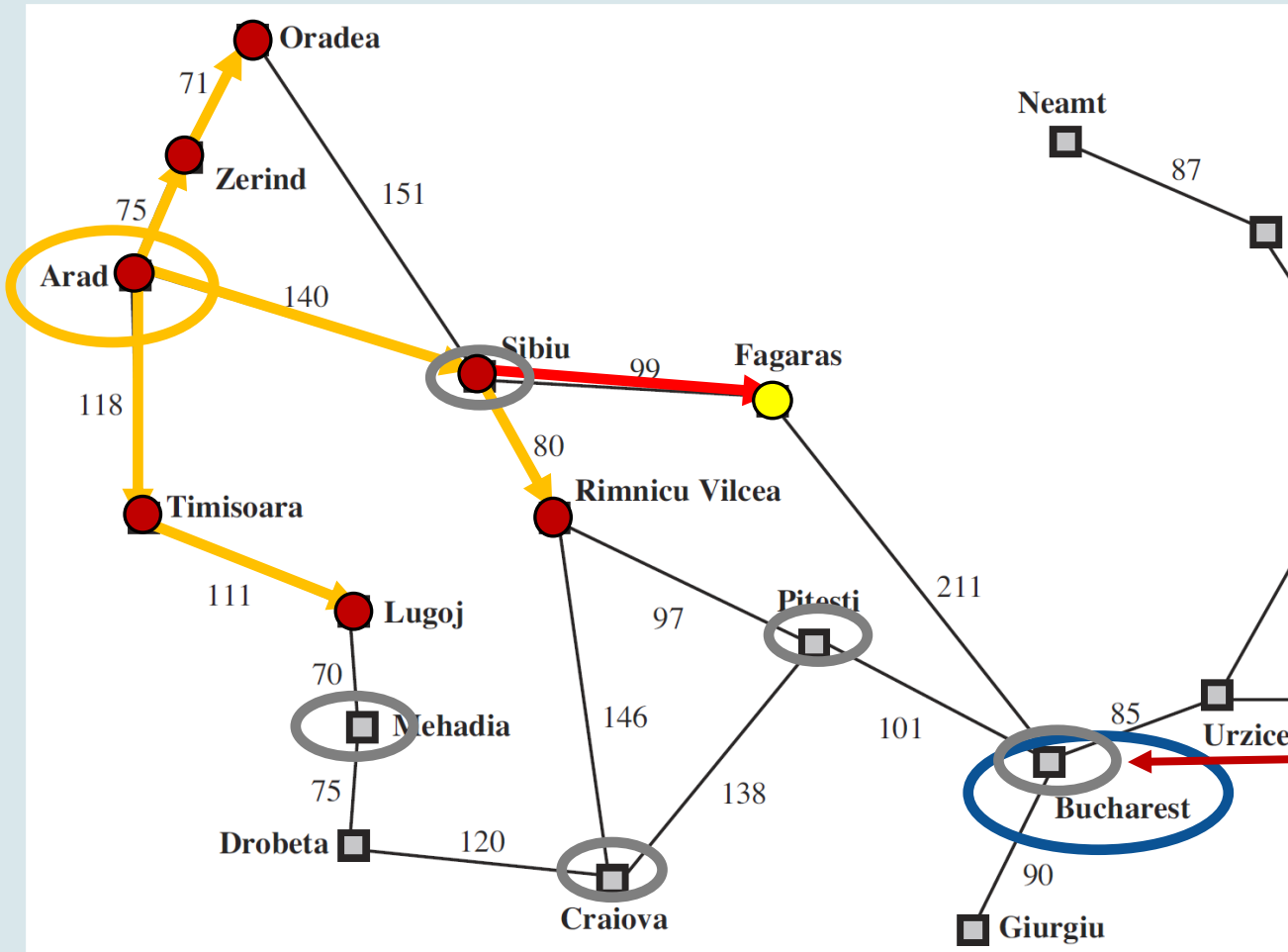
Uniform Cost Search

14

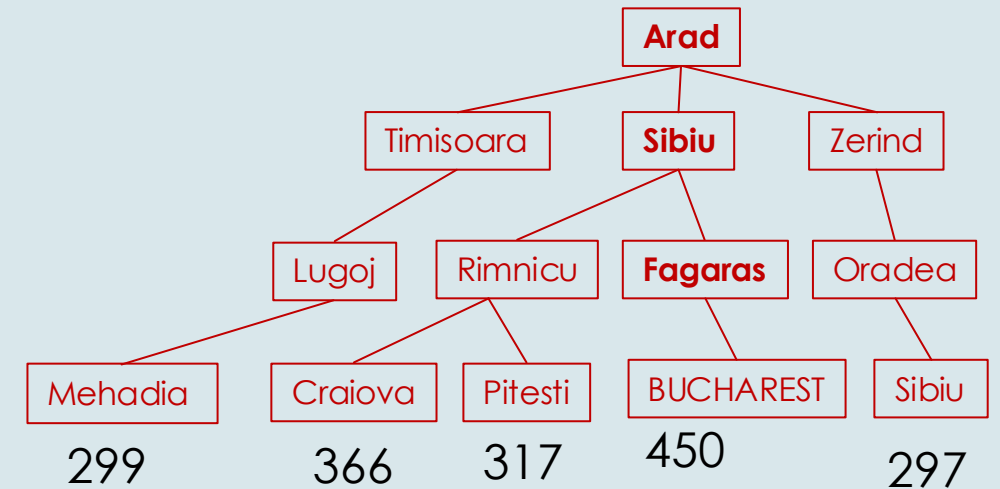


Uniform Cost Search

15



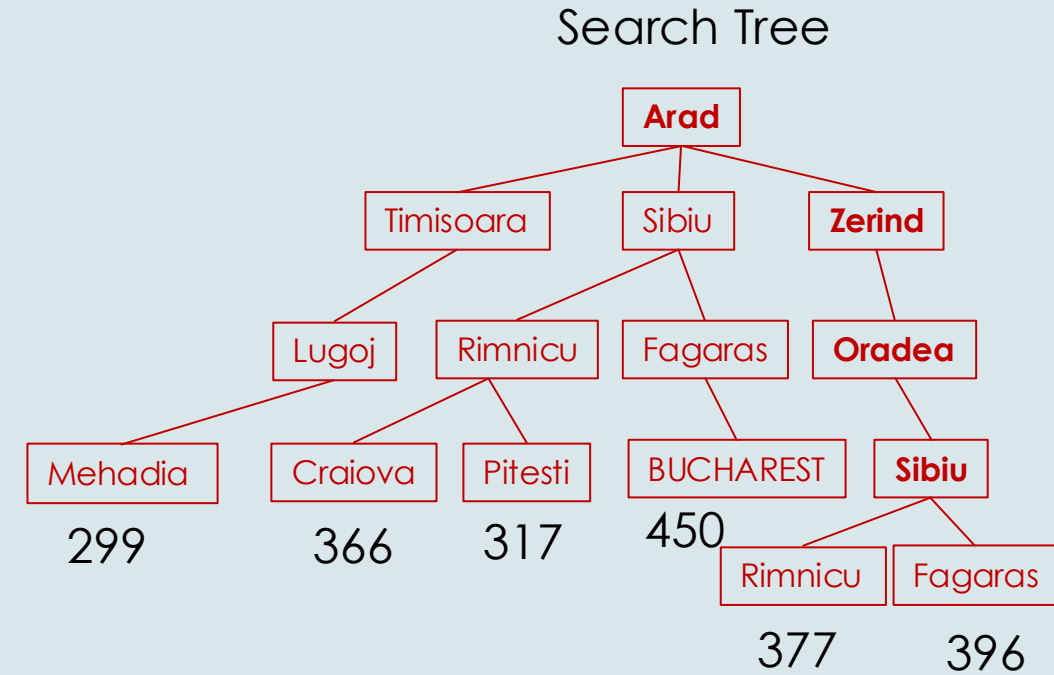
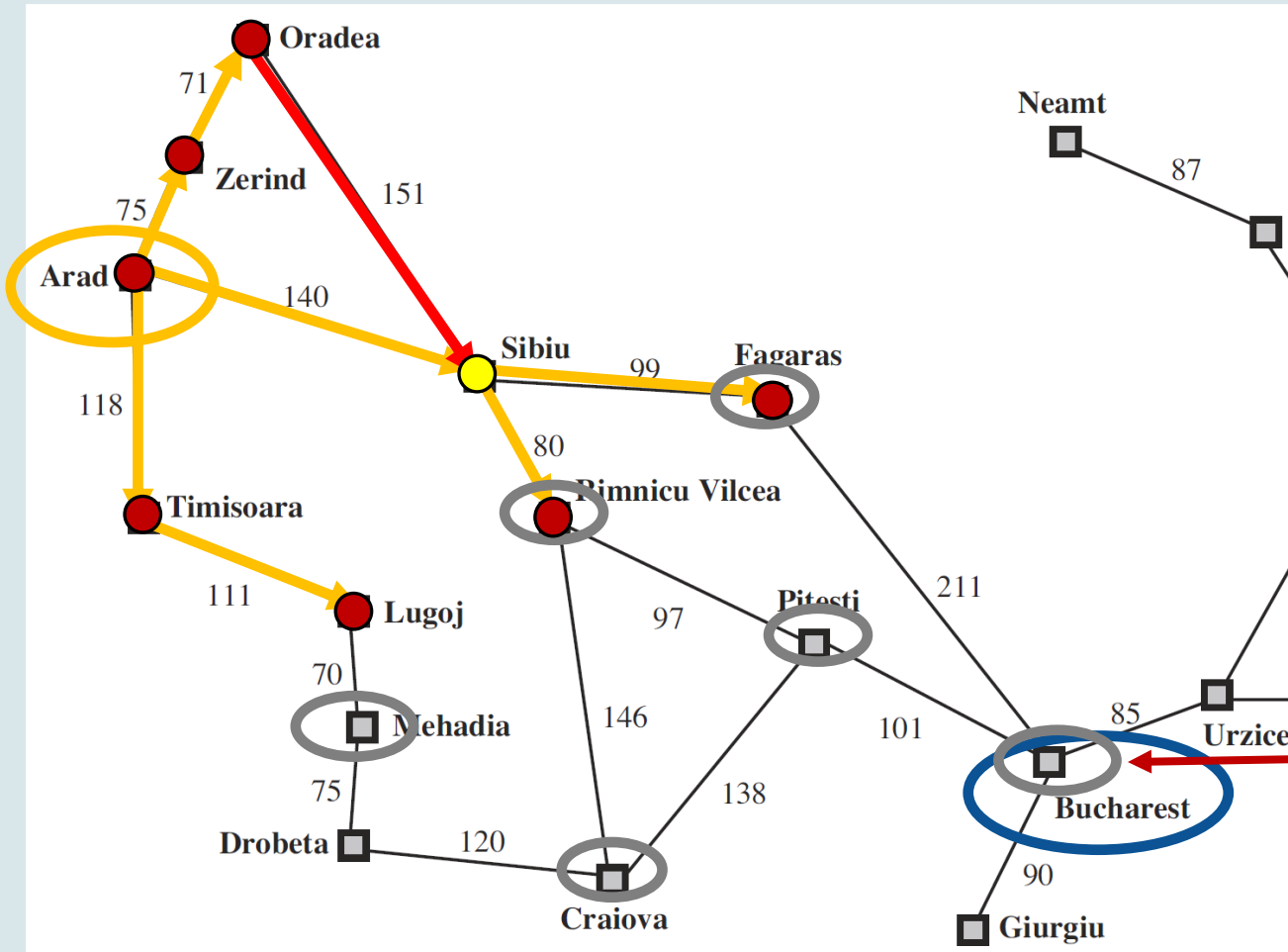
Search Tree



Are we done?

Uniform Cost Search

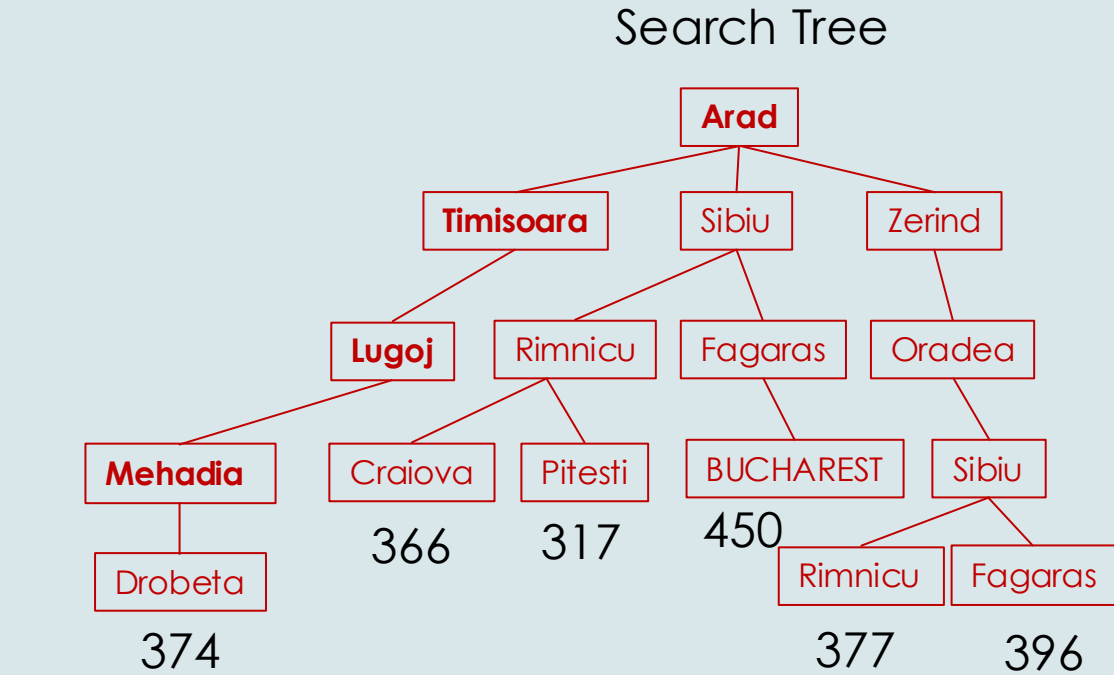
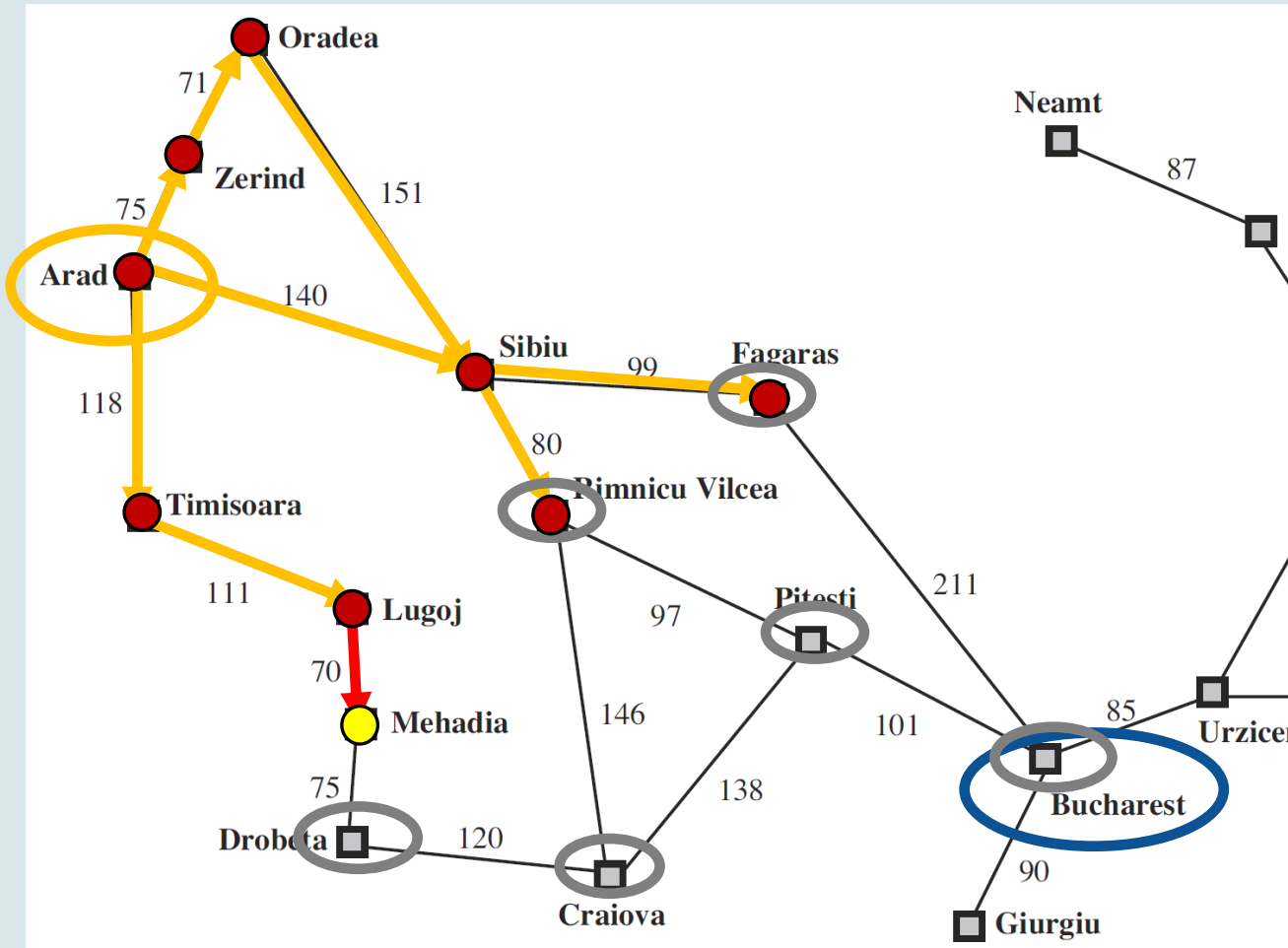
16



Are we done now?

Uniform Cost Search

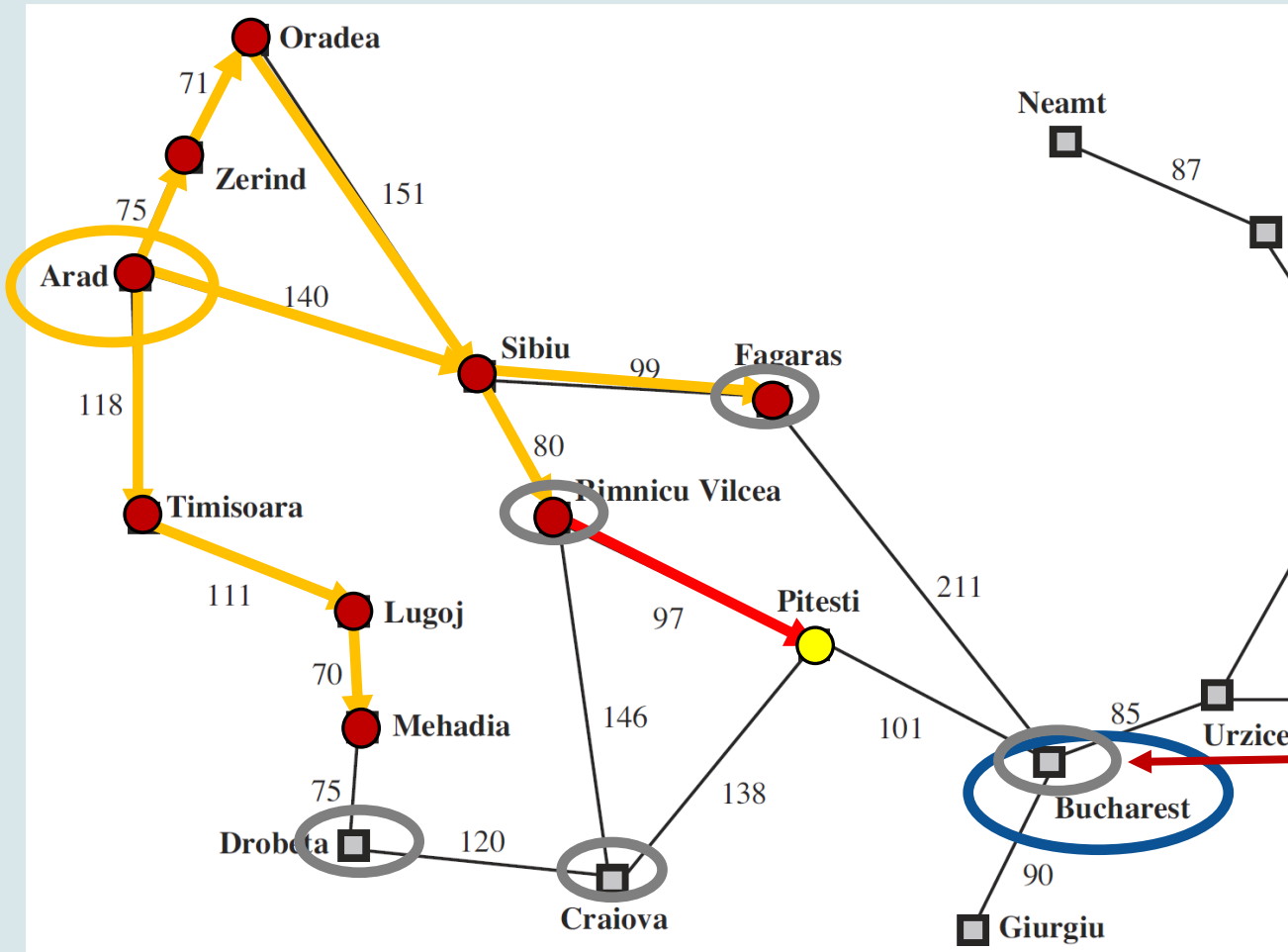
17



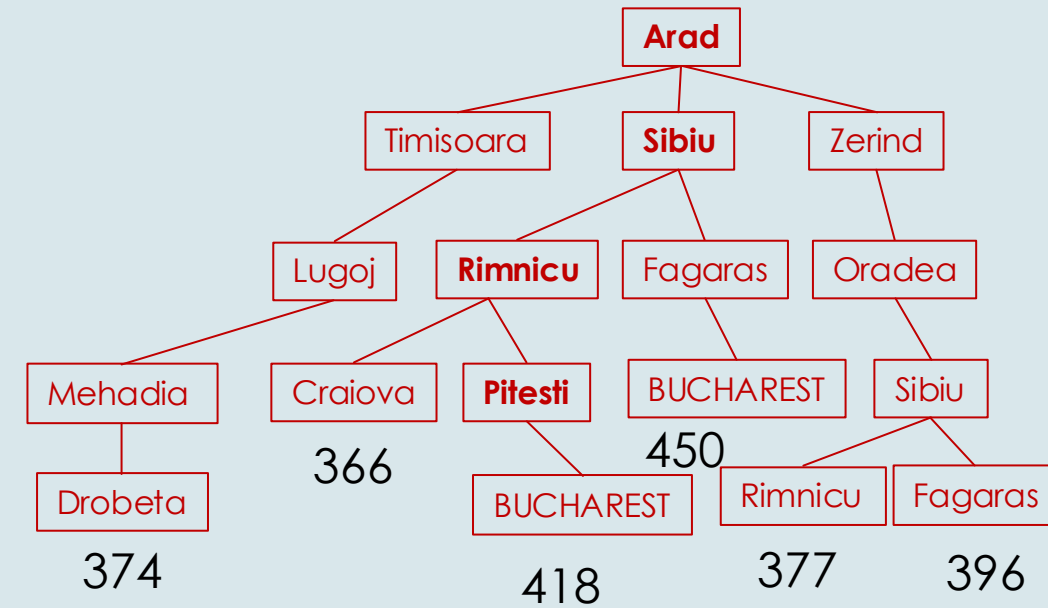
Picture: Russell & Norvig, Chapter 3

Uniform Cost Search

18



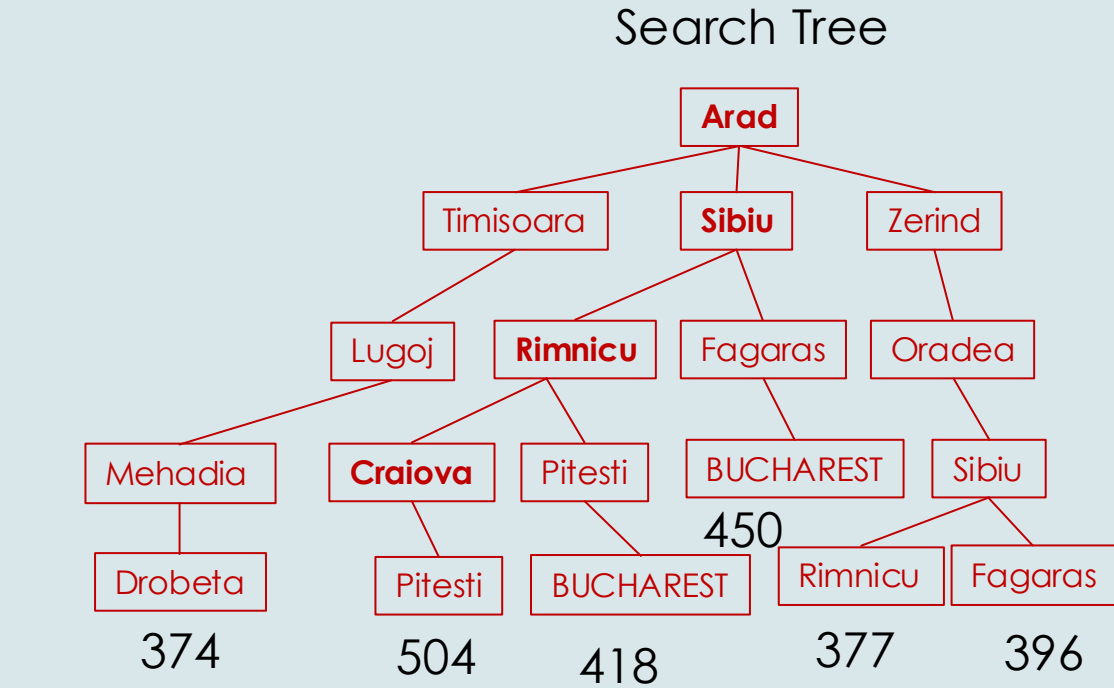
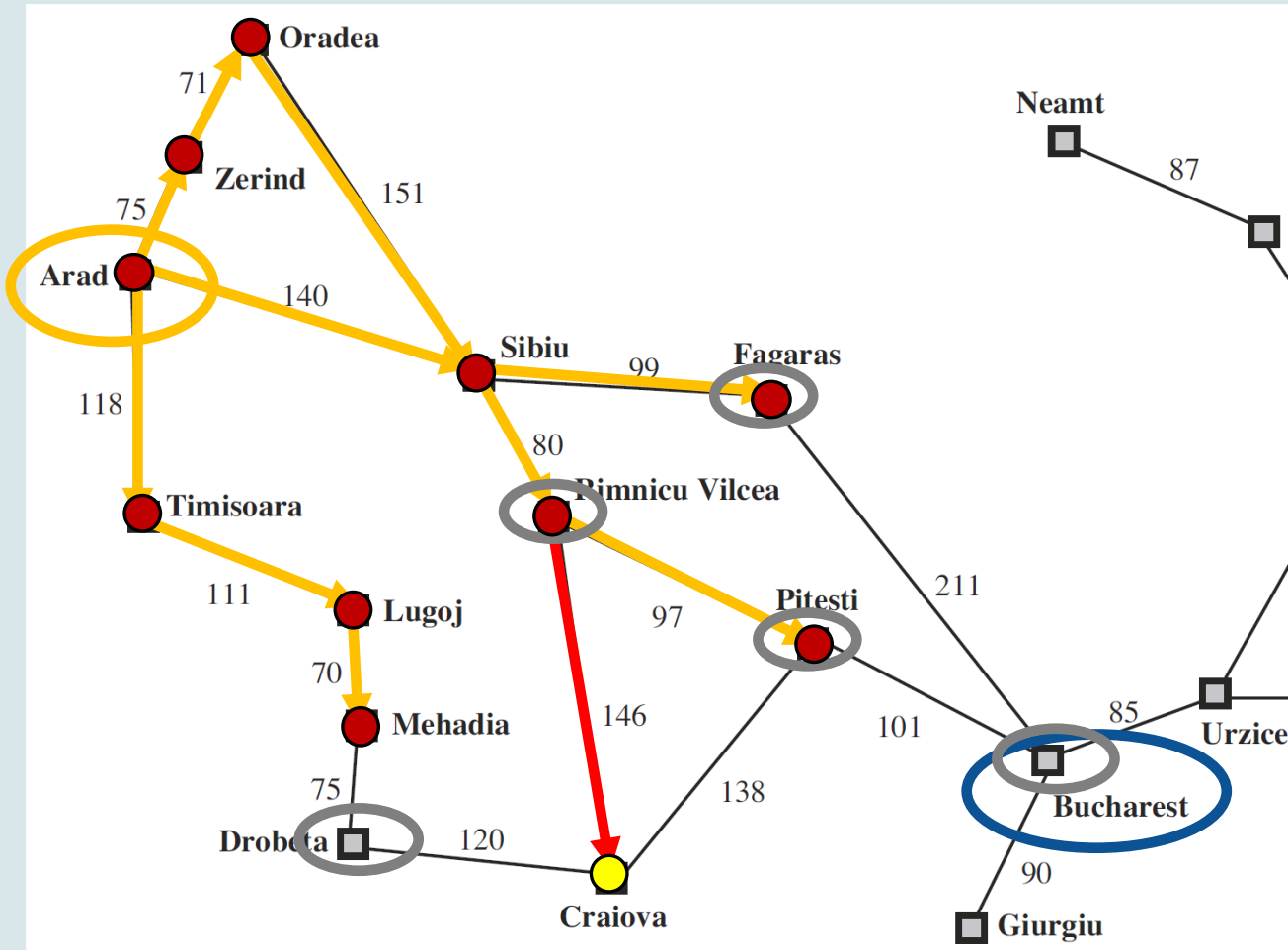
Search Tree



Are we done now?

Uniform Cost Search

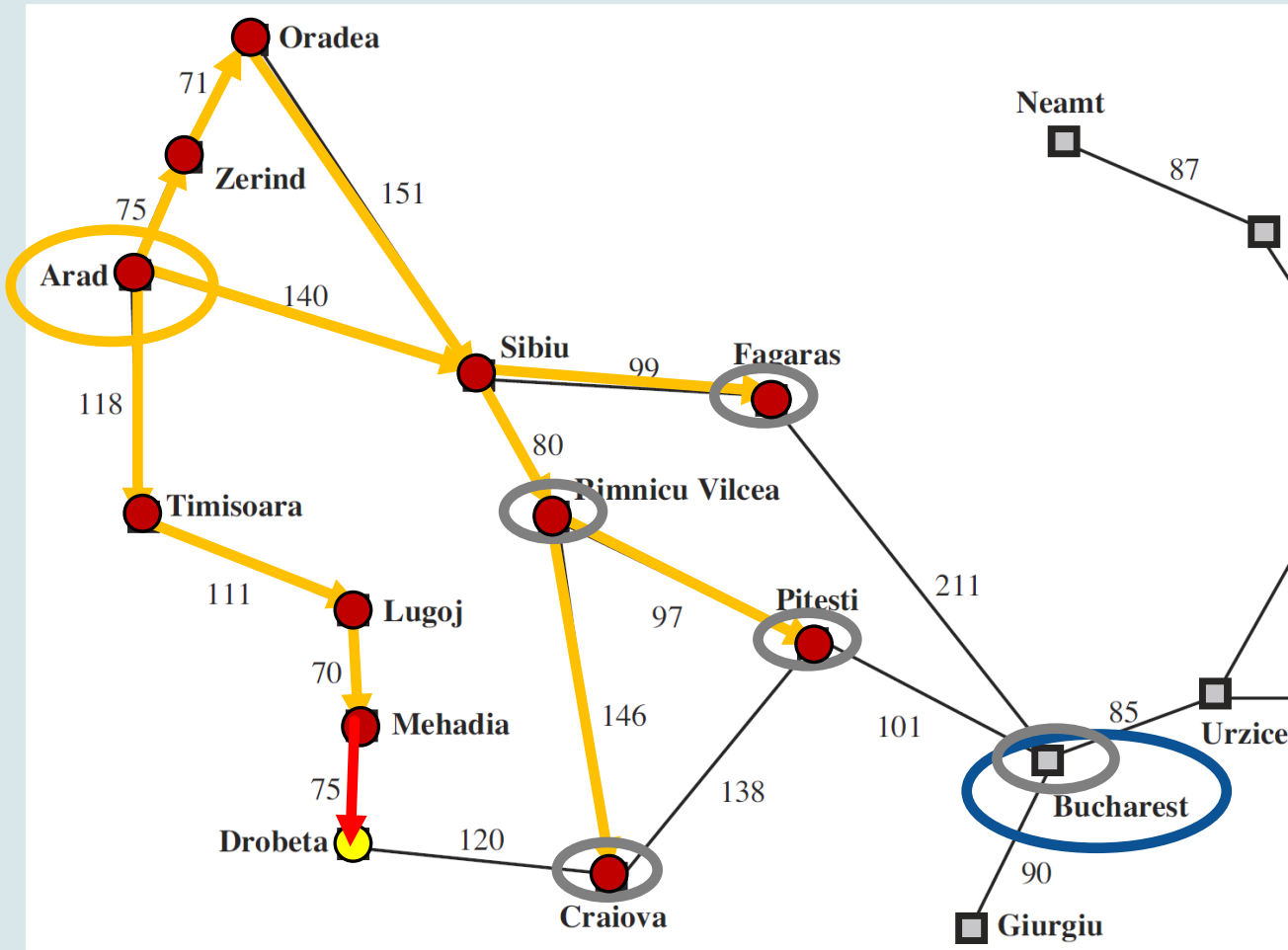
19



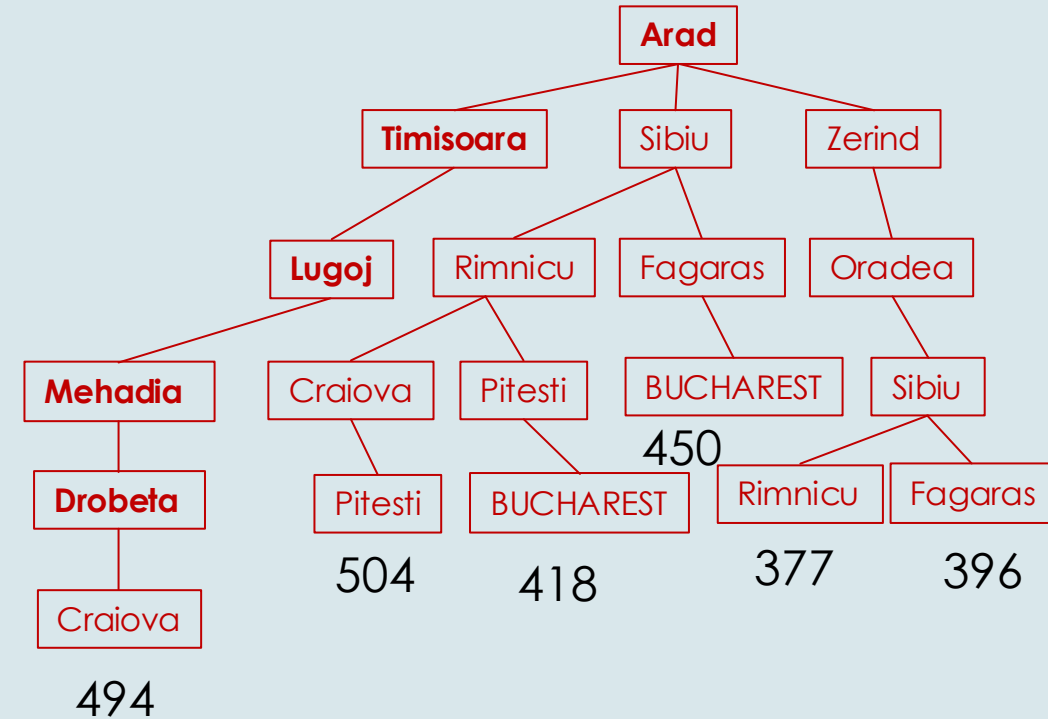
Picture: Russell & Norvig, Chapter 3

Uniform Cost Search

20

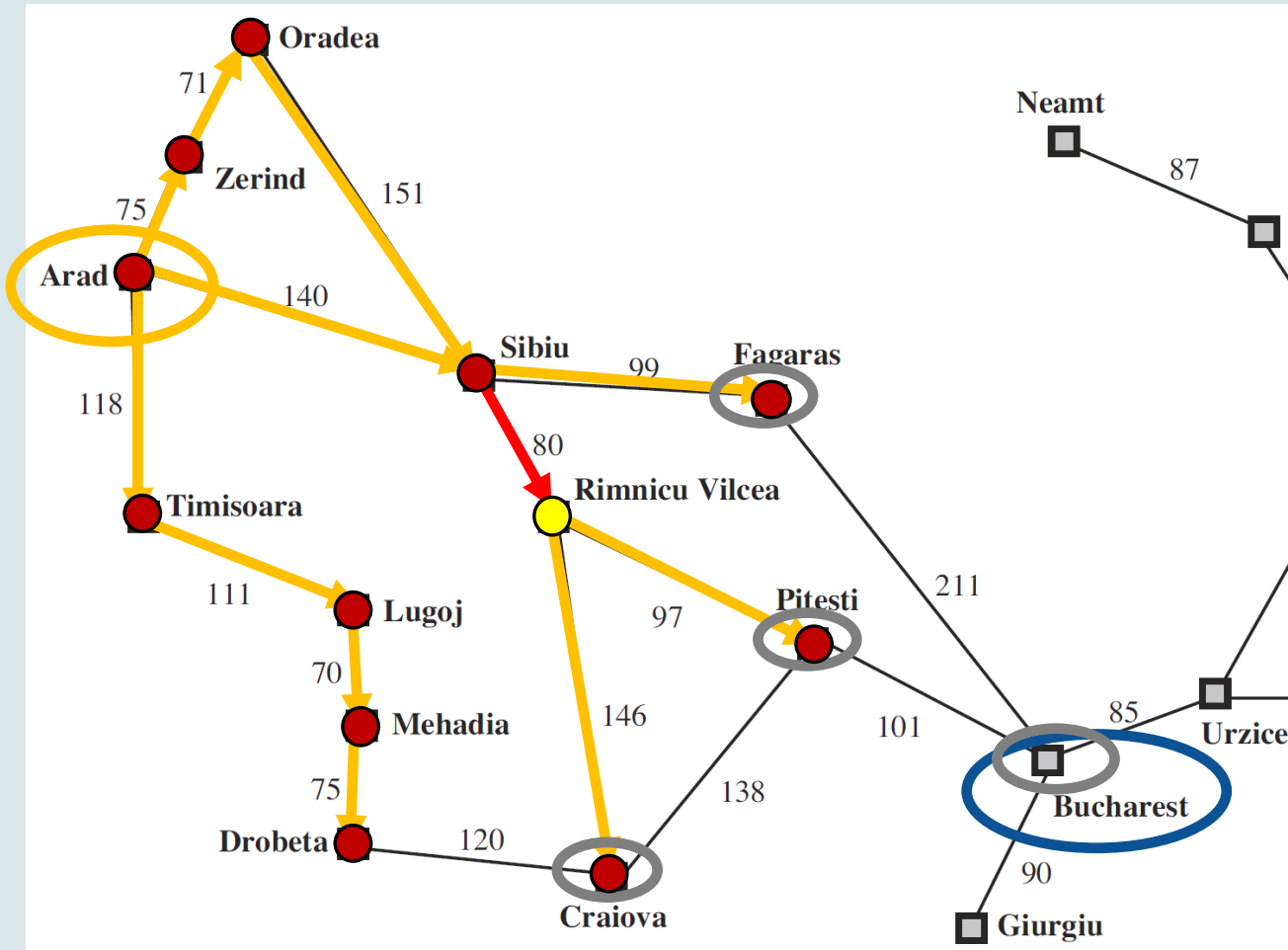


Search Tree

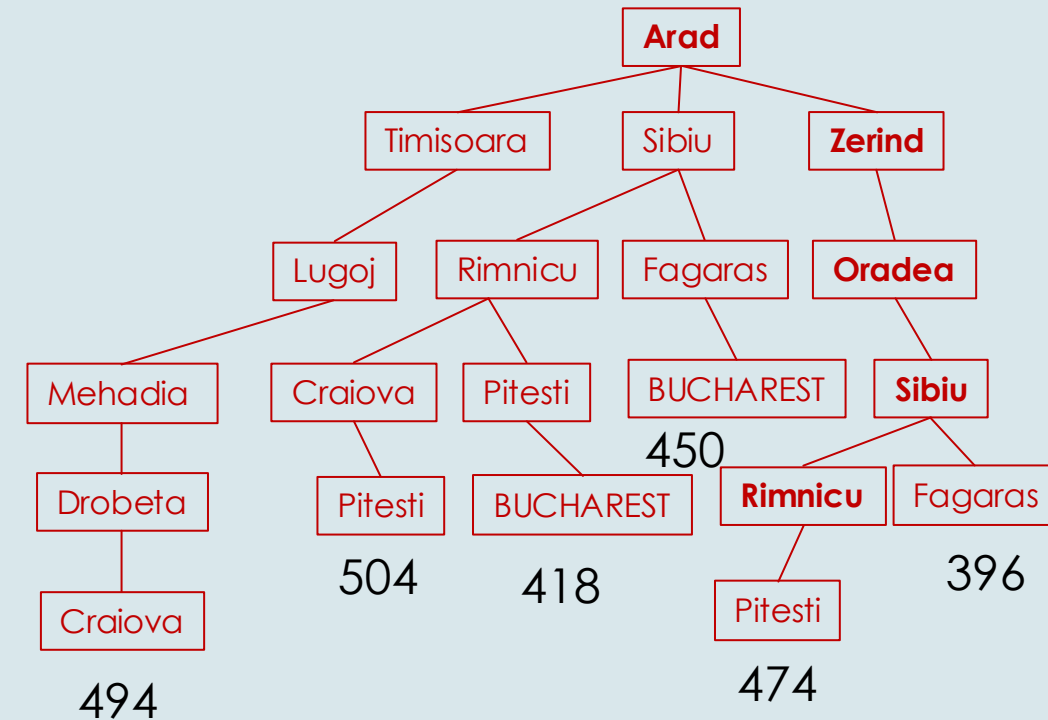


Uniform Cost Search

21

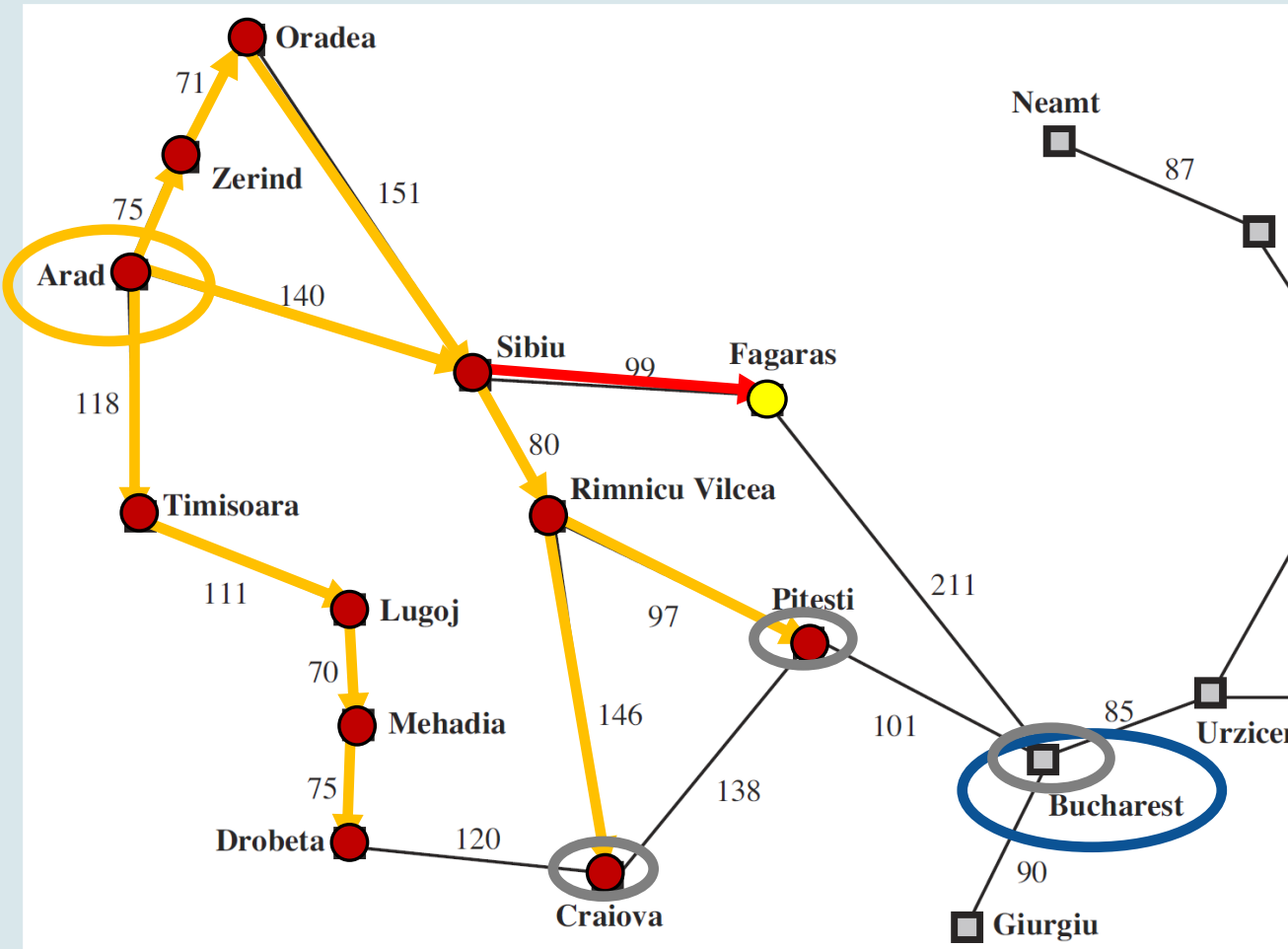


Search Tree

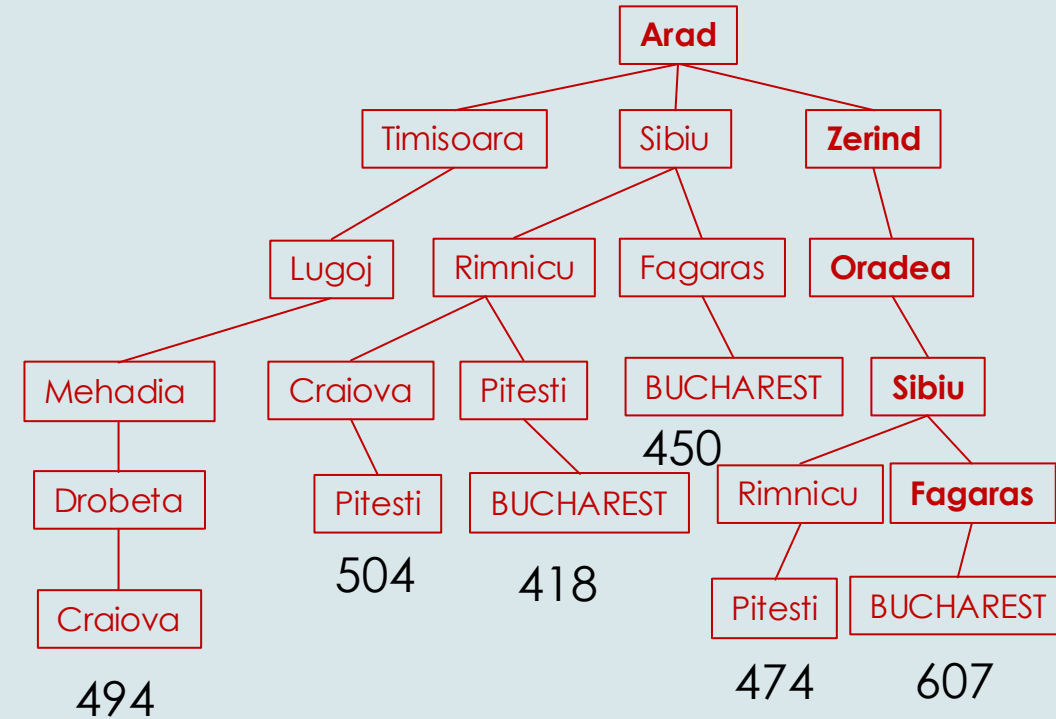


Uniform Cost Search

22

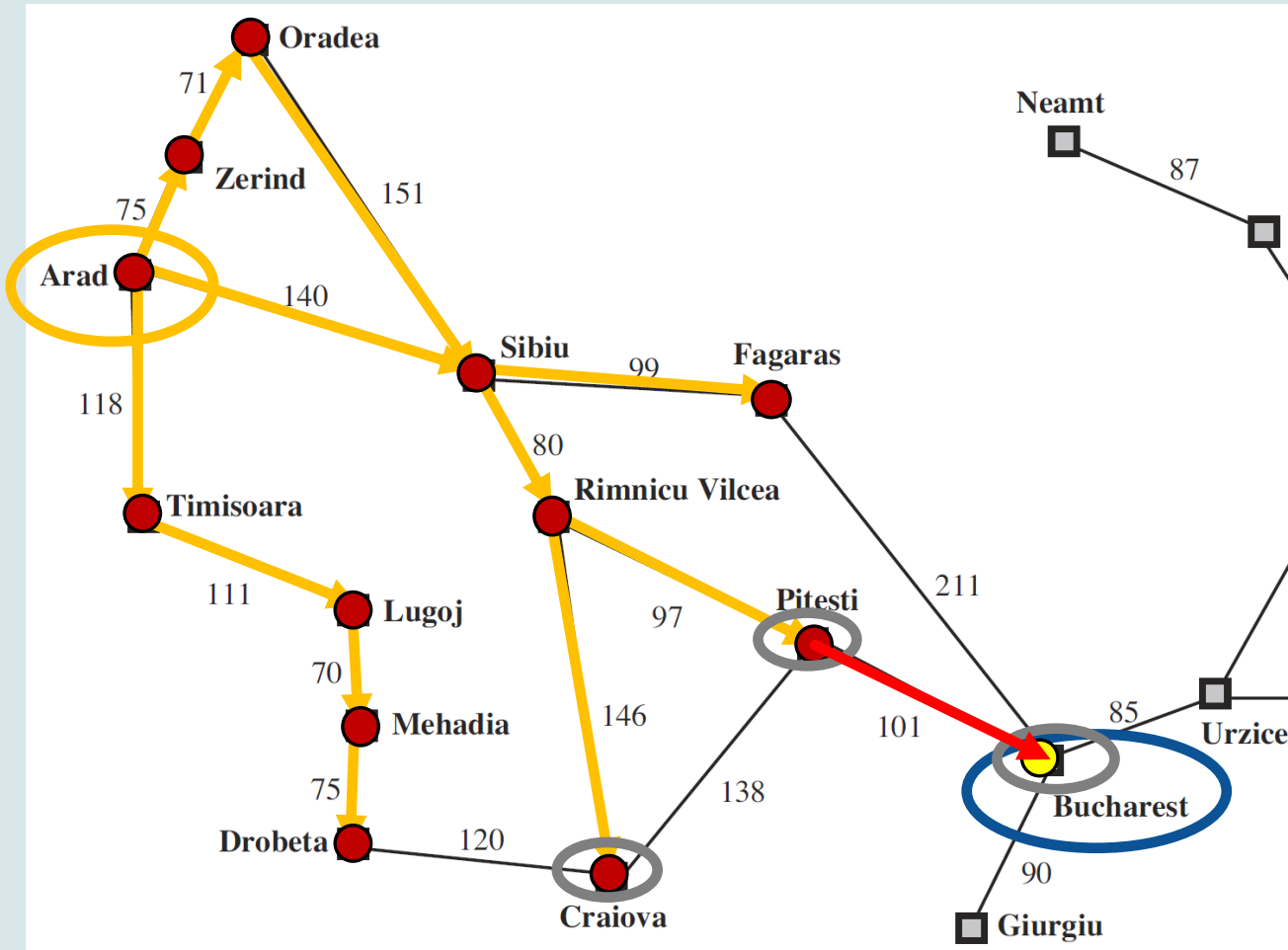


Search Tree

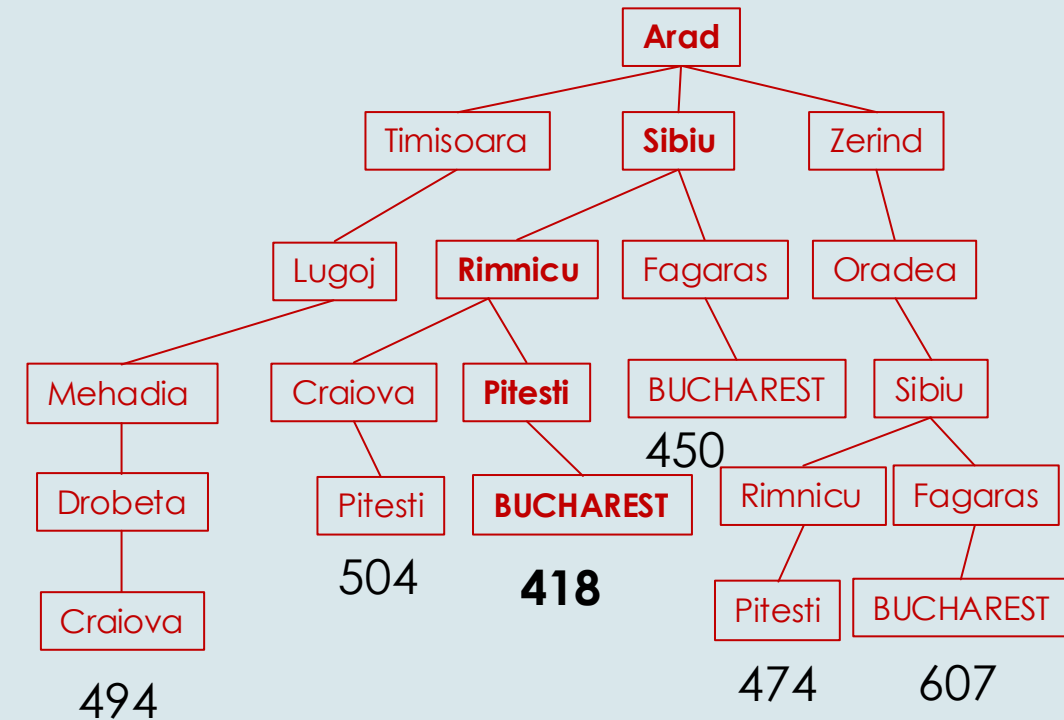


Uniform Cost Search

23

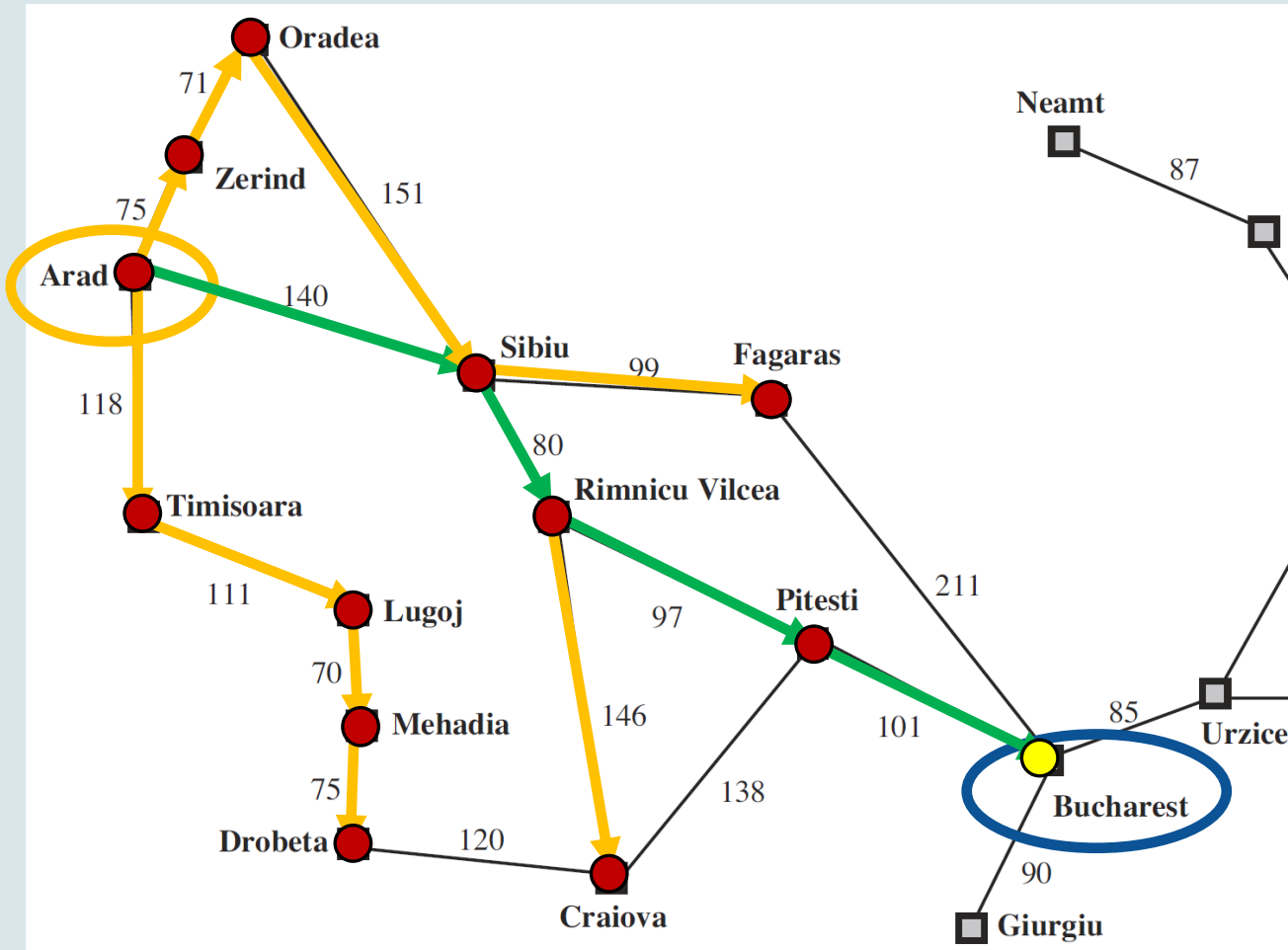


Search Tree

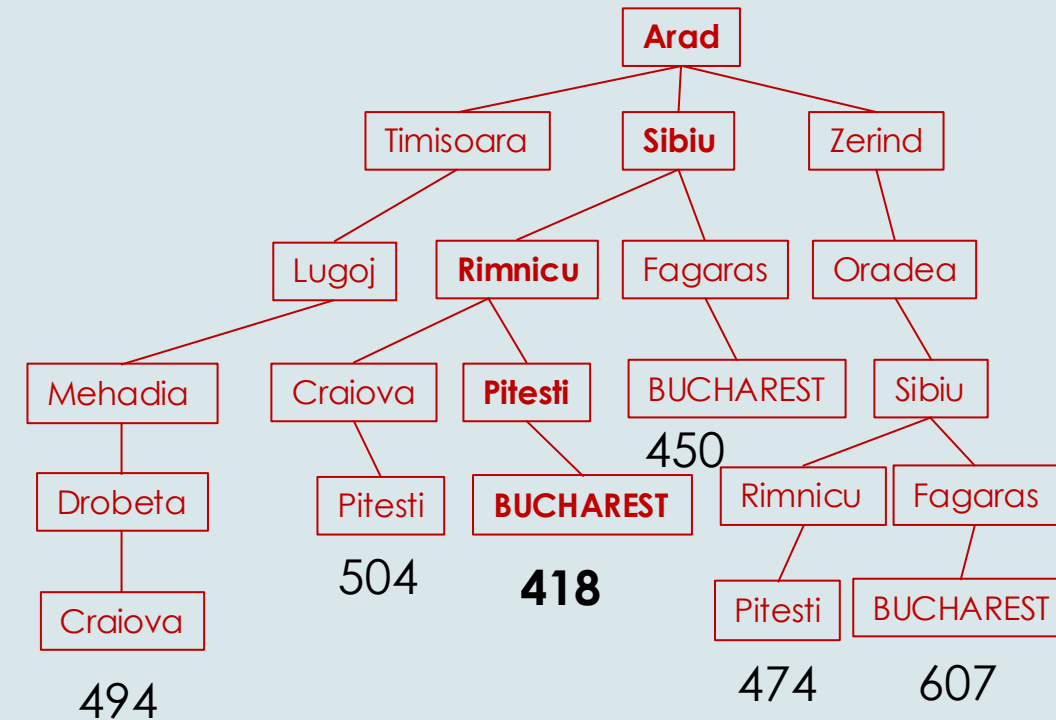


Uniform Cost Search

24



Search Tree

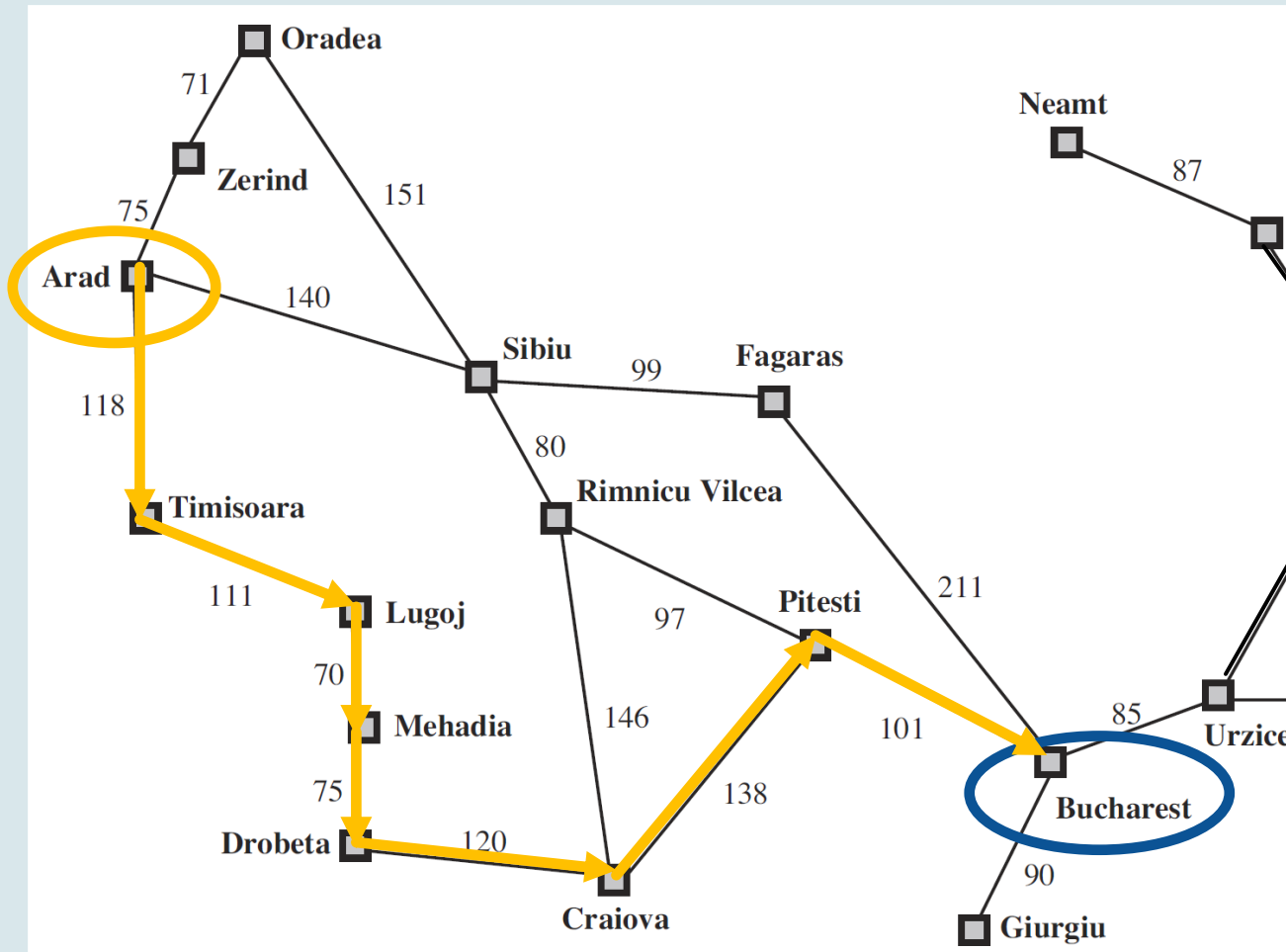


Optimal solution found!

Nodes in memory: 21 Expanded: 13

Solution of Depth-first

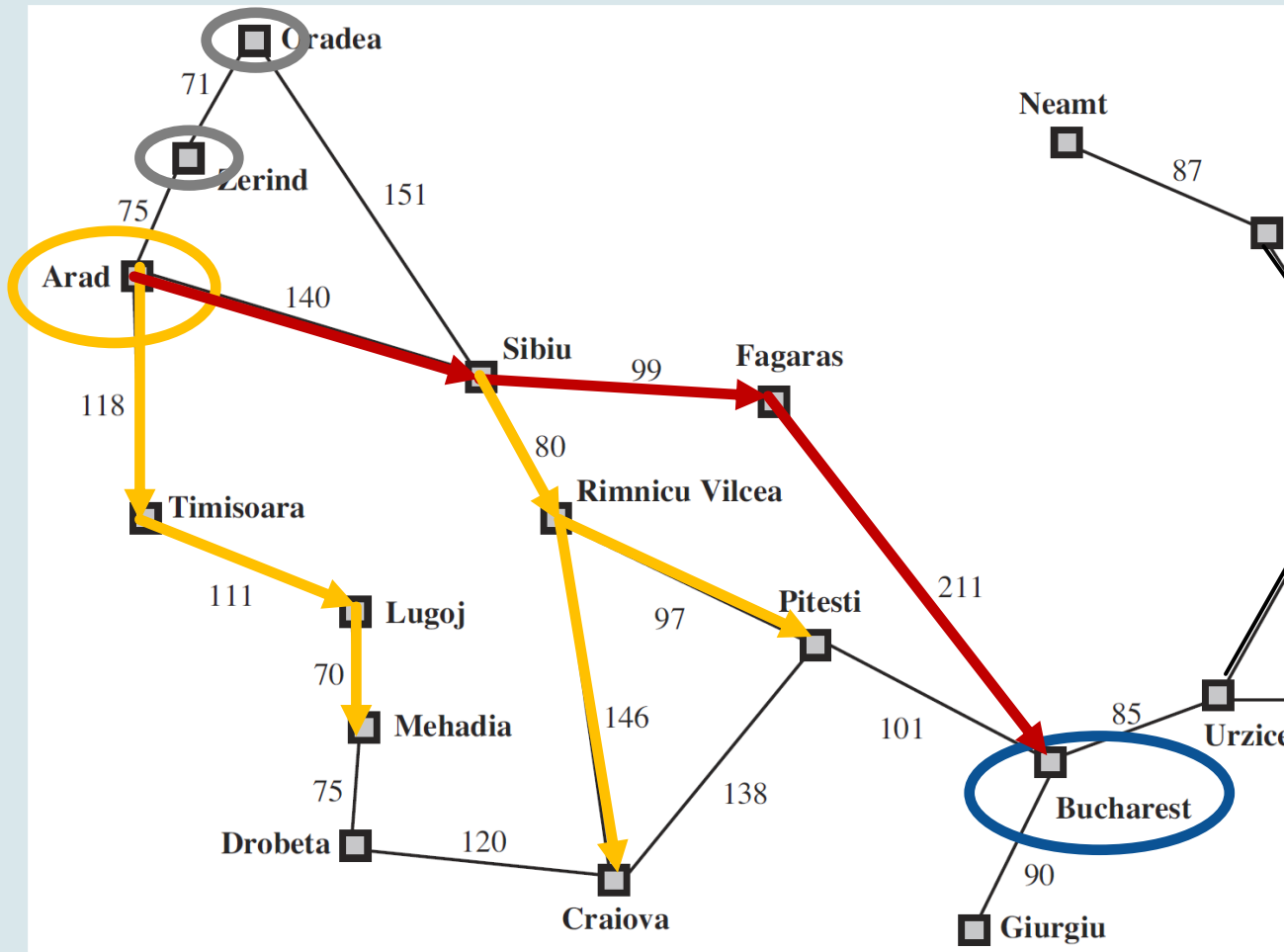
25



Picture: Russell & Norvig, Chapter 3

Solution of Limited Depth-First

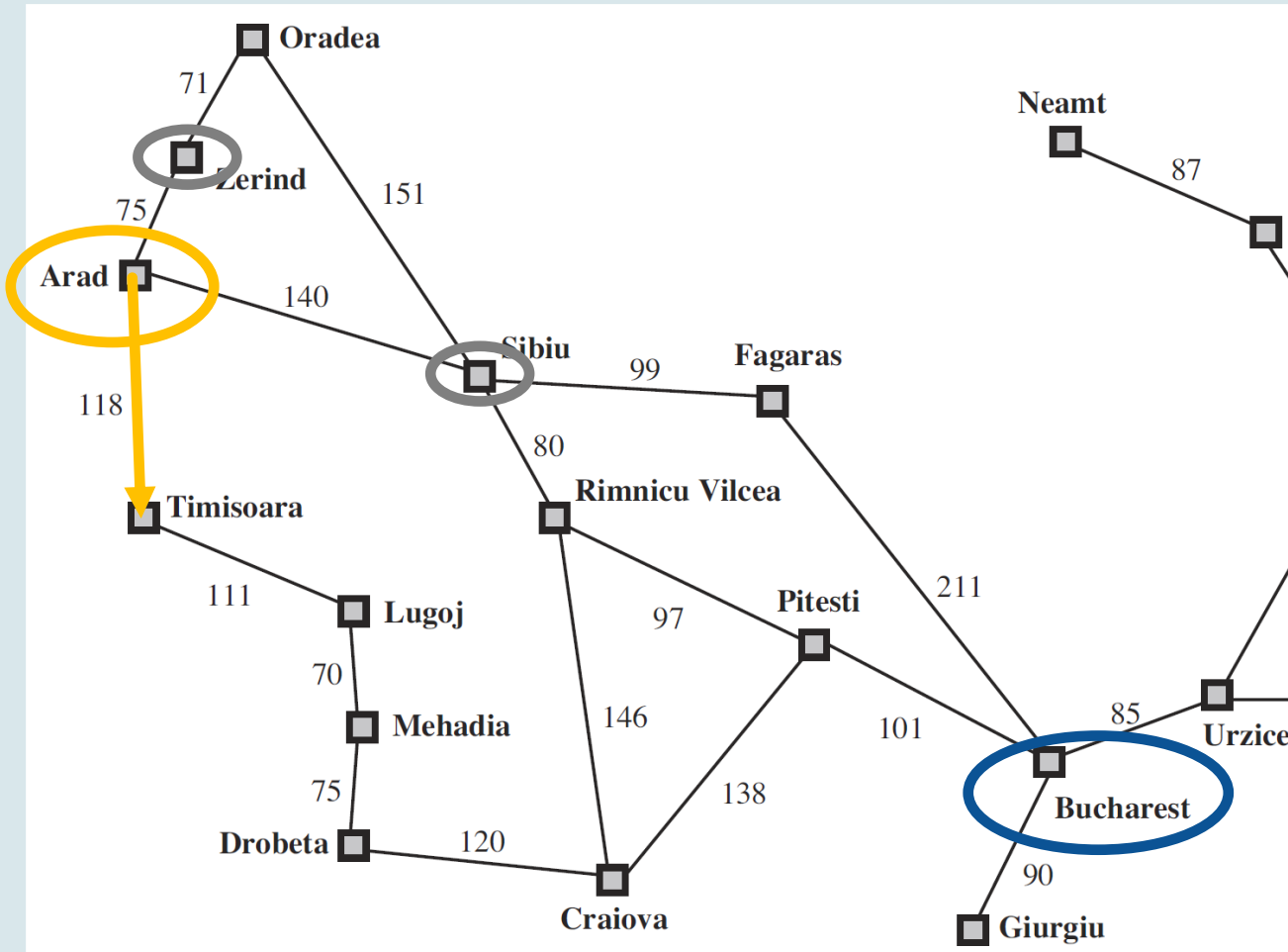
26



Picture: Russell & Norvig, Chapter 3

Iterative Deepening Search

27

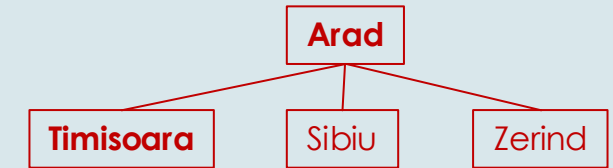


Operations: 1

Depth limit: 1

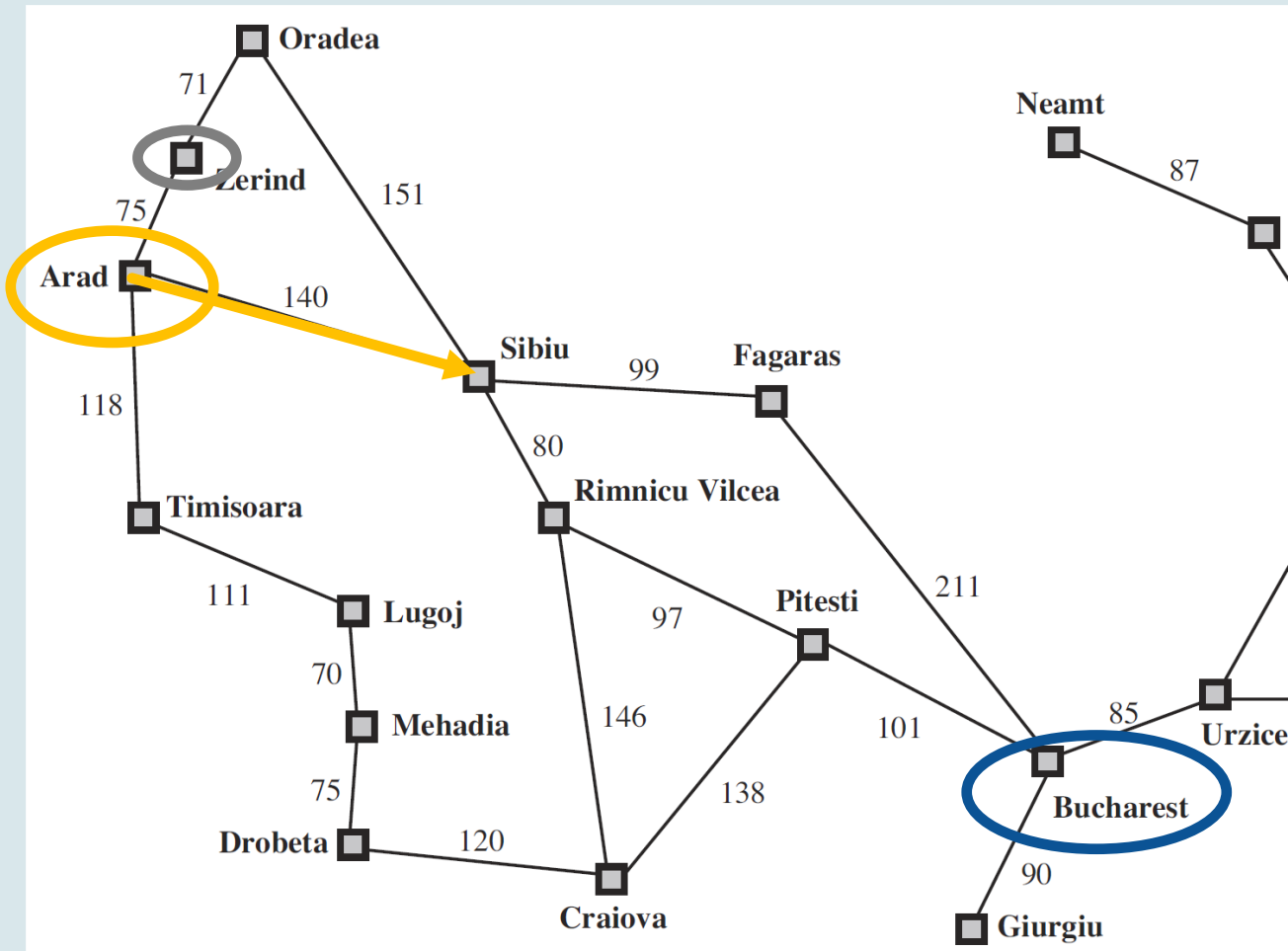
Nodes in
memory: 4

Search Tree



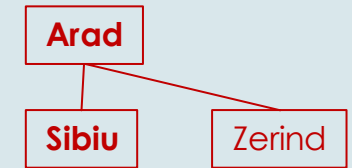
Iterative Deepening Search

28



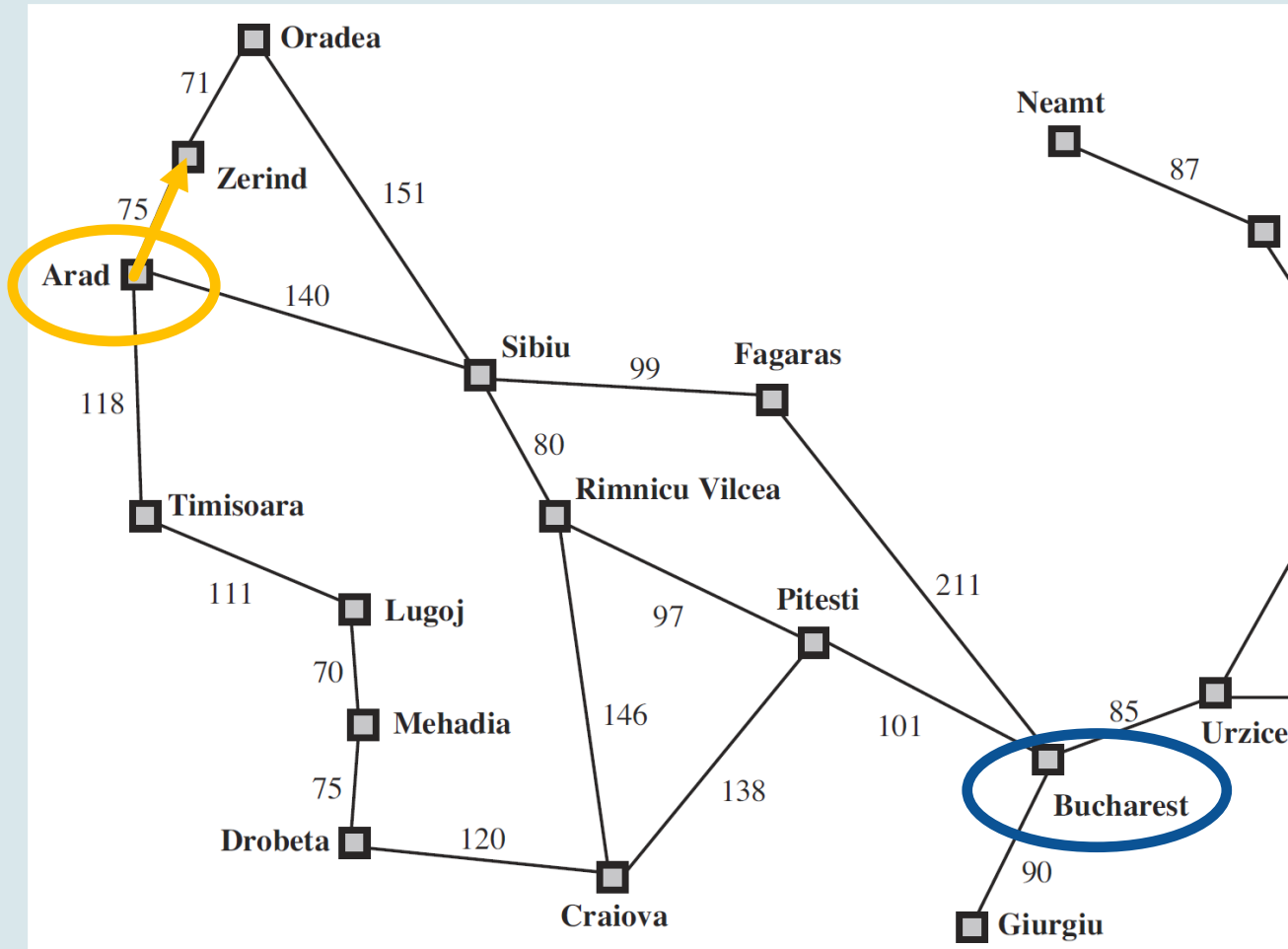
Operations: 2
Depth limit: 1
Nodes in memory: 3

Search Tree



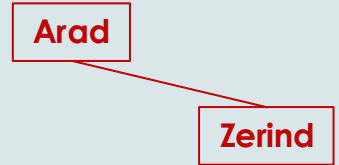
Iterative Deepening Search

29



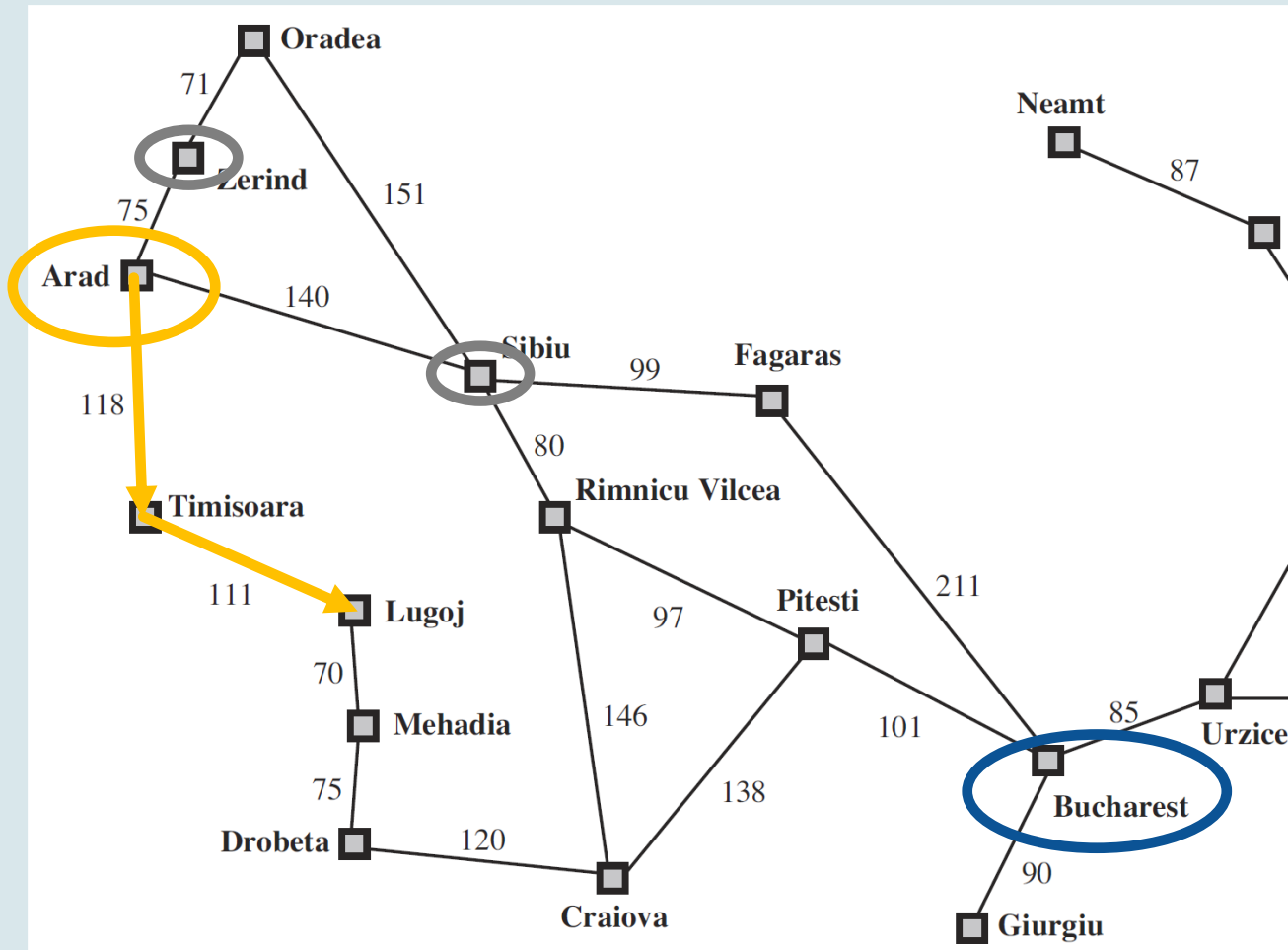
Operations: 3
Depth limit: 1
Nodes in memory: 2

Search Tree



Iterative Deepening Search

30

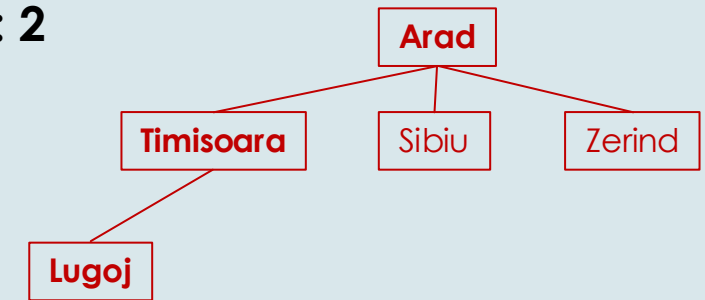


Operations: 5

Depth limit: 2

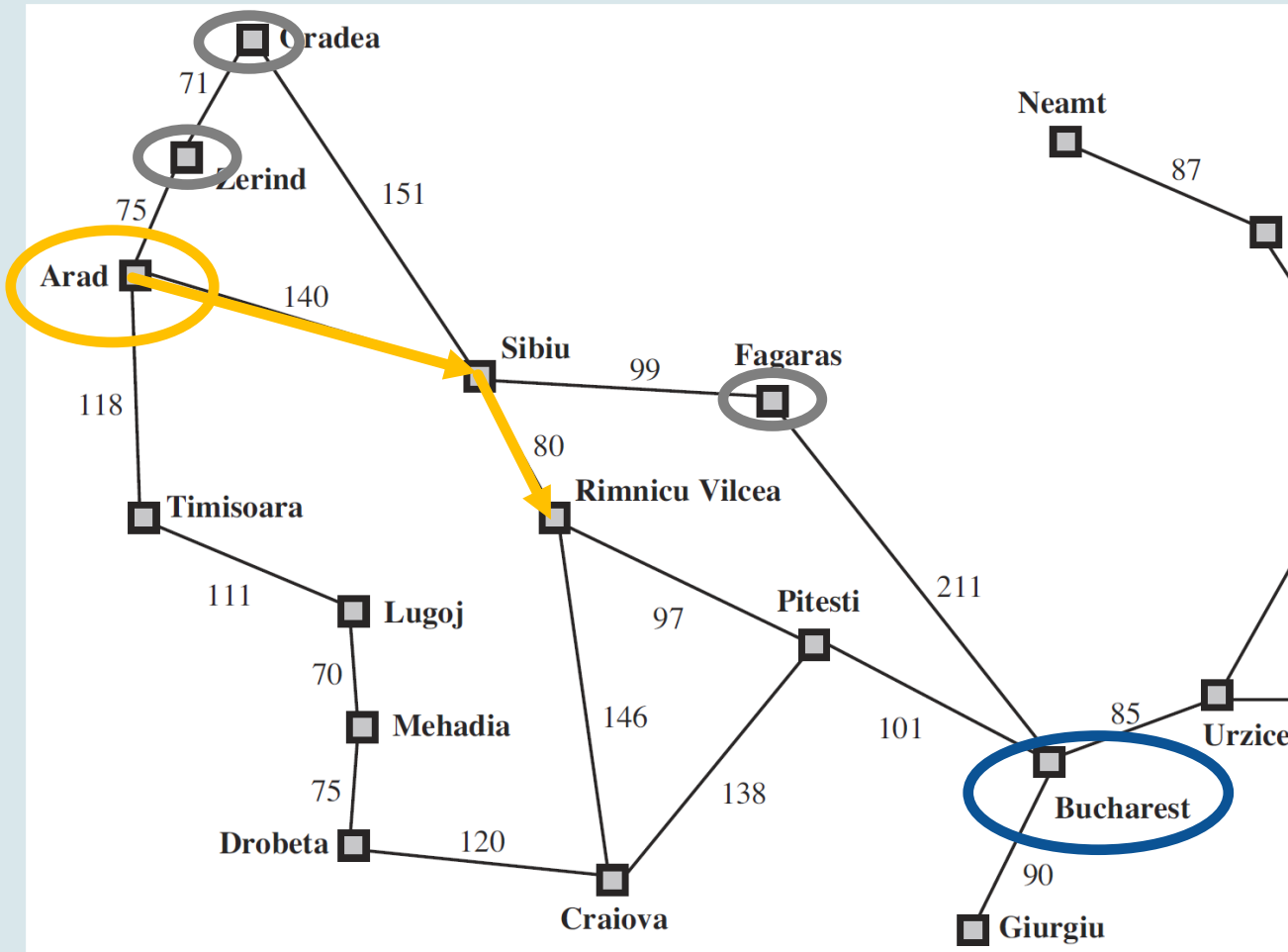
Nodes in
memory: 5

Search Tree



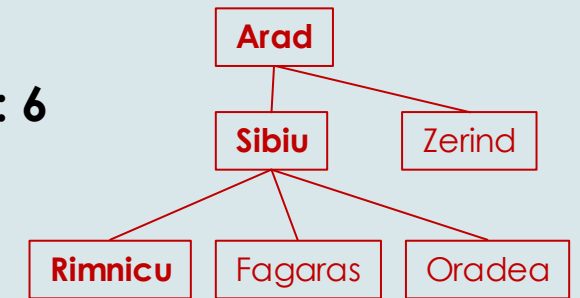
Iterative Deepening Search

31



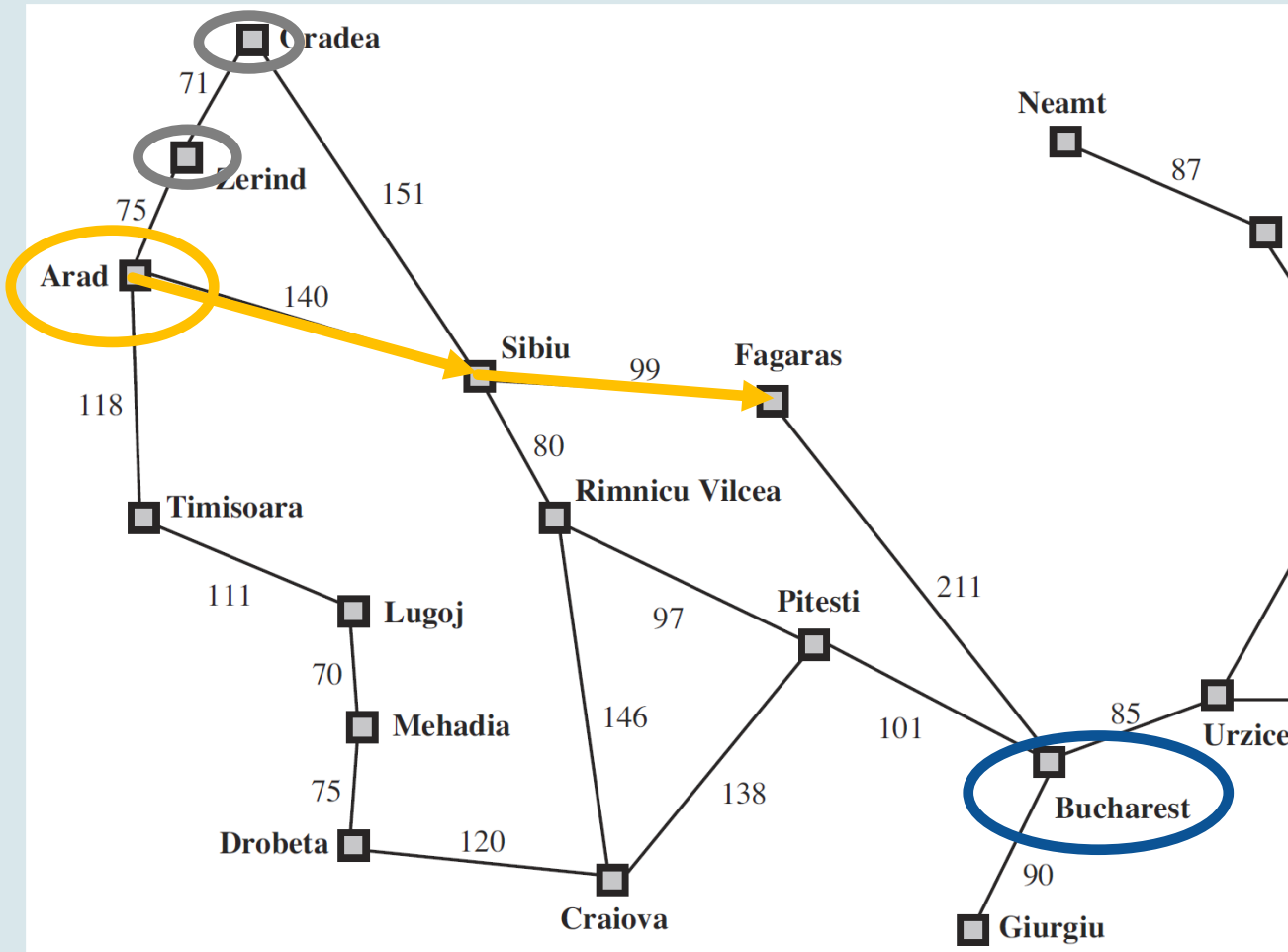
Operations: 7
Depth limit: 2
Nodes in memory: 6

Search Tree



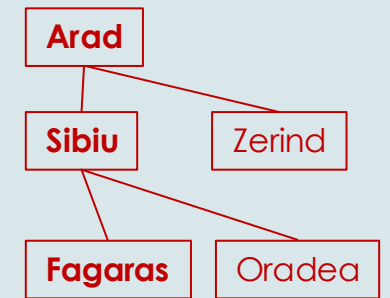
Iterative Deepening Search

32



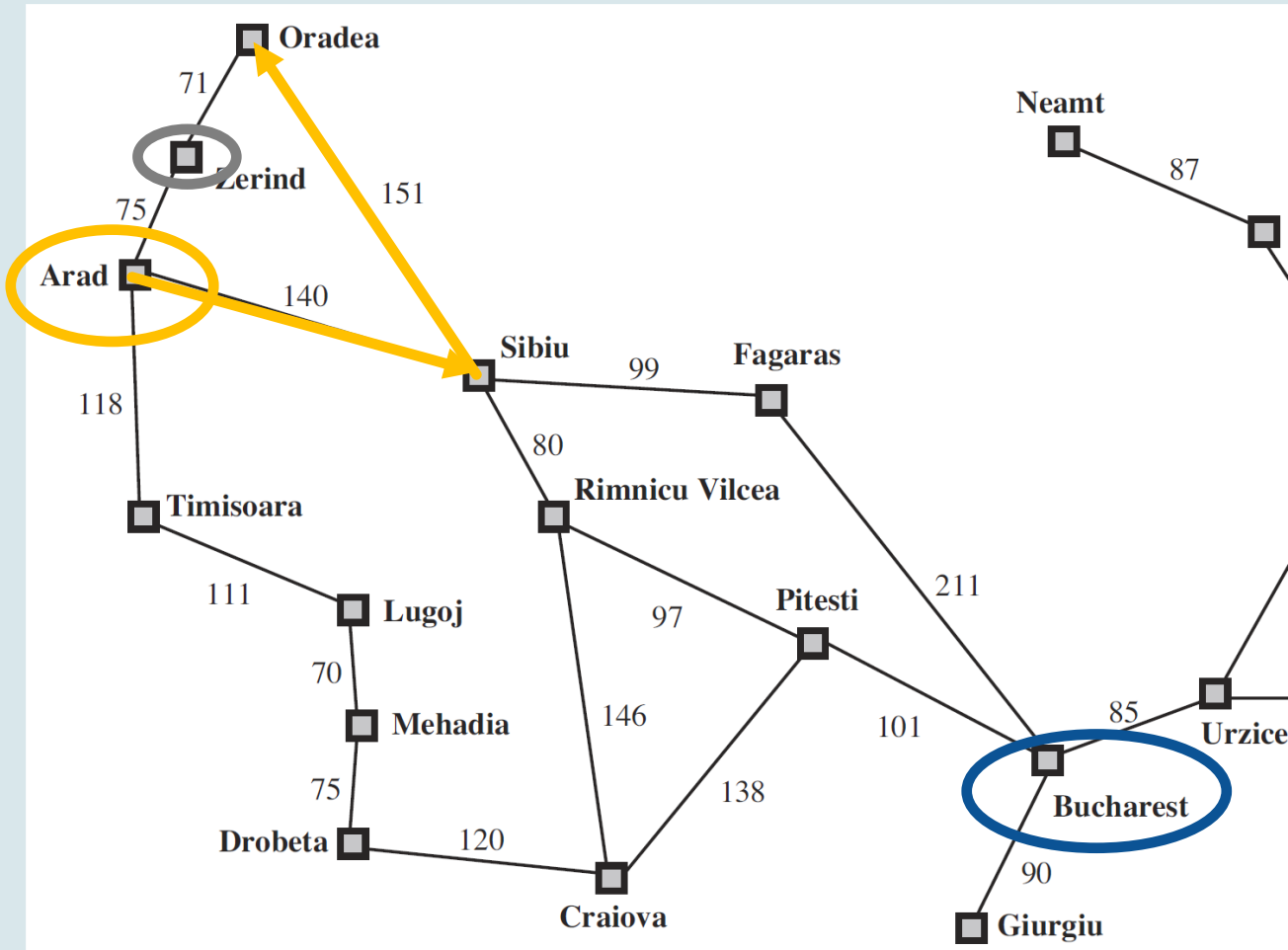
Operations: 8
Depth limit: 2
Nodes in memory: 5

Search Tree



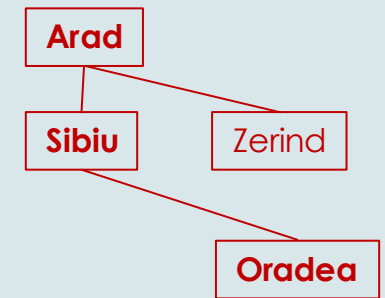
Iterative Deepening Search

33



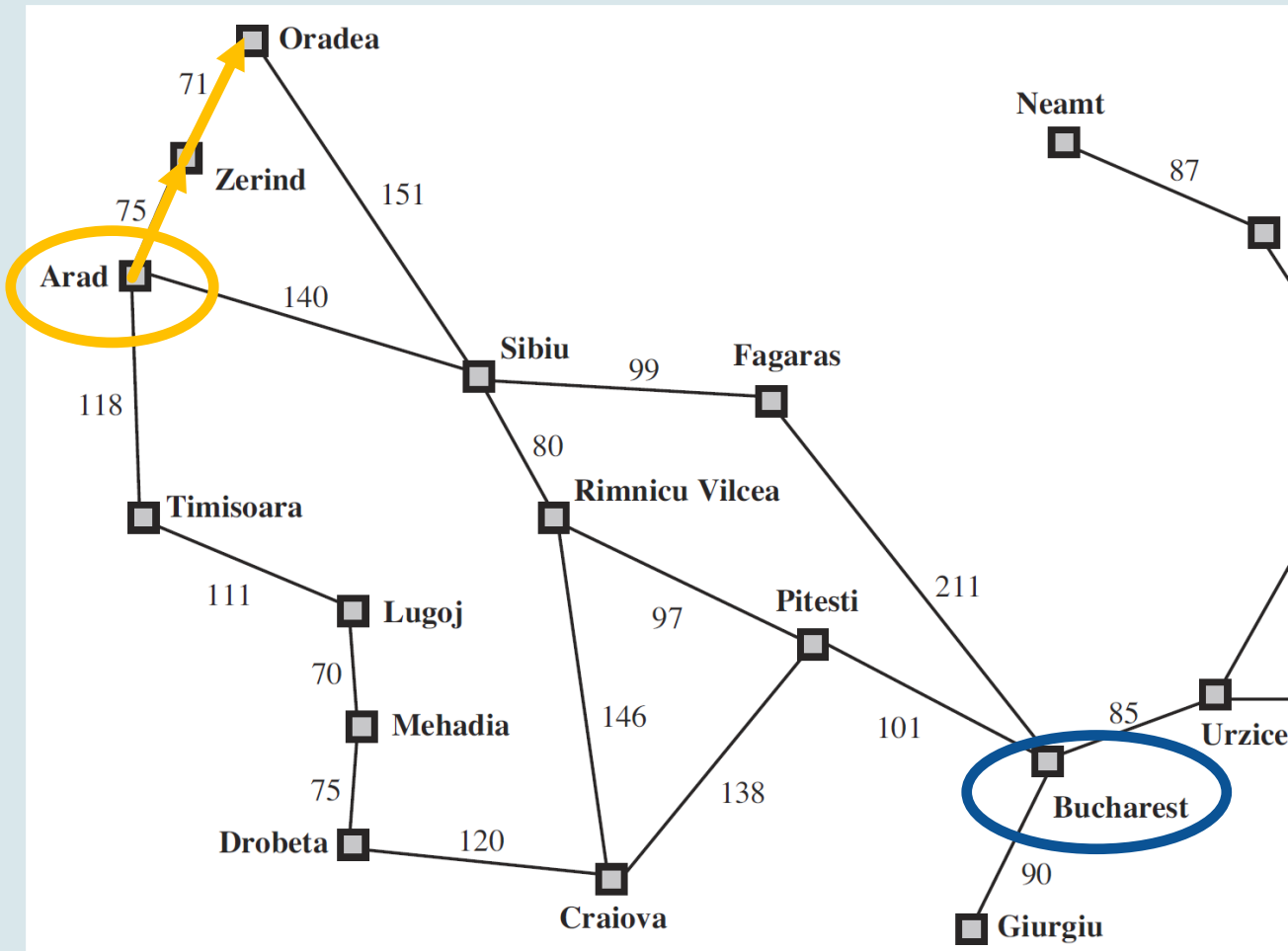
Operations: 9
Depth limit: 2
Nodes in memory: 4

Search Tree



Iterative Deepening Search

34

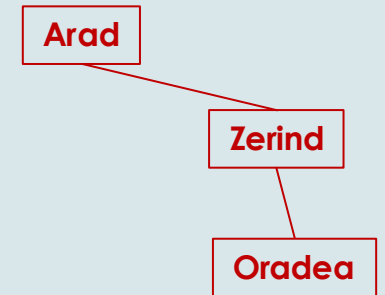


Operations: 11

Depth limit: 2

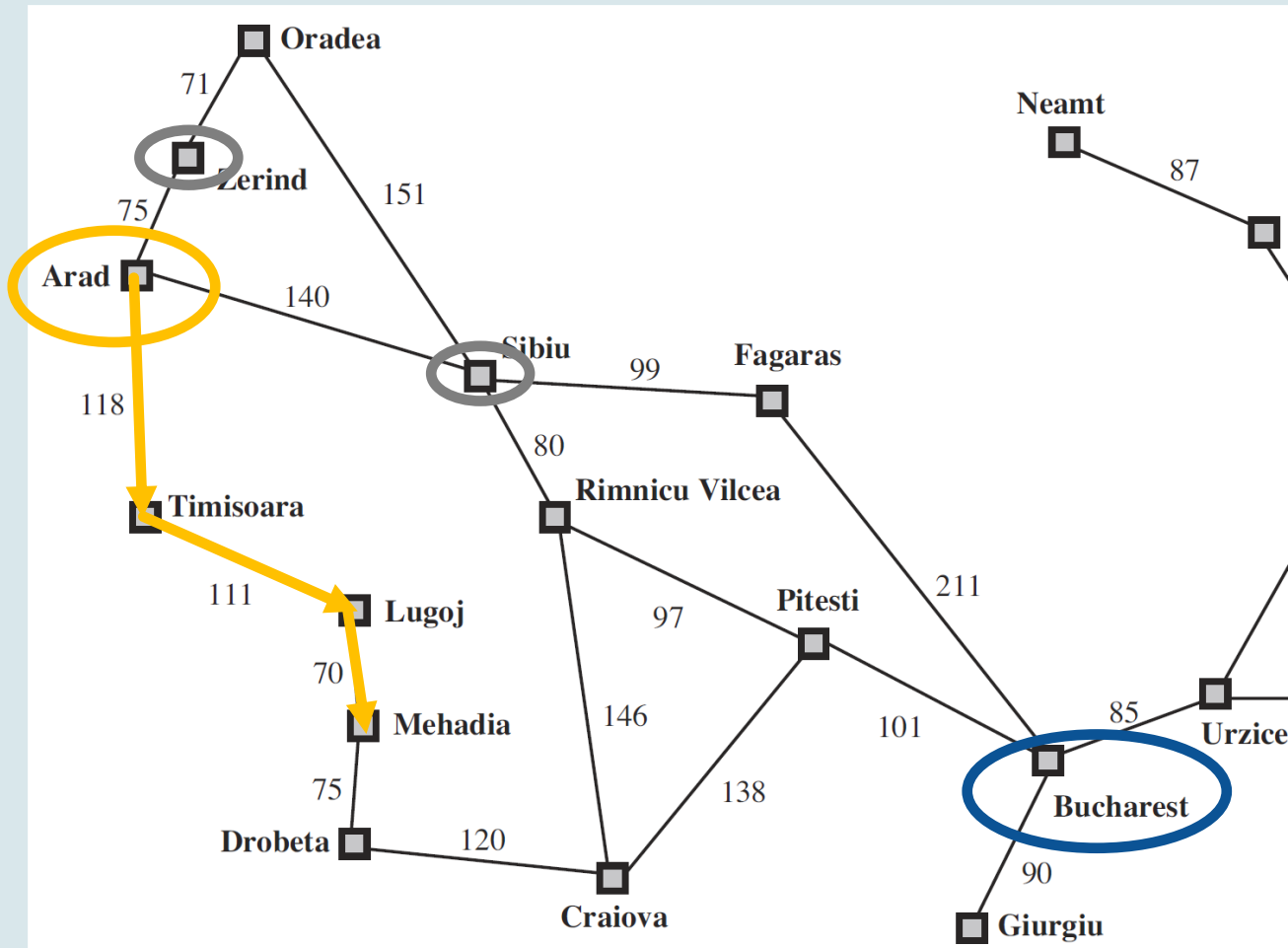
Nodes in memory: 3

Search Tree



Iterative Deepening Search

35

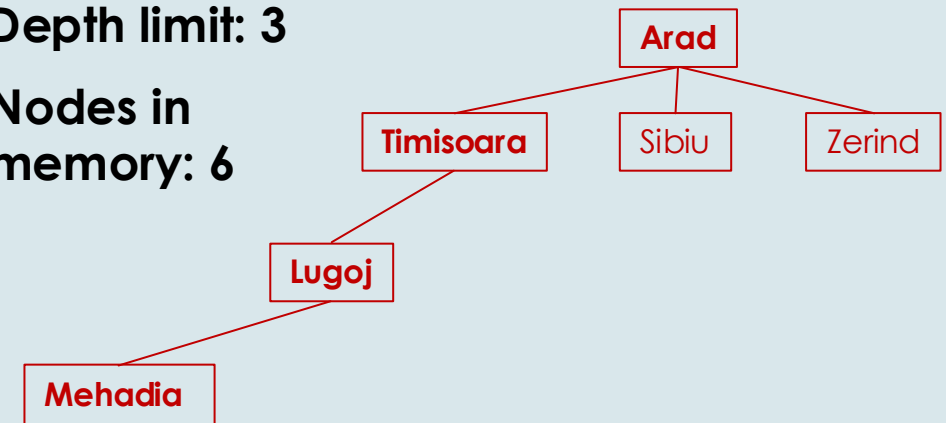


Operations: 14

Depth limit: 3

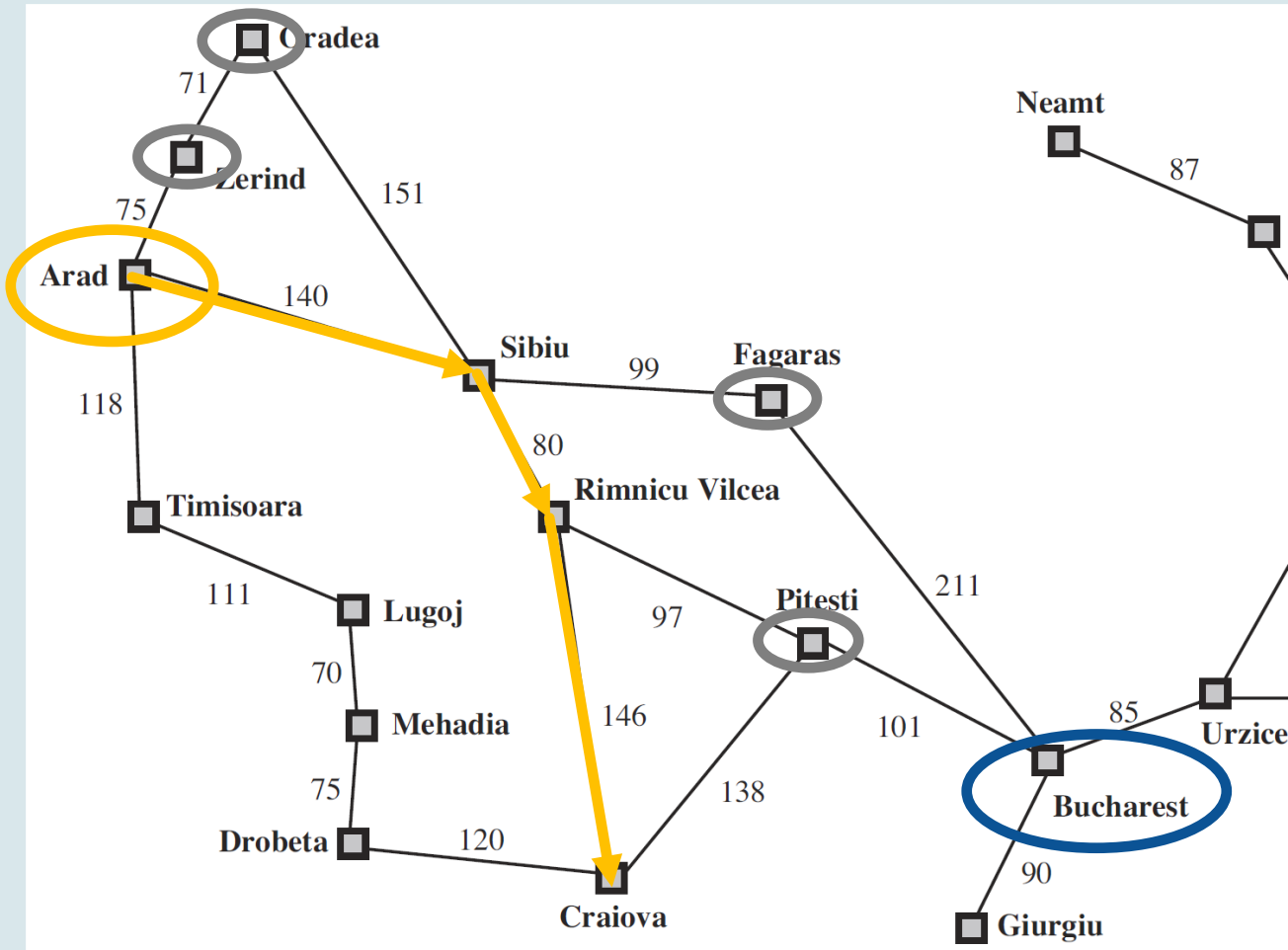
Nodes in
memory: 6

Search Tree



Iterative Deepening Search

36

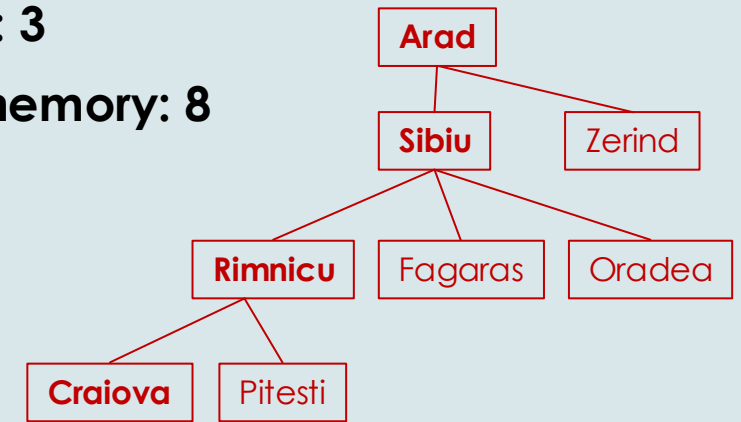


Operations: 17

Depth limit: 3

Nodes in memory: 8

Search Tree



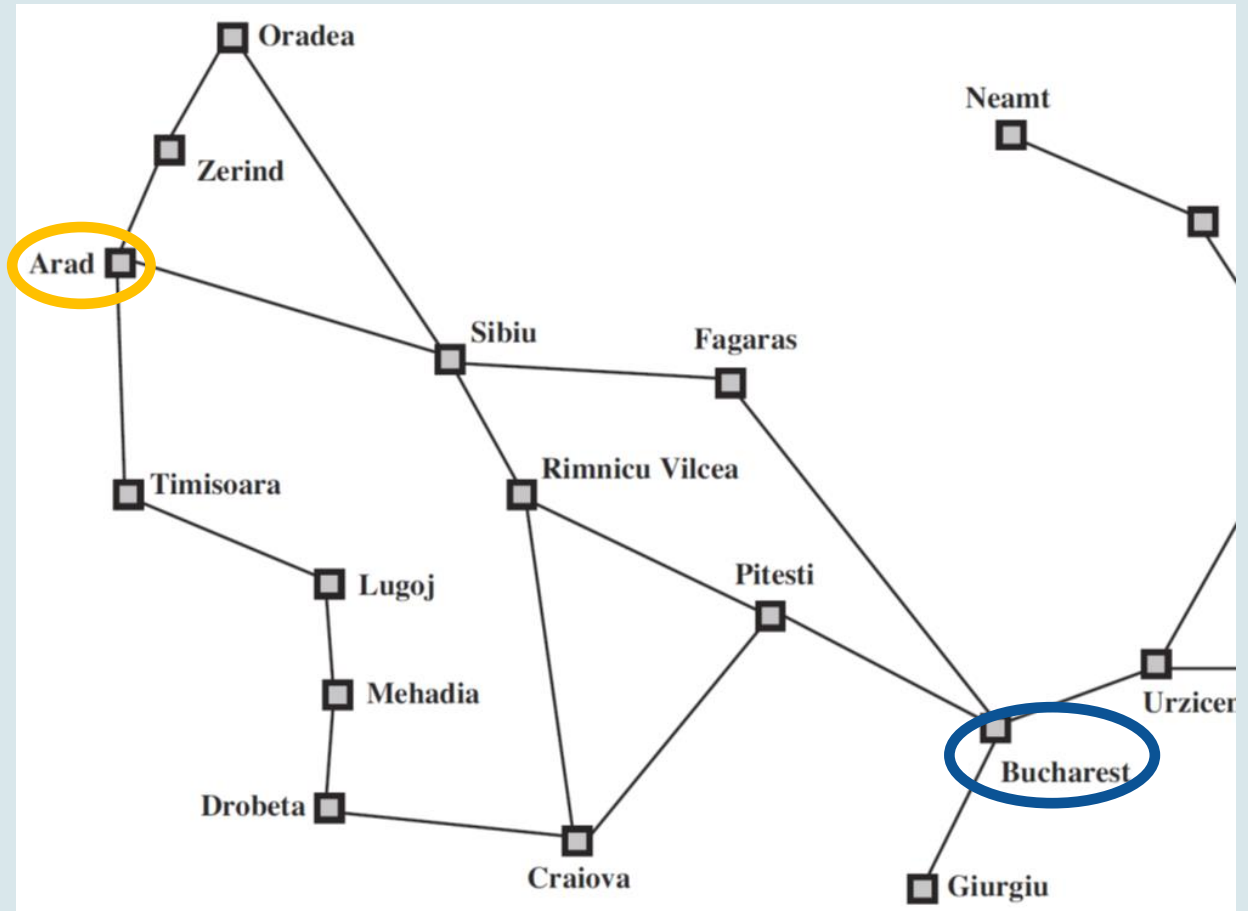
Heuristic Search

Greedy Best-First Search

- In greedy best-first search, the evaluation function contains a simple heuristic

$$f(n) = h(n)$$

that estimates the cost of the cheapest path from the state at the current node to the node that has the goal state.



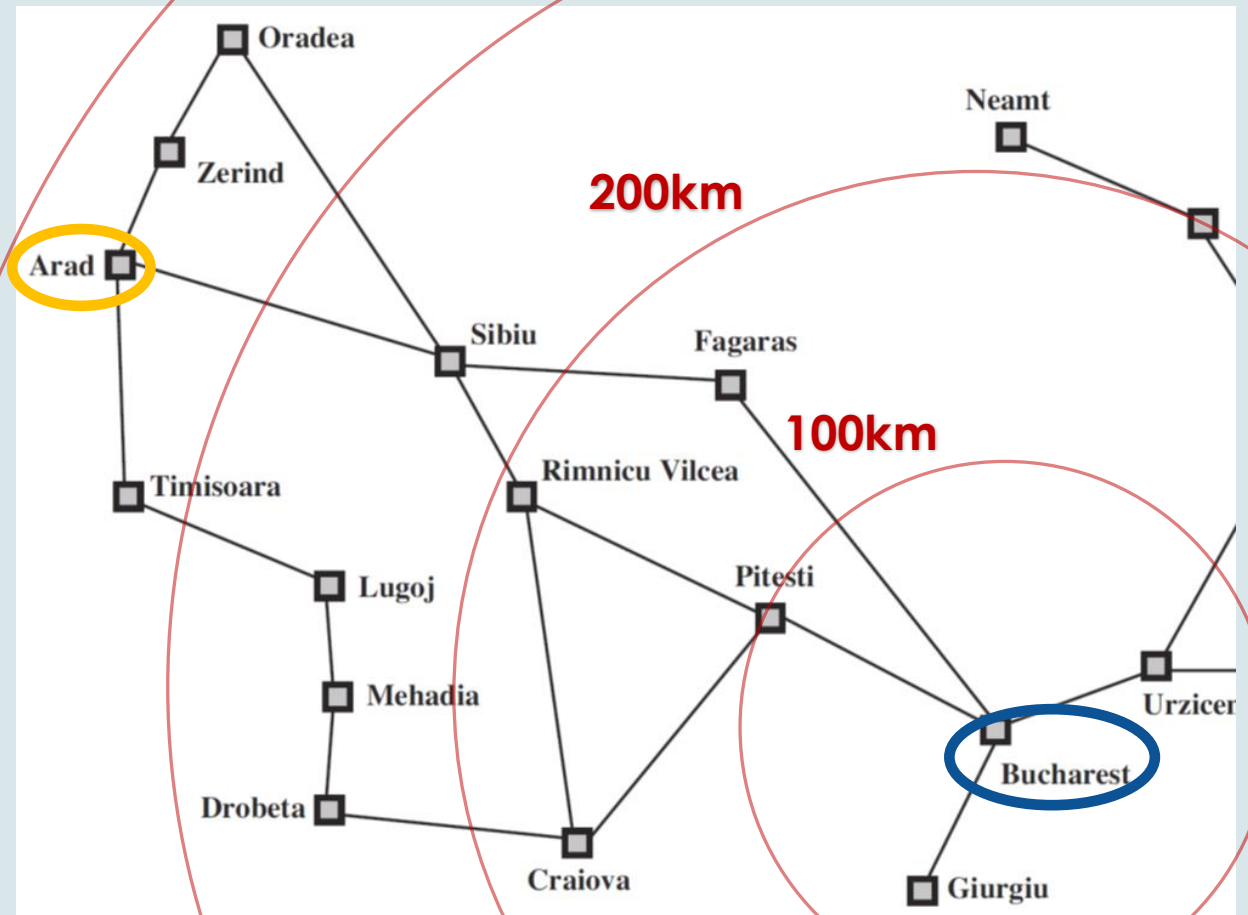
Heuristic Search

Greedy Best-First Search

- In greedy best-first search, the evaluation function contains a simple heuristic

$$f(n) = h(n)$$

that estimates the cost of the cheapest path from the state at the current node to the node that has the goal state (straight-line distance).



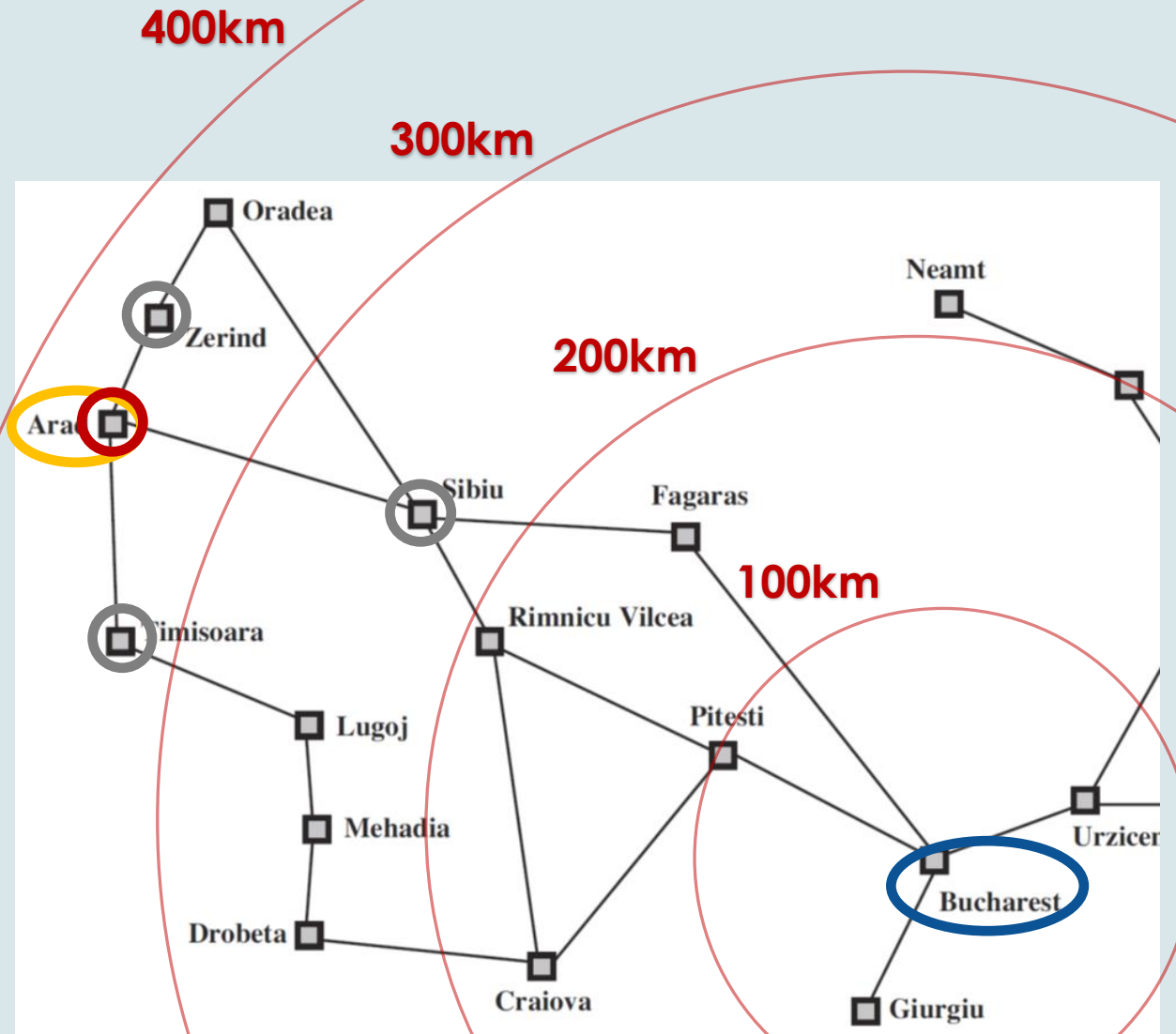
Heuristic Search

Greedy Best-First Search

- In greedy best-first search, the evaluation function contains a simple heuristic

$$f(n) = h(n)$$

that estimates the cost of the cheapest path from the state at the current node to the node that has the goal state (straight-line distance).



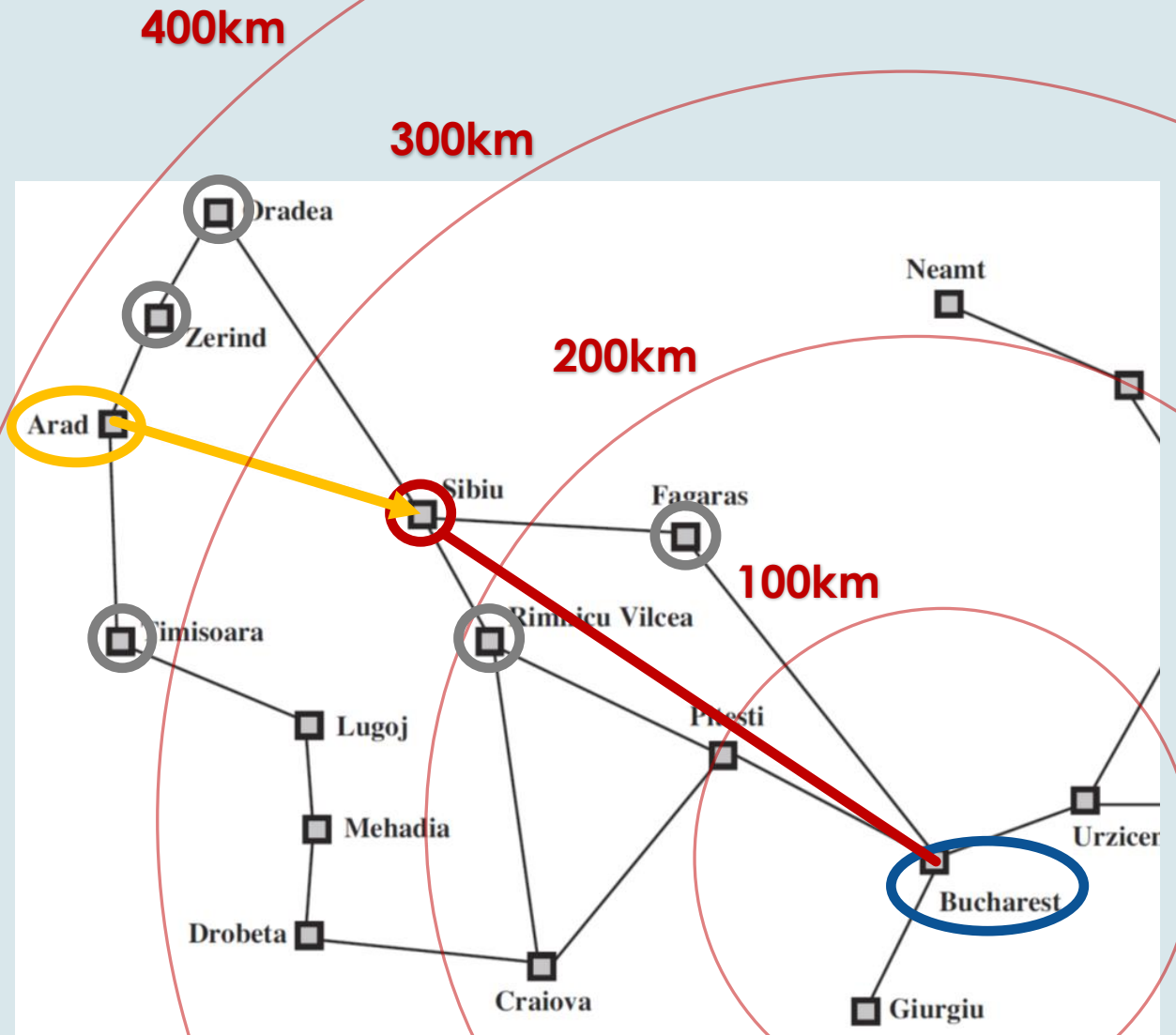
Heuristic Search

Greedy Best-First Search

- In greedy best-first search, the evaluation function contains a simple heuristic

$$f(n) = h(n)$$

that estimates the cost of the cheapest path from the state at the current node to the node that has the goal state (straight-line distance).



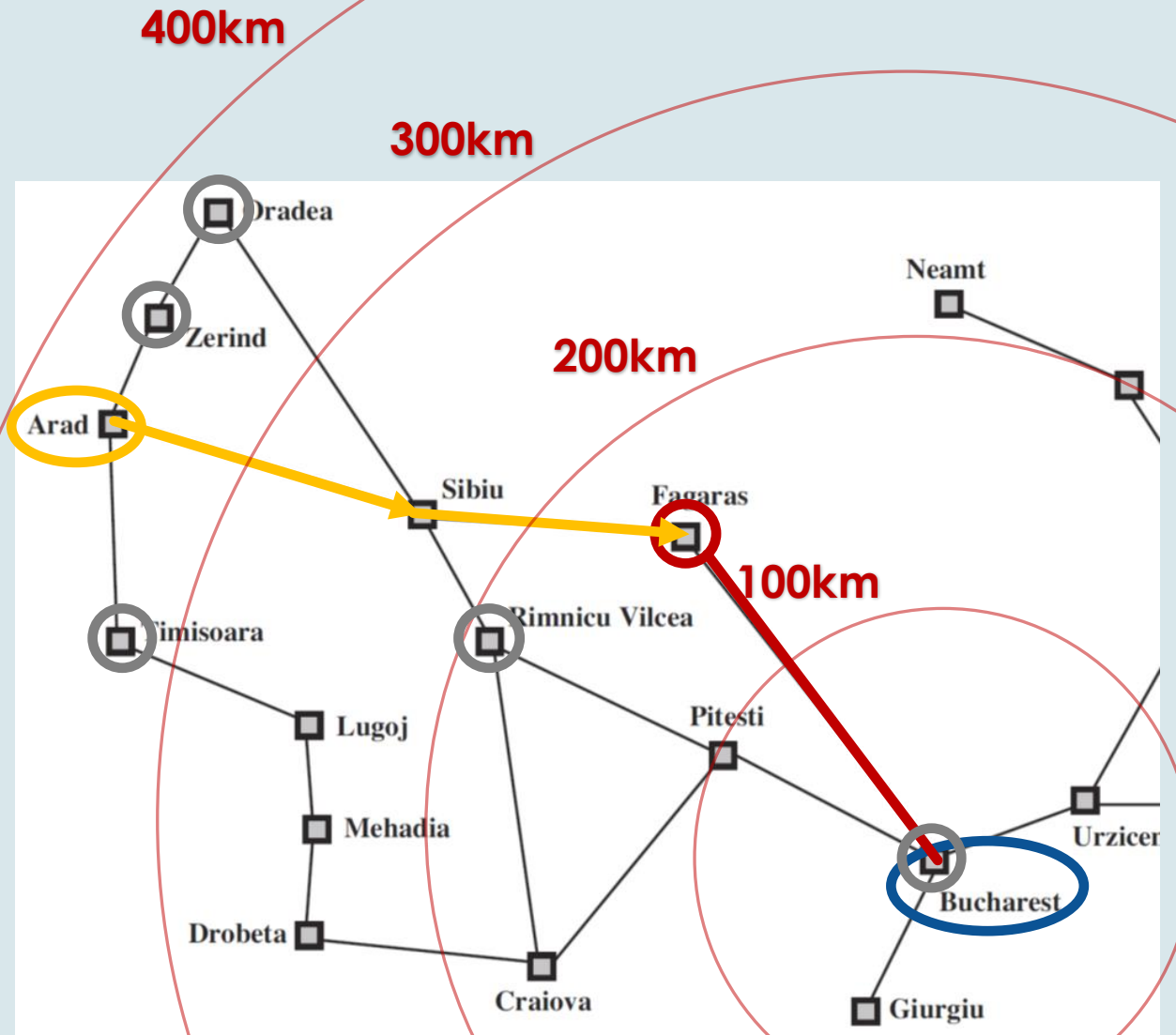
Heuristic Search

Greedy Best-First Search

- In greedy best-first search, the evaluation function contains a simple heuristic

$$f(n) = h(n)$$

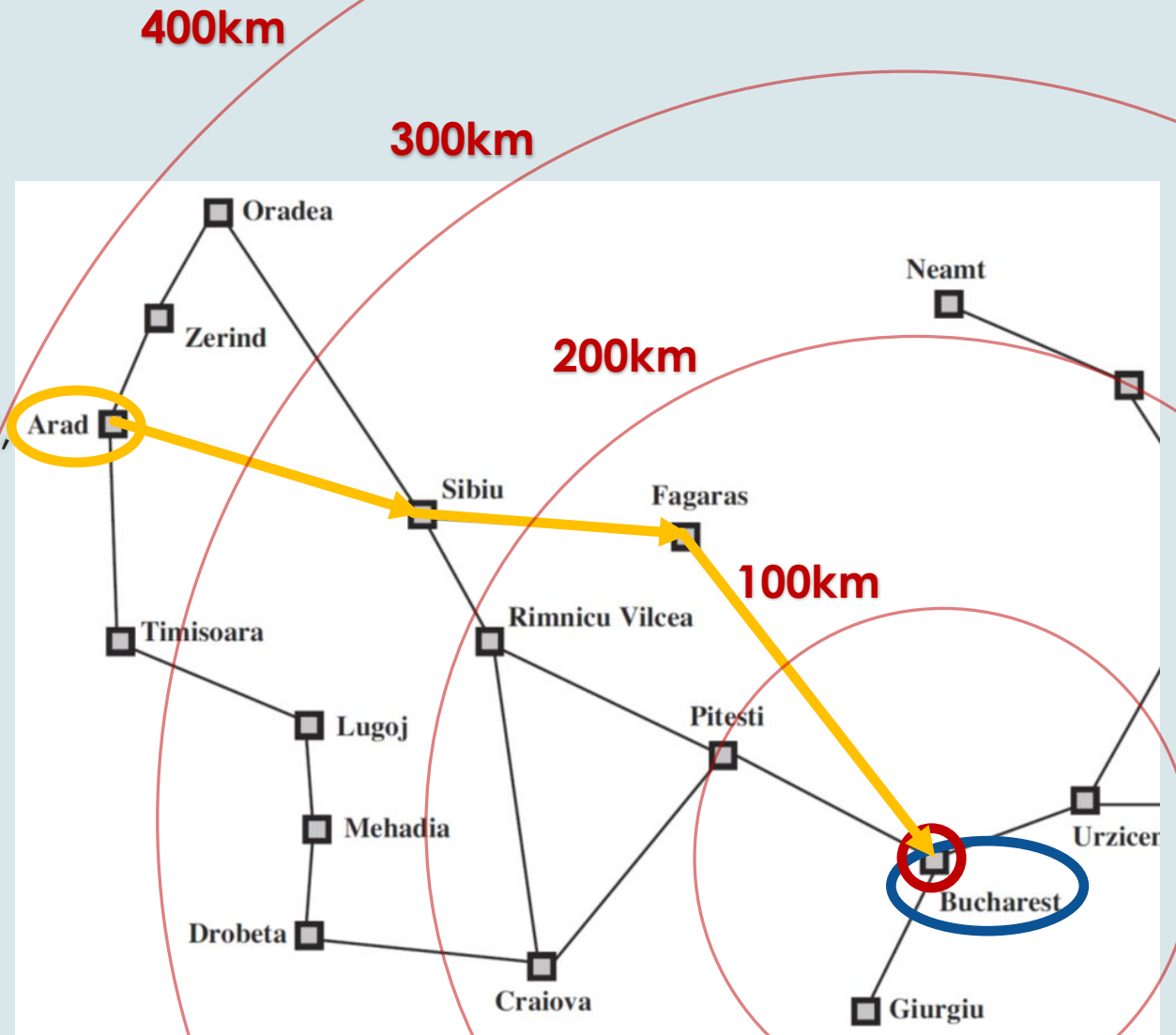
that estimates the cost of the cheapest path from the state at the current node to the node that has the goal state (straight-line distance).



Heuristic Search

Greedy Best-First Search

- Found a direct path for this problem
- Is the greedy strategy guaranteed to find the shortest path to Bucharest, i.e. an **optimal** solution?



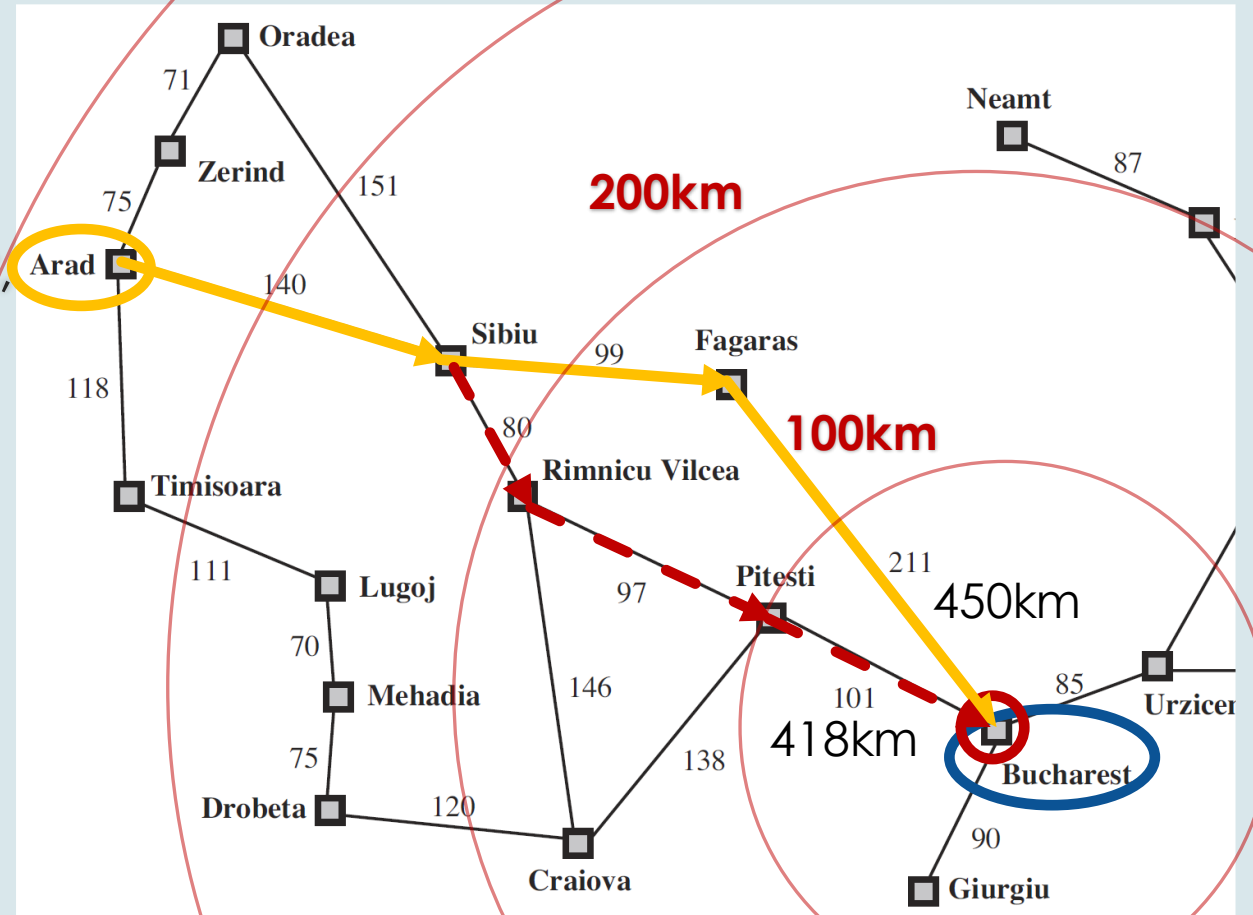
Heuristic Search

Greedy Best-First Search

- Found a direct path for this problem
- Is the greedy strategy guaranteed to find the shortest path to Bucharest, i.e. an **optimal** solution?

NO. There is another, optimal solution, and it was missed.

- Is the algorithm **complete**?



Heuristic Search

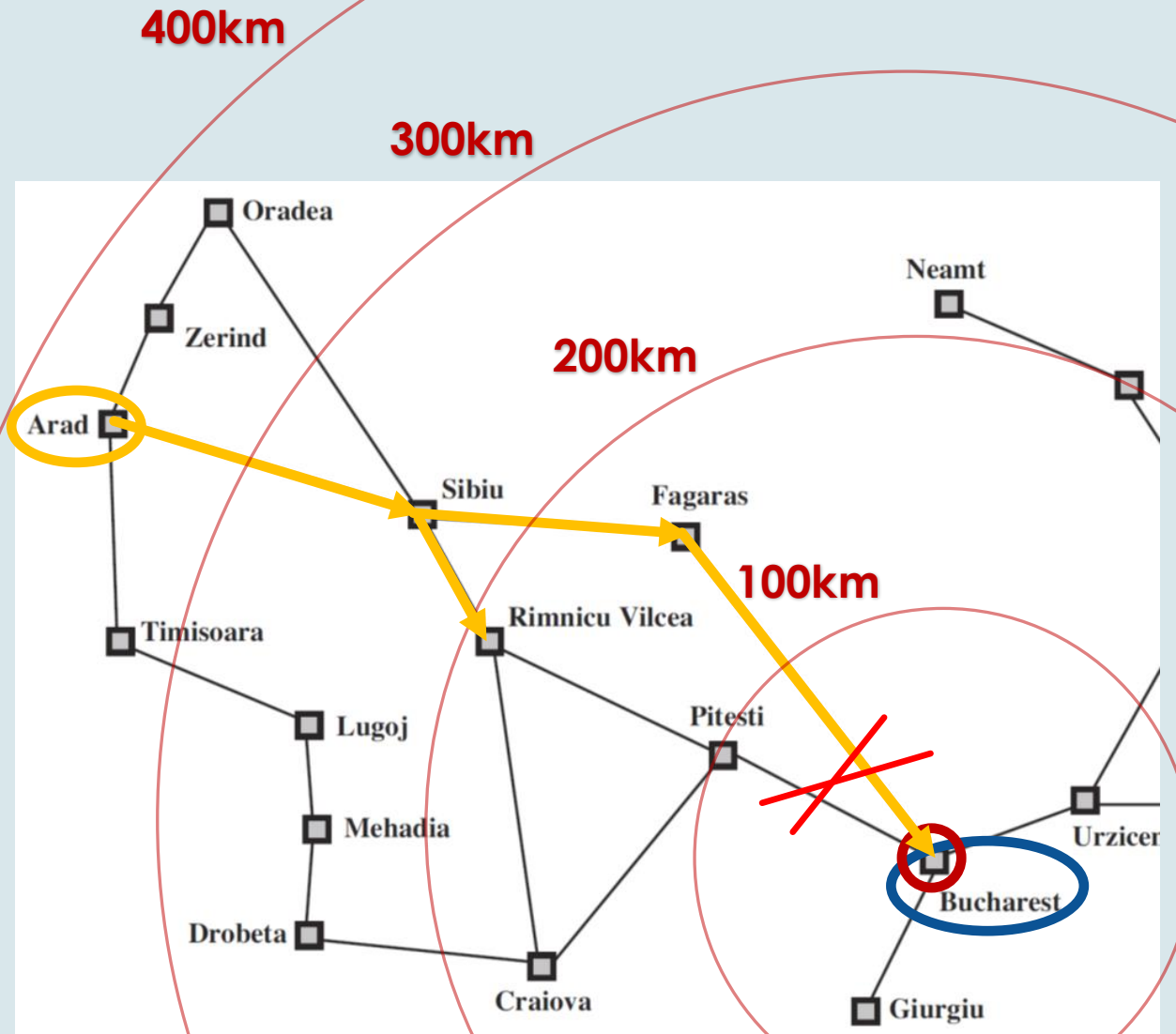
Greedy Best-First Search

- Found a direct path for this problem
- Is the greedy strategy guaranteed to find the shortest path to Bucharest, i.e. an **optimal** solution?

NO. There is another, optimal solution, and it was missed.

- Is the algorithm **complete**?

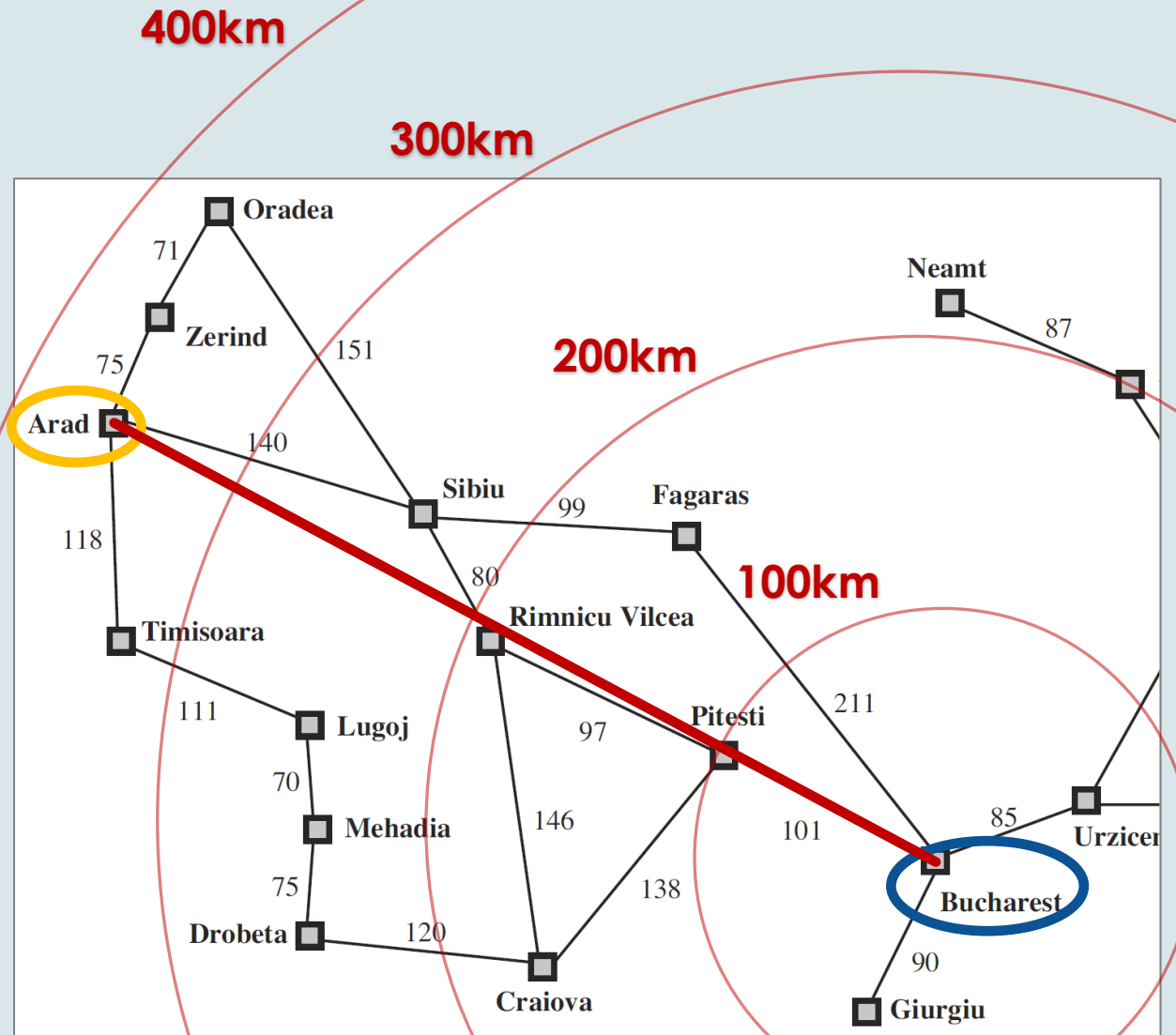
IT DEPENDS. If the algorithm had to choose a node that is farther away than another to find a solution path, then it may get stuck (*in tree search*)



Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal
- Initial State:
Arad-Bucharest: 365km

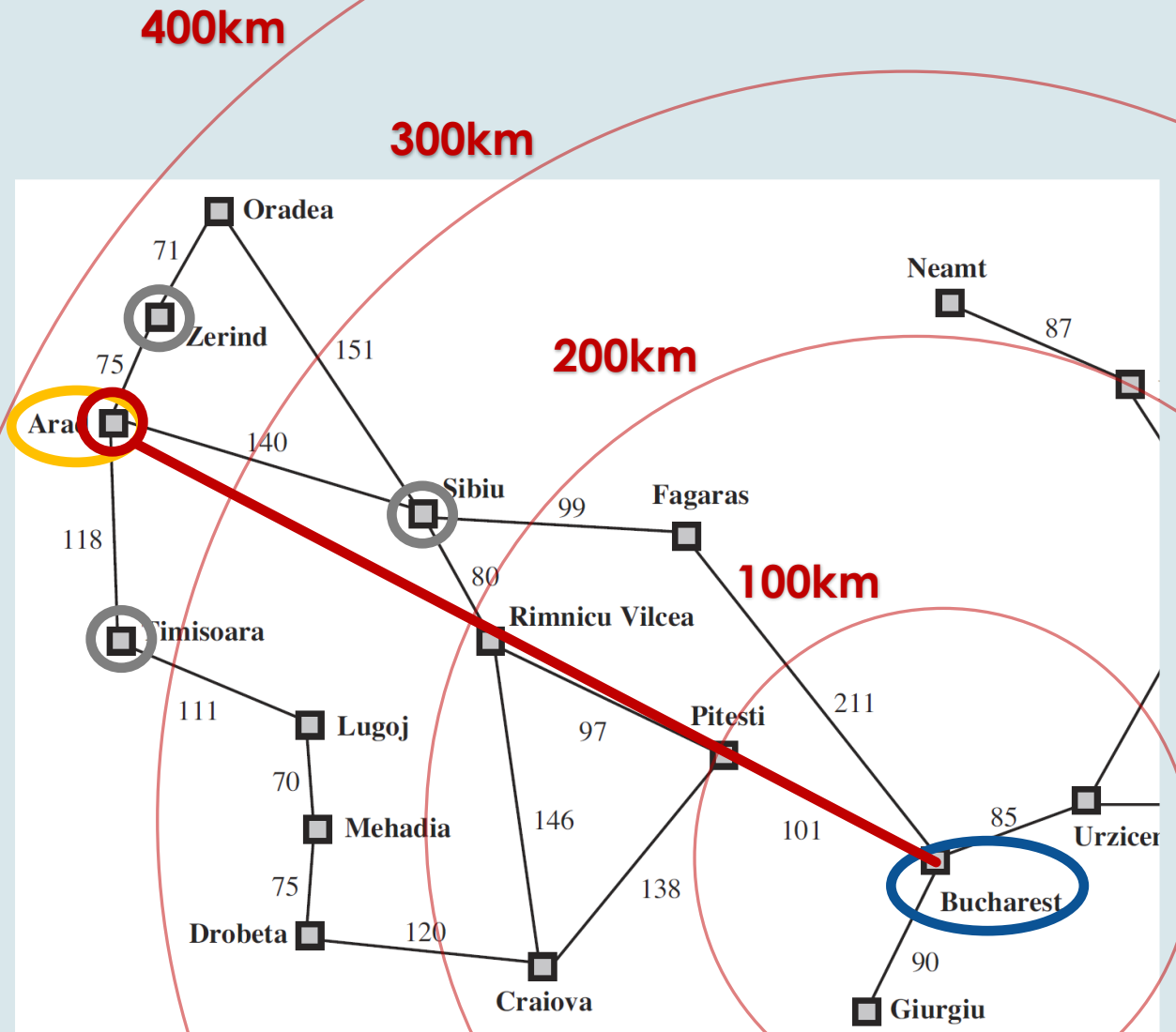


Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

- Sibiu: 140km + 240km
- Timisoara: 118km + 320km
- Zerind: 75km + 370km

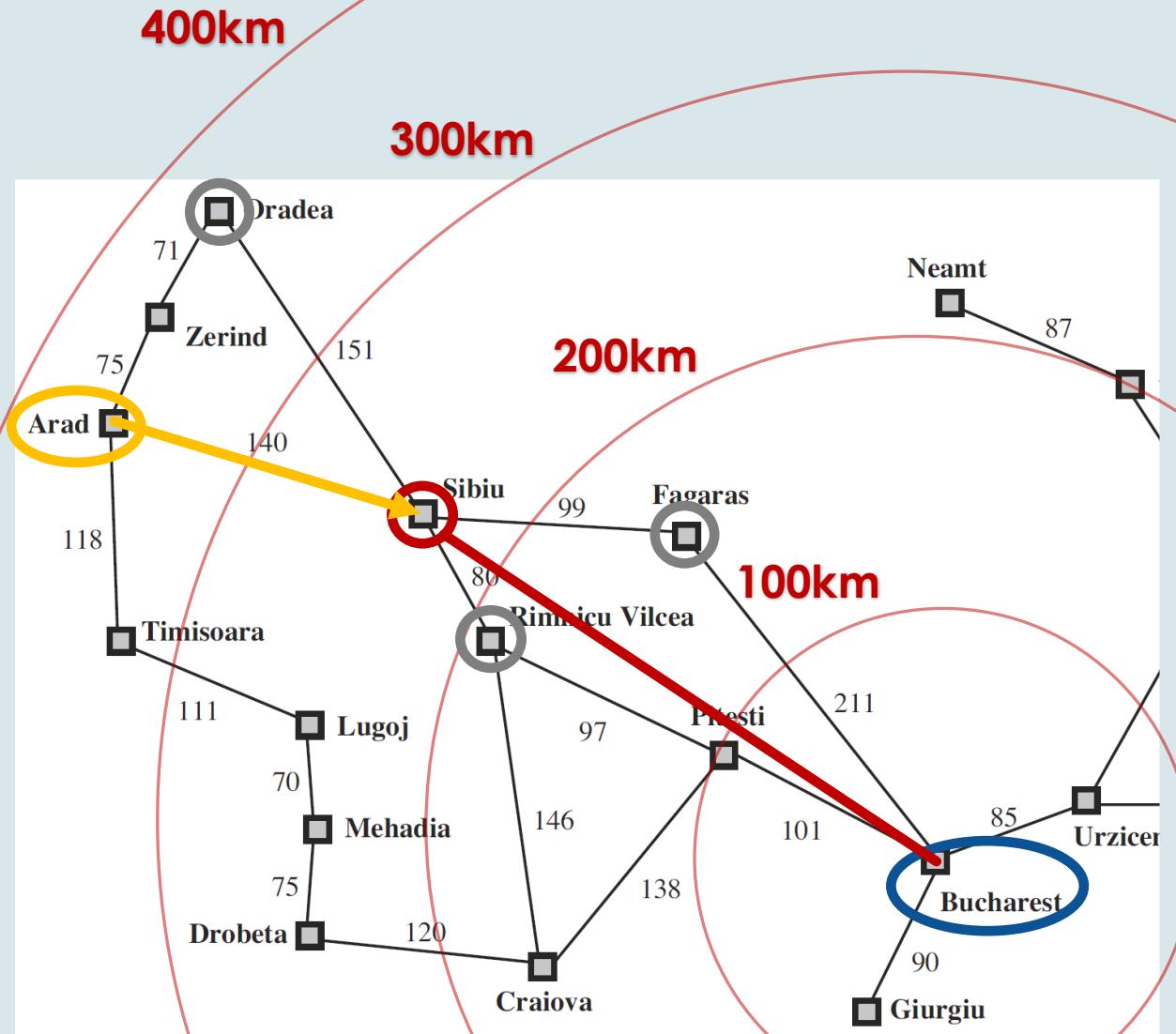


Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

- **Sibiu:** 380km
- Timisoara: 438km
- Zerind: 445km



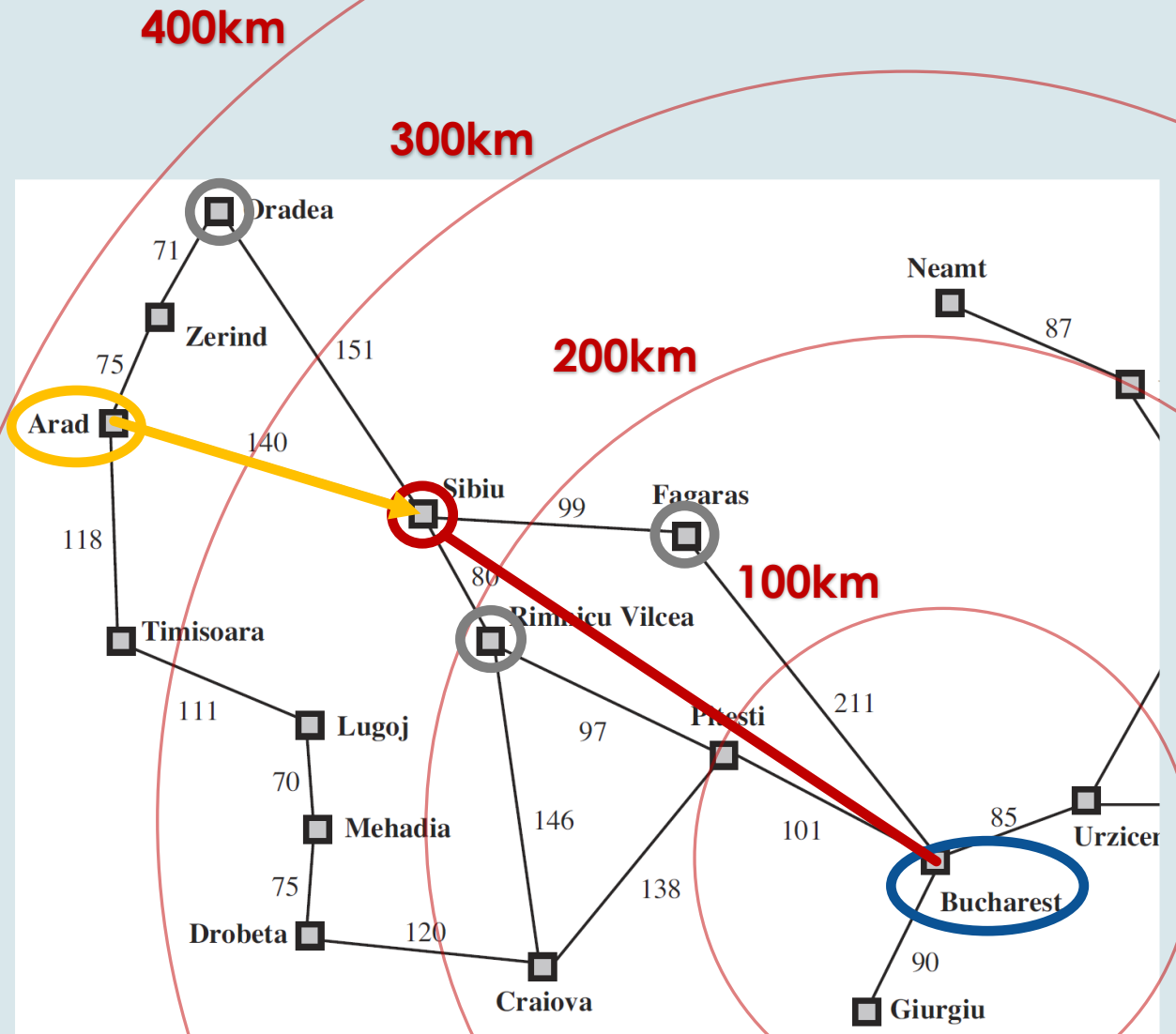
Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

- Fagaras: 239km + 175km
- Oradea: 291km + 380km
- Rimnicu: 220km + 190km

Timisoara: 438km, Zerind: 445km



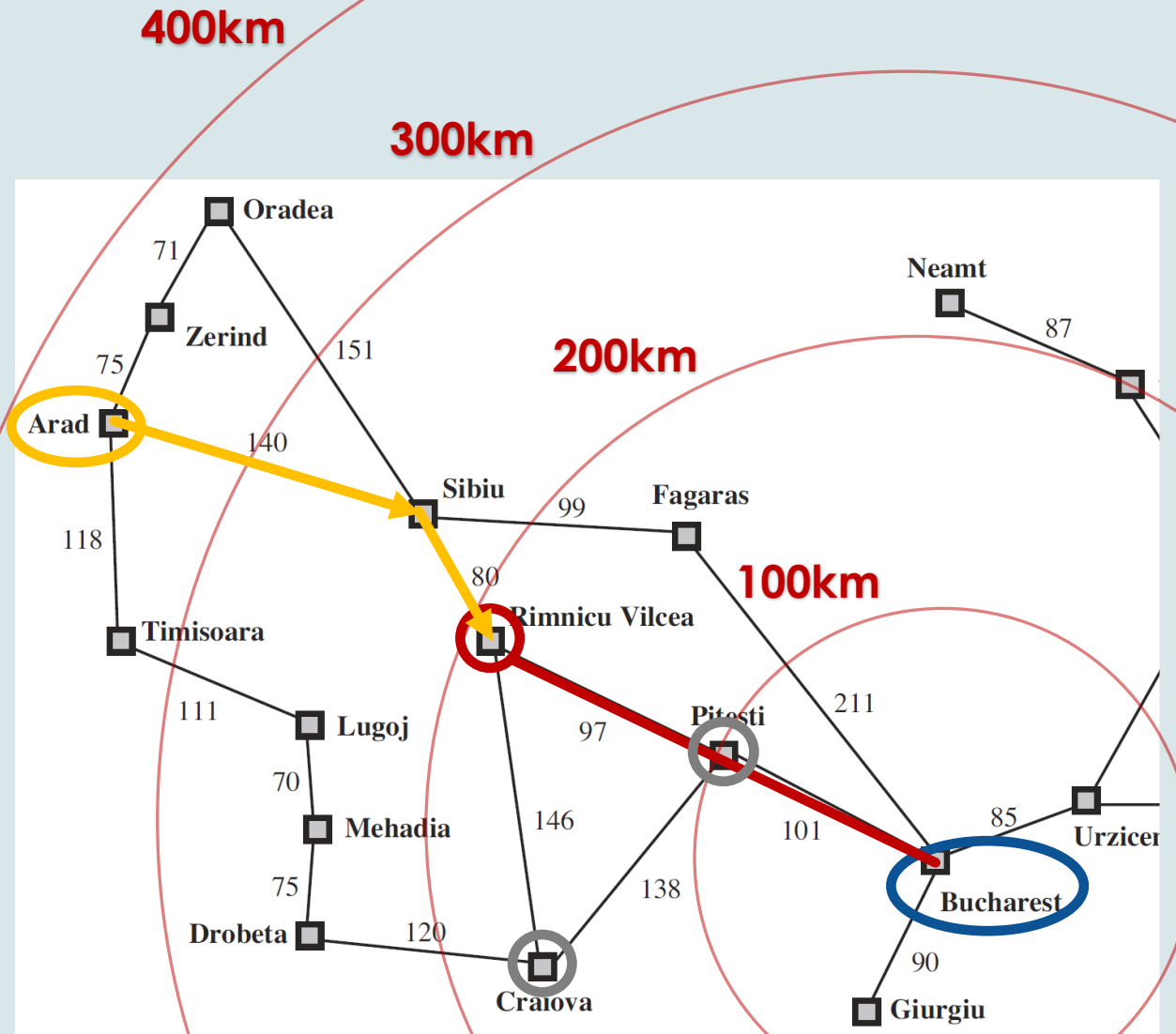
Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

- Fagaras: 414km
- Oradea: 671km
- **Rimnicu: 410km**

Timisoara: 438km, Zerind: 445km



Heuristic Search

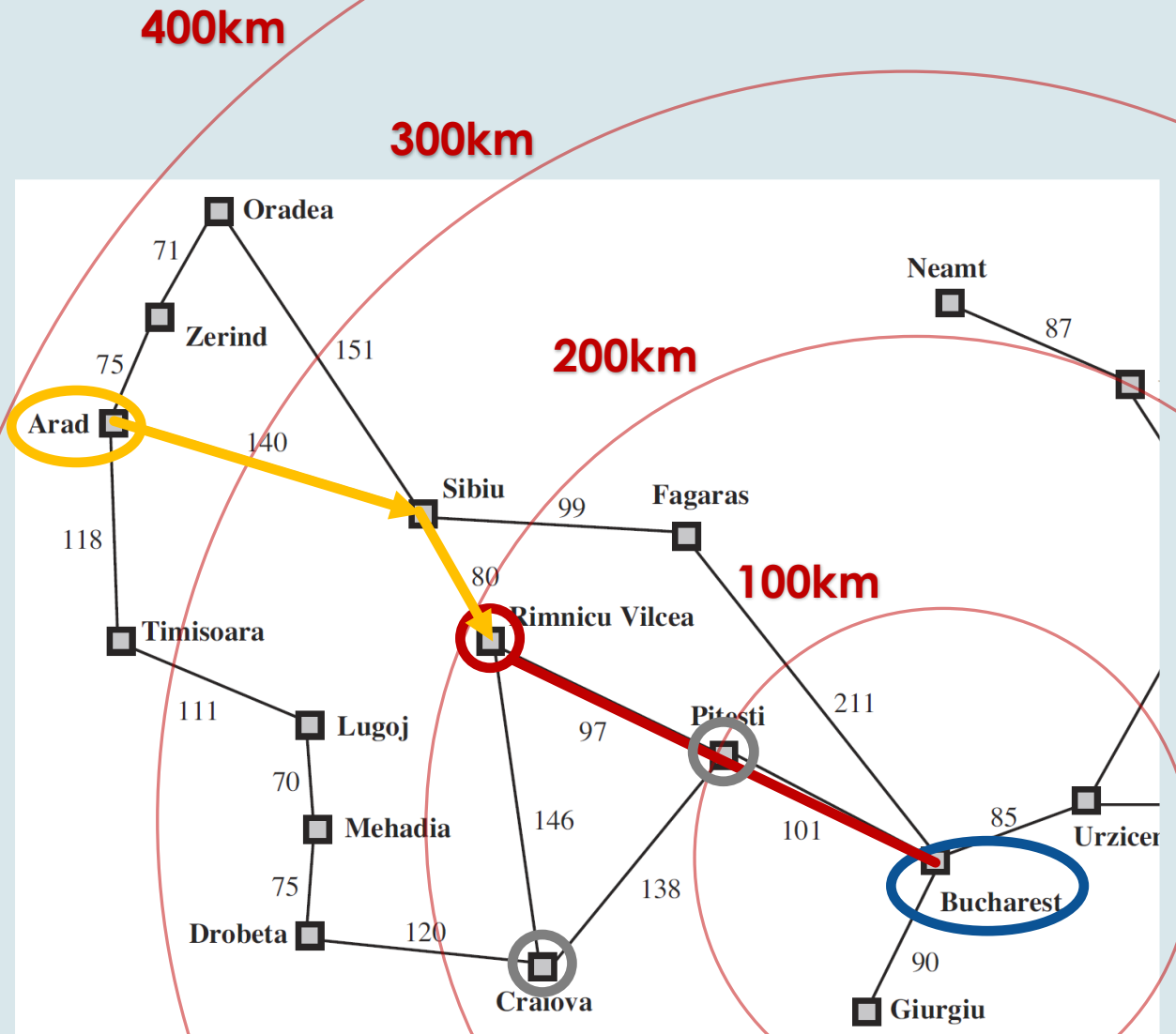
A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

○ Pitesti: 317km + 100km

○ Craiova: 366km + 180km

Timisoara: 438km, Zerind: 445km, Arad: 645km,
Fagaras: 414km, Oradea: 671km



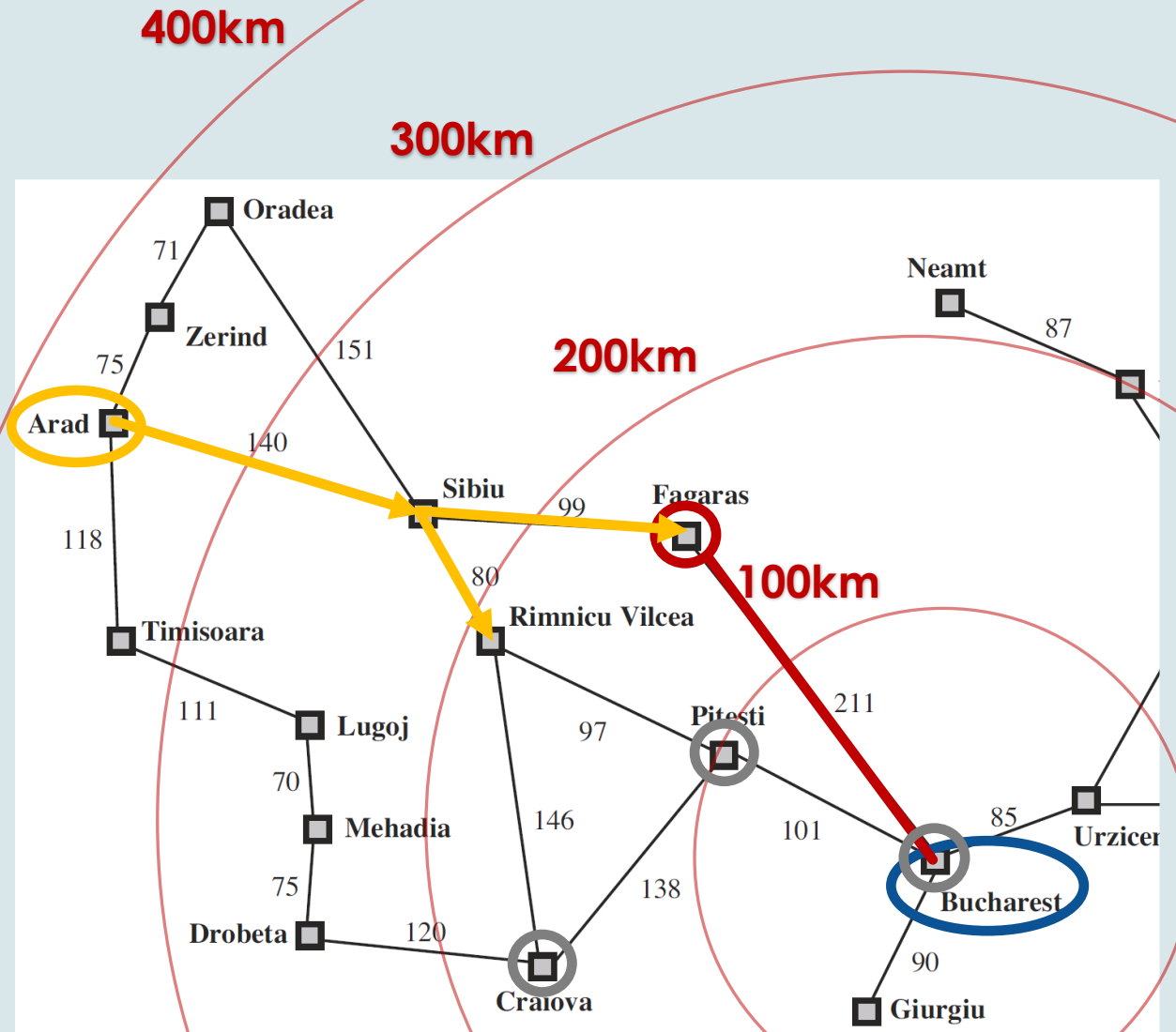
Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

- Pitesti: 417km
- Craiova: 546km
- Bucharest: $239 + 211 = 450\text{km}$

Timisoara: 438km, Zerind: 445km, Arad: 645km,
Fagaras: 414km, Oradea: 671km



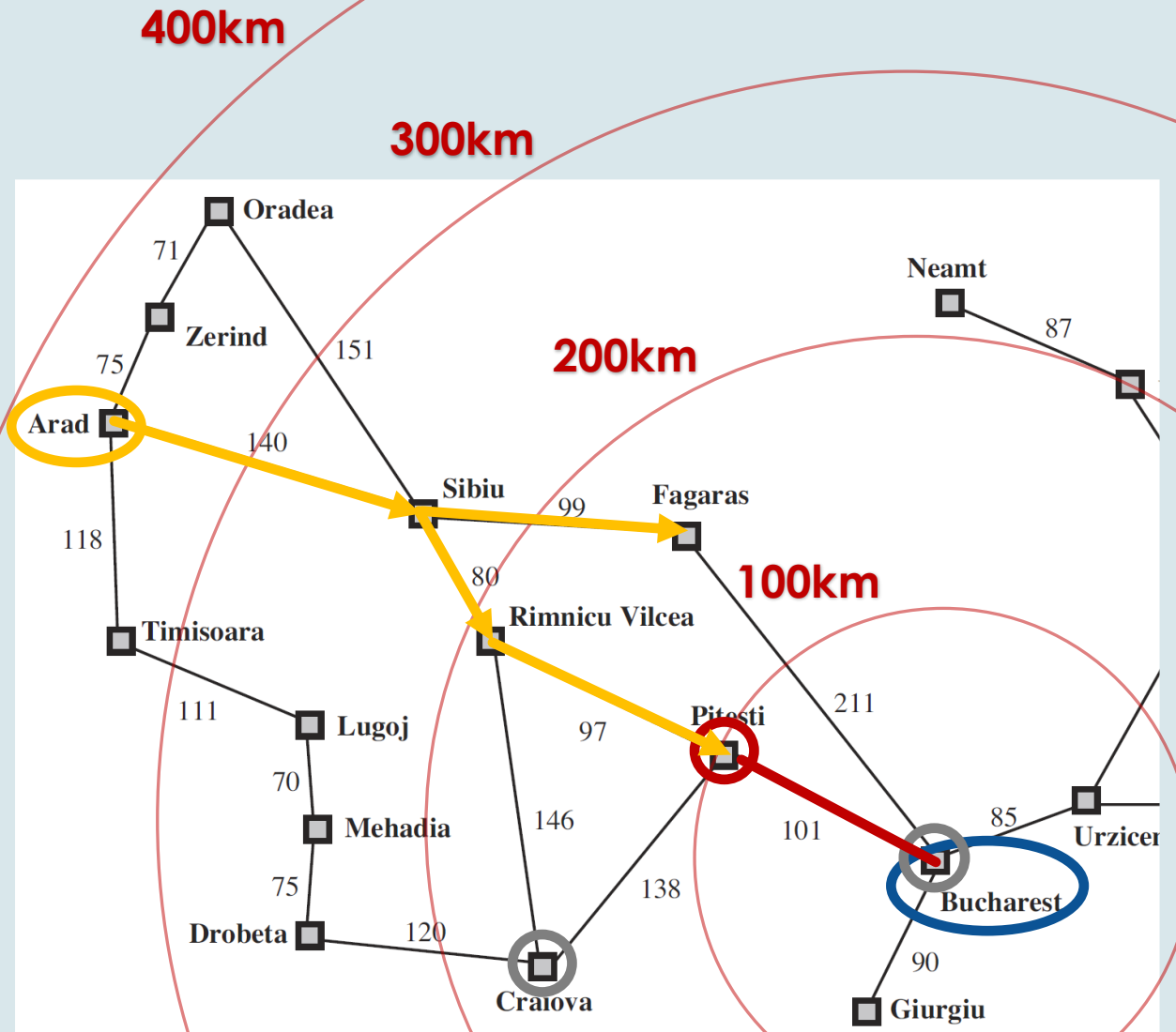
Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

- **Pitesti:** **417km**
- Craiova: 546km
- Bucharest: $239 + 211 = 450\text{km}$

Timisoara: 438km, Zerind: 445km, Arad: 645km,
Oradea: 671km, Bucharest: 450km



Heuristic Search

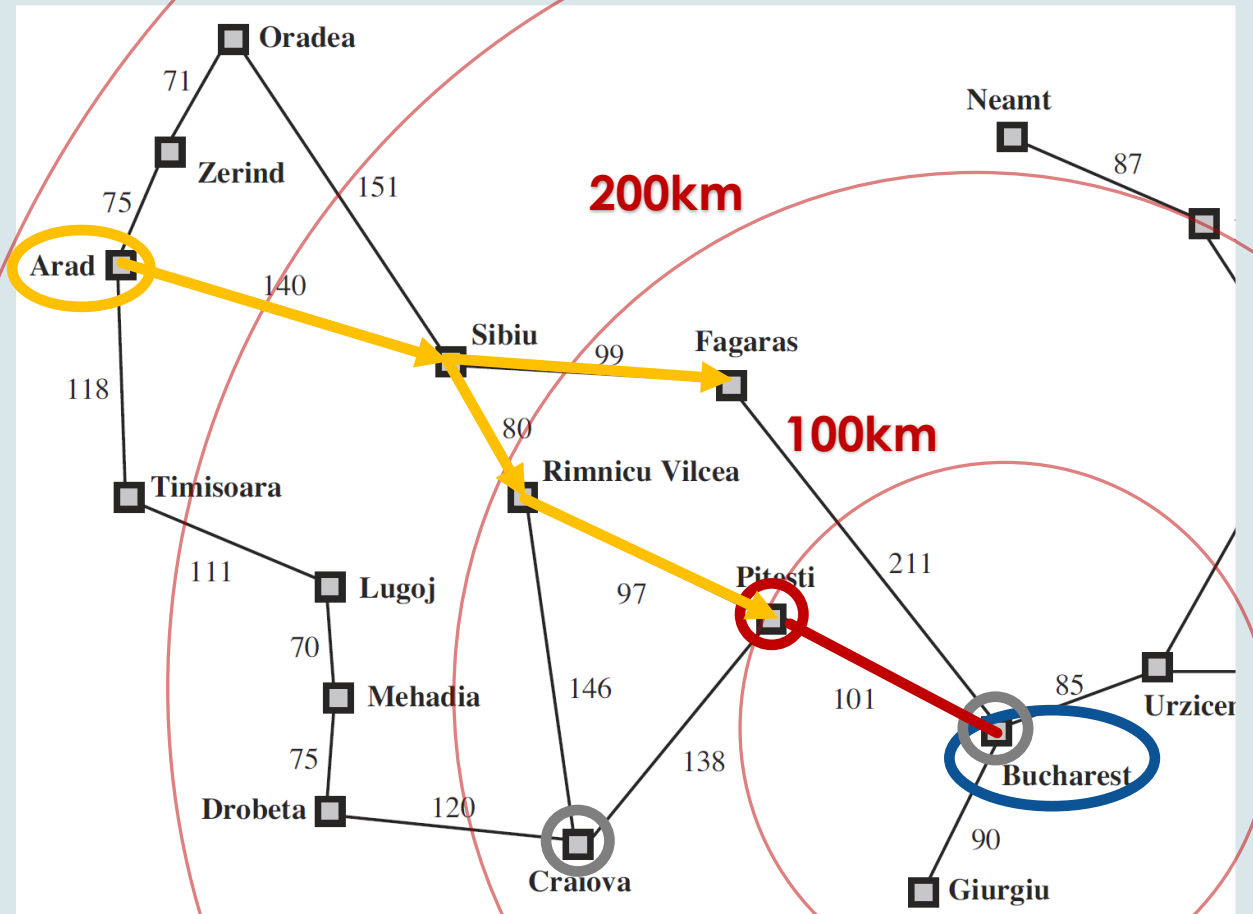
A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

○ Craiova: 455km + 180km

○ Bucharest: 418km + 0km

Timisoara: 438km, Zerind: 445km, Arad: 645km,
Oradea: 671km, Sibiu: 540km, Bucharest: 450km, Craiova: 546km



Heuristic Search

A* Search

- $f(n) = g(n) + h(n)$
- $g(n)$: accumulated path cost
- $h(n)$: estimated distance to goal

○ Craiova: 635km

○ **Bucharest: 418km**

Optimal?

Complete?

Timisoara: 438km, Zerind: 445km, Arad: 645km,
Oradea: 671km, Sibiu: 540km, Bucharest: 450km, Craiova: 546km

